

Wrocław University of Science and Technology
Faculty of Information and Communication Technology

Field of study: **Applied Computer Science (IST)**

Speciality: **Software Engineering (IOP)**

MASTER THESIS

**Evolutionary optimization of deep neural
networks**

Nina Bartnik

Supervisor

Associate Prof. Michał Przewoźniczek, PhD, DSc

Keywords: neural architecture search, multi-objective optimization, surrogate-assisted evolutionary algorithms, convolutional neural networks

ABSTRACT

Neural Architecture Search can design neural networks without manual trial and error. However, it can be costly. Judging a candidate normally includes training and evaluating it first. Methods that optimize accuracy alone tend to favour larger and more energy-consuming architectures. This thesis treats the task as a search for two objectives at once, classification accuracy and energy consumption. It also measures energy directly on the GPU that runs the network.

The search method adapts SA-P3-GOMEA, a surrogate assisted parameterless evolutionary algorithm developed for a single-objective ensemble learning problem, to the multi-objective problem. An architecture is encoded as six discrete variables, each selecting one of the five operations, which defines a search space of 15625 architectures from NAS-Bench-201 evaluated on CIFAR-10. A surrogate model predicts both objectives so that most candidates are filtered and filters most candidates out of real evaluation.

Three mechanisms extend the original algorithm. The surrogate can be probabilistic, so acceptance of the new solution is determined by the uncertainty of the surrogate. Instead of fixing one model, the algorithm can select the surrogate from the pool as the run proceeds. Moreover, the algorithm tunes the hyperparameters automatically.

The variants are compared against surrogate-free multi-objective SA-P3-GOMEA and multi-objective local search. The budget of real evaluations is set to 500. This thesis uses hypervolume and IGD+ as comparative measures.

On training energy the probabilistic surrogates and the online choice of surrogate reach the highest hypervolume, well above baselines, and a probabilistic surrogate has the lowest IGD+. The differences are small on inference energy, where the surrogates barely improve on the baselines.

CONTENTS

Abstract	1
List of abbreviations	4
Usage of AI	6
1. Introduction	7
2. Objectives	9
3. Scope	11
4. Related work	12
4.1. Introduction to artificial neural networks	12
4.2. Challenges of training deep neural networks	13
4.3. Evolutionary algorithms	15
4.3.1. Algorithm families	16
4.3.2. Linkage learning, optimal mixing, population pyramid	17
4.4. Evolutionary optimization of neural networks	20
4.4.1. Evolution of weights and hybrid methods	21
4.4.2. Neural architecture search	22
4.4.3. Data processing and AutoML	25
4.5. Multi-objective problems	25
4.5.1. Pareto front	26
4.5.2. EAs with multi-objective problems	28
4.5.3. Surrogates	31
4.5.4. EA-based SA optimization	32
4.6. Summary	35
5. The proposed Multi-objective SA-P3-GOMEA	36
5.1. NAS	36
5.2. Adapting SA-P3-GOMEA to multi-objective NAS problem	37
5.2.1. Objective 1 - accuracy	37
5.2.2. Objective 2 - usage of GPU power	37
5.3. Surrogates	38
5.4. Accepting new solution and dynamic threshold	39
5.5. Summary	39
6. Methods	41
6.1. Solution encoding	41

6.2.	Measuring power	42
6.2.1.	Convergence study of the measurement protocol	43
6.3.	Deterministic surrogates	43
6.4.	Surrogates with probabilistic distributions	45
6.4.1.	Probabilistic acceptance	46
6.4.2.	Threshold	47
6.4.3.	Dynamic surrogate selection and hyperparameter tuning	48
6.4.4.	Comparison	49
6.4.5.	Description of proposed sequential modification	50
7.	Experiments	51
7.1.	Experimental setup	51
7.1.1.	Research protocol	51
7.1.2.	Measures	51
7.1.3.	Stopping criteria	51
7.1.4.	Optimizers	51
7.2.	Overall comparison at $\eta = 0.999$	52
7.3.	MO-DSA-P3-GOMEA	53
7.3.1.	Sensitivity to the gate relaxation rate	55
7.3.2.	MO-DSA-P3-GOMEA with online surrogate selection	59
7.4.	MO-PSA-P3-GOMEA	61
7.4.1.	Sensitivity to the gate relaxation rate	63
7.4.2.	MO-PSA-P3-GOMEA with online surrogate selection	66
7.5.	Findings	68
8.	Conclusion	70
8.1.	Work	70
8.2.	Discussion	70
8.3.	Future work	72
	Bibliography	74
	List of Figures	82
	List of Algorithms	83
	List of Tables	84
	Appendix	86
A.	Full campaign report	87

LIST OF ABBREVIATIONS

AI	Artificial Intelligence
ANN	Artificial Neural Network
AutoML	Automated Machine Learning
AWS	Amazon Web Services
CMA-ES	Covariance Matrix Adaptation Evolution Strategy
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CUDA	Compute Unified Device Architecture
DL	Deep Learning
DNN	Deep Neural Network
DRAM	Dynamic Random-Access Memory
EA	Evolutionary Algorithm
EC	Evolutionary Computation
EDL	Evolutionary Optimization of Deep Learning
ENAS	Evolutionary-Algorithm-based Neural Architecture Search
EP	Evolutionary Programming
ES	Evolution Strategies
FIHC	First Improvement Hill Climber
FLOPs	Floating Point Operations
FOS	Family of Subsets
FPGA	Field-Programmable Gate Array
GA	Genetic Algorithm
GBoost	Gradient Boosting
GOM	Gene-pool Optimal Mixing
GOMEA	Gene-pool Optimal Mixing Evolutionary Algorithm
GP	Genetic Programming
GPU	Graphics Processing Unit
HV	Hypervolume
IGD	Inverted Generational Distance
IGD+	Modified Inverted Generational Distance
IoT	Internet of Things
IQR	Interquartile Range

JSON	JavaScript Object Notation
LLM	Large Language Model
LTGA	Linkage Tree Genetic Algorithm
ML	Machine Learning
MLP	Multi-Layer Perceptron
MO	Multi-objective
MO-DSA-P3-GOMEA	Multi-objective Deterministic-Surrogate-Assisted P3-GOMEA
MOEA	Multi-objective Evolutionary Algorithm
MOEA/D	Multi-objective Evolutionary Algorithm based on Decomposition
MO-GOMEA	Multi-objective GOMEA
MO-LS	Multi-objective Local Search
MO-P3-GOMEA	Multi-objective P3-GOMEA
MO-PSA-P3-GOMEA	Multi-objective Probabilistic-Surrogate-Assisted P3-GOMEA
MO-SA-P3-GOMEA	Multi-objective Surrogate-Assisted P3-GOMEA
NAS	Neural Architecture Search
NLP	Natural Language Processing
NN	Neural Network
NSGA-II	Non-dominated Sorting Genetic Algorithm II
NVML	NVIDIA Management Library
OM	Optimal Mixing
P3	Parameterless Population Pyramid
P3-GOMEA	Parameterless Population Pyramid GOMEA
PBI	Penalty-based Boundary Intersection
PSU	Power Supply Unit
RBF	Radial Basis Function
RF	Random Forest
ReLU	Rectified Linear Unit
RL	Reinforcement Learning
RQ	Research Question
SA	Surrogate-assisted
SA-P3-GOMEA	Surrogate-Assisted P3-GOMEA
SOTA	State of the Art
SVM	Support Vector Machine
SVR	Support Vector Regression
TPU	Tensor Processing Unit

USAGE OF AI

The author used an AI-based tool during the preparation of this thesis. The tool was Claude (Anthropic), used through the Claude Code interface, with the Claude Opus 4.8 model. Its scope of use is stated below.

- Language editing, including correction of grammar and spelling, unification of spelling conventions, and rephrasing of selected passages.
- Preparation of initial drafts of selected passages.
- Generation of result tables from data supplied by the author.
- Assistance with document formatting.
- Generation of mathematical formulas in LaTeX from references supplied by the author.
- Searching for and summarising references.
- Reworking the C++ implementation of the algorithm into its multi-objective variant.

AI tools were not used to produce or fabricate experimental data. All measurements and results come from the experiments described in this thesis. All AI-assisted output was verified by the author of this thesis.

1. INTRODUCTION

Artificial Neural Networks (ANNs) address many of today’s most complex global challenges, seeing widespread use in fields such as Natural Language Processing (NLP), reinforcement learning, and computer vision tasks like image classification. The concept of artificial neural networks started in 1943, when two American scientists, Warren McCulloch and Walter Pitts proposed a mathematical model of activity of neurons inside the brain [1]. This work led to the proposition of the perceptron. The perceptron was first described in 1958 by Frank Rosenblatt. The original perceptron’s output was a binary value of 0 or 1. Moreover, the first perceptron did not have hidden units. Multilayer perceptrons were proposed later in 1960. The real breakthrough started between 1960 and 1970 when the first deep learning papers were proposed and in 1979 Kunihiko Fukushima presented his idea of CNN architecture.

Nowadays, the field of Artificial Intelligence is dominated by the Transformer architecture, after it was presented in 2017 [2]. Transformer architecture is widely used in Large Language Models (LLMs). Unlike earlier sequential or purely convolutional models, modern networks use self-attention mechanisms to process vast amounts of data in parallel.

However, implementing any AI method can come with a significant cost. Multi-layer architectures are often highly demanding in terms of time, computational power and environmental impact. According to research conducted in 2022, generating one image uses as much electricity as fully charging a mobile phone. Two models solving the same problem can differ in energy demand by up to two orders of magnitude [3]. A follow-up study covering 88 models across 10 tasks states that inference energy varies among tasks. Task-specific models are cheaper to run than general-purpose ones on the same problem [4].

As models become deeper and more complex to handle difficult tasks, efficient optimization is a necessity. Optimization in neural networks spans from hyperparameter tuning (e.g., finding the right learning rate) and architectural search (defining the number of layers) to model compression (pruning and quantisation) [5, 6]. Among these, the design of the architecture itself dominates the total cost, because a poorly chosen architecture because is harder to salvage. The field of Neural Architecture Search (NAS) addresses this by automating architecture design [7, 8]. Yet, many NAS methods focus on a single objective,

usually validation accuracy [9, 10]. In practice, more accurate networks tend to be larger and more energy-consuming, so accuracy alone is an incomplete target.

Evolutionary algorithms (EAs) are a natural fit for this task. They operate on populations of candidate solutions, do not require gradient information and handle combinatorial problems well [11]. They are also well studied for multi-objective (MO) optimization, where the goal is to approximate the Pareto front of non-dominated solutions instead of one optimum [12, 13]. Multi-objective (MO) methods for NAS, including methods such as NSGA-Net [14] and LEMONADE [15], proved that EAs can search through accuracy and a resource-related second objective (floating point operations (FLOPs), parameters, latency) [16].

Each candidate architecture in the NAS search space must be trained from scratch on the target dataset before its objective value is known, and this evaluation cost dominates the overall search budget. Surrogate-assisted (SA) evolutionary optimization addresses this by replacing most real evaluations with predictions from a pre-trained model and reserving full training for solutions the surrogate believes are promising [17, 18, 19].

This thesis investigates the application of evolutionary methods to MO NAS for the image classification task, with the two optimization objectives being classification accuracy on the target dataset and the measured energy consumption for inference and training.

The algorithm is an adaptation of the SA parameterless variant of GOMEA (SA-P3-GOMEA), originally presented in 2021 by Dushatskiy [20]. This thesis also applies probabilistic surrogates in place of deterministic ones, which is a direction a reference paper mentioned as future work. The related GOMEA family has previously been applied to MO NAS with objectives being accuracy and FLOPs [21].

None of the surveyed works combines a surrogate, the parameterless P3 variant, and measure inference and training energy as a second objective. The contribution of this work is the combination of mentioned methods. The proposed algorithm further extends the original SA-P3-GOMEA with online selection of the surrogate variant, automated hyperparameter tuning of the surrogate model, and adaptation to probabilistic surrogates. Proposed algorithm is evaluated on CIFAR-10 image classification benchmark [22], uses NAS-Bench-201 [23] as a benchmark set and is compared against MO local search and MO GOMEA without surrogate, which are the MO adaptation of benchmarks used in reference study.

2. OBJECTIVES

The work is guided by two research questions.

1. RQ1: can a surrogate-assisted parameterless GOMEA approximate the Pareto front between accuracy and energy at a lower evaluation cost than the same evolutionary search without a surrogate?
2. RQ2: do probabilistic surrogates, dynamic surrogate-type selection, and automated hyperparameter tuning improve this result over a fixed deterministic surrogate?

The aim of this thesis is to design and experimentally evaluate an evolutionary algorithm for multi-objective Neural Architecture Search that optimizes the classification accuracy of a convolutional neural network together with its measured energy consumption, while keeping the number of full network trainings low through a surrogate model.

The algorithm is based on the SA-P3-GOMEA introduced for partition-based ensemble learning [20]. The original method optimizes a single objective with a deterministic surrogate. This work adapts it to a multi-objective NAS setting and extends its surrogate mechanism. The search space is taken from NAS-Bench-201 [23], and the second objective is energy measured on the hardware that runs the evaluation.

To reach this aim, the thesis addresses the following research steps:

1. Propose a multi-objective SA-P3-GOMEA (MO-SA-P3-GOMEA) for multi-objective NAS and encode each architecture using the proposed NAS-Bench-201 cell-based structure.
2. Define and integrate the second objective as energy consumption measured during training and inference.
3. Propose algorithms for obtaining and utilising the probabilistic surrogate. Introducing these mechanism into the proposed MO-SA-P3-GOMEA.
4. Add dynamic selection of the surrogate type and automated hyperparameter tuning during the run.
5. Evaluate the MO-SA-P3-GOMEA on the CIFAR-10 image classification benchmark, and compare it against multi-objective Local Search and multi-objective GOMEA (both adapted from state-of-the-art study).

The surveyed literature does not combine a surrogate, the parameterless P3 variant, and measured training and inference energy as a second objective. This thesis contributes:

- a multi-objective neural network architecture search that combines surrogate-assisted optimization, the parameterless pyramid operator, OM operator, and training and inference energy measured on the target hardware as the second objective;
- a probabilistic surrogate with a Pareto-dominance acceptance rule based on its predictive uncertainty;
- online selection of the surrogate type from a pool of candidates;
- automated tuning of the surrogate hyperparameters.

3. SCOPE

This thesis investigates the application of evolutionary methods to multi-objective NAS for image classification with the following two optimization objectives:

1. classification accuracy on the target dataset,
2. measured energy consumption of the network during inference and training.

The methodology backbone is a surrogate-assisted parameterless pyramid gene-pool optimal mixing evolutionary algorithm (SA-P3-GOMEA) [20]. This algorithm was originally applied to partition-based ensemble learning. The study also explores the possibilities of applying probabilistic surrogates and online selection of surrogate type.

Evaluation is performed on the CIFAR-10 [22] image classification benchmark. The proposed algorithm is compared against multi-objective GOMEA without the surrogate, and standard multi-objective Local Search. Both algorithms are based on implementations provided by reference paper [24].

4. RELATED WORK

This chapter focuses on reviewing the fundamentals for the proposed modification, including the basics of artificial neural networks, the cost of training models, multi-objective optimization, surrogates and more. The aim is not to cover any of these fields exhaustively, but to establish the state-of-the-art methods.

4.1. INTRODUCTION TO ARTIFICIAL NEURAL NETWORKS

Artificial Neural Networks (ANNs) are computational models inspired by biological neural networks found inside brains. ANNs, also known as Neural Networks (NNs), consist of connected layers, typically an input layer, hidden layers and an output layer [25, 26].

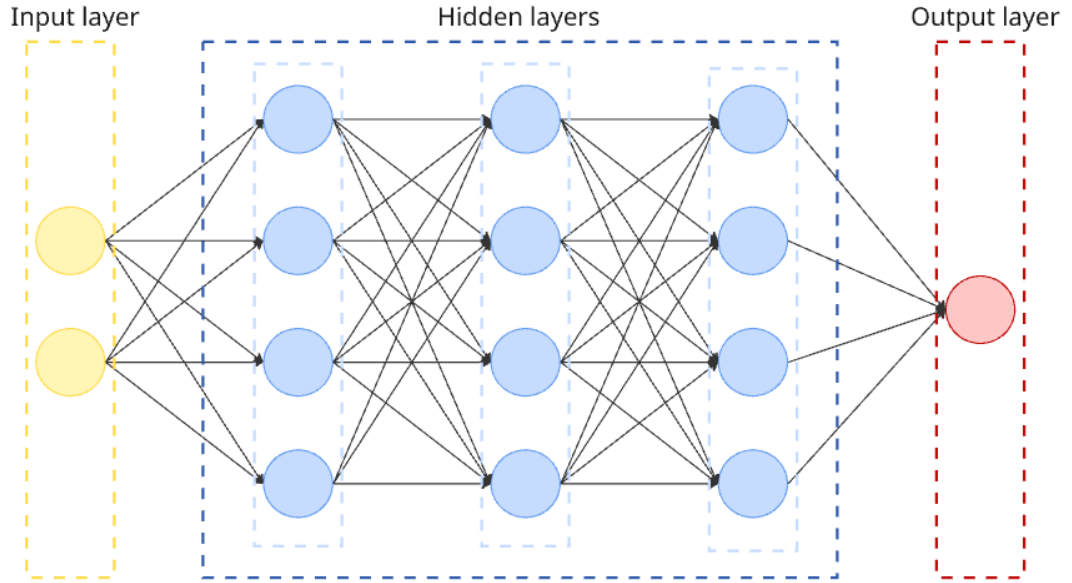


Fig. 4.1. Example of ANN layers. Own work.

Every layer is built from a number of units called perceptrons. A single perceptron computes a weighted sum of its inputs, adds a bias and passes the result through a nonlinear activation function. Inputs are represented by an array or vector of numbers. Formally, for inputs $x_1, \dots, x_n$ with associated weights $w_1, \dots, w_n$ and bias b , the output is

$$y = \sigma \left(\sum_{i=1}^n w_i x_i + b \right), \quad (4.1)$$

where $\sigma(\cdot)$ is the activation function.

The Rectified Linear Unit (ReLU) is a widely used function, mostly because of its cheap computational impact and reduction of the vanishing gradient problem that affects sigmoidal activations [27]. The mathematical representation of the ReLU function is presented below:

$$f(z) = \max(0, z), \quad (4.2)$$

The weights and biases of an ANN are learned from data. The training process involves iterative adjustment and correction of the weights by computing the network's prediction in a forward pass, measuring the difference between the prediction and the target by a loss function, and updating each parameter in the direction that reduces the loss [28].

NNs can consist of multiple layers of perceptrons, where every layer between input and output is called a hidden layer. This architecture is also called multi-layer perceptrons or Deep Neural Network (DNN). Deep learning (DL) is a subcategory of machine learning mainly focusing on models of this kind and their learning [25]. A special architecture of DNNs, where the input consists of a 2 or more dimensional vector, is called Convolutional Neural Network (CNN). CNNs are widely used in tasks such as image classification and segmentation, computer vision or language processing.

Image classification is a task of predicting the class label for an image. Standard benchmarks for evaluating CNN-based classifiers are CIFAR-10 and CIFAR-100 [22].

4.2. CHALLENGES OF TRAINING DEEP NEURAL NETWORKS

DNNs and CNNs remain essential in many practical applications. State-of-the-art use cases include computer vision (e.g., cameras and medical scanners), image classification (including medical tasks such as brain tumor recognition), image segmentation, object detection, and speech recognition [29]. However, using models in the real world is demanding. Among common challenges are complex DNN structures that require numerous parameters. Training of such deep models is resource intensive and time-consuming and comes with the requirement of significant computing power and large amounts of memory. Moreover, DNNs with many parameters are hard to deploy on end devices, often IoT devices, which come with tight limits on processing power and battery life [29, 30]. One of the key challenges in designing DNNs and CNNs is the balance between model accuracy and inference or training speed.

Wider and deeper architectures grow the parameter count exponentially, which directly increases memory usage and training time [31]. In addition, reusing or adapting trained

models for new tasks is equally costly, as practitioners regularly face poorly documented architectures and expensive retraining procedures [32].

Common memory reduction techniques for CNNs include knowledge distillation, using smaller batch sizes, mixed precision training or model quantisation. Computational complexity can be reduced by pointwise convolution and transposed convolution. When it comes to addressing the issue of balance between model accuracy and inference speed, model pruning, quantisation and low-rank approximation are standard methods among scientists [30].

The energy consumption of training and running neural networks is also worth considering. Researchers can measure for instance training or inference energy. Training energy is an energy consumed to train a neural network on a dataset. Inference is the stage of the model life cycle where a trained network is run on new inputs to produce outputs, as opposed to training, the stage in which the network’s parameters are learned. Luccioni et al. [4] treat inference as the deployment stage. In this study, inference energy consumption was measured across 88 models covering 10 tasks and 5 modalities, with all experiments run on NVIDIA A100 GPUs hosted on AWS. It varied by a factor of over 1450 across tasks. Text classification consumed about 0.002 kWh per 1000 queries, while image generation reached 2.907 kWh for the same number. Image classification fell in between at around 0.007 kWh per 1000 inferences. Finetuned models were far cheaper to run than general-purpose generative ones on the same problem. A finetuned encoder emitted approximately 0.3 grams of CO₂eq per 1000 extractive question-answering inferences, versus about 10 grams for a multi-purpose decoder on the same dataset.

The study also compared inference costs against training costs. Table 4.2 shows this comparison for the BLOOMz model family, combining publicly available training logs with per-query inference measurements. Training a 7-billion-parameter model consumed over 51000 kWh; finetuning added another 7500 kWh. For services handling millions of requests per day, that threshold is crossed within weeks.

Model	Training (kWh)	Finetuning (kWh)	Inference (kWh)	Cost parity (#)
BLOOMz-7B	51686	7571	1.0×10^{-4}	5.93×10^8
BLOOMz-3B	25634	3242	7.3×10^{-5}	3.96×10^8
BLOOMz-1B	17052	1081	6.2×10^{-5}	2.92×10^8
BLOOMz-560M	10505	543	5.4×10^{-5}	2.05×10^8

Table 4.1. Training, finetuning, and inference energy for the BLOOMz model family. Cost parity shows how many inferences are needed to equal the combined training and finetuning energy. Adapted from Luccioni et al. [4].

To overcome the issue of high-demanding architectures and the resource-intensive training

process, the field of architecture optimization is constantly developing. This optimization of deep neural networks can take many different approaches such as parameter pruning, network quantisation, dataset or fitness function optimization [5].

A promising approach is Neural Architecture Search (NAS), which automates architecture design by evaluating many candidates and selecting the best. The bottleneck is that each candidate typically requires a full training run, the most expensive stage of the model lifecycle. Surrogate-assisted optimization addresses this by replacing most real evaluations with cheaper predictions, saving both time and energy. The efficiency gap between task-specific and general-purpose models also suggests that searching for a task-optimized architecture is more practical than relying on oversized general-purpose networks. Among methods used to optimize and prune networks, evolutionary algorithms are one of the leading techniques [6].

4.3. EVOLUTIONARY ALGORITHMS

Evolutionary algorithms (EAs) are algorithms that mimic genetic processes found in nature to solve demanding problems [11]. Evolutionary computation include algorithms that belong to the family of methods inspired directly by the process of evolution. The range of nature inspired algorithms is wide and includes algorithms based on the behaviour of colony animals like bees or ants [33, 34]. Note, that many animal-based and metaphor-based optimizers are currently under heavy criticism [35, 36].

Concept	Description
Individual	A single candidate solution within the population, typically represented as a chromosome or genotype.
Population	A set of potential solutions (individuals) currently being maintained by the algorithm.
Selection	The process of choosing individuals from the current population to act as parents for the next generation, usually based on their fitness.
Reproduction	The overall process of creating a new generation of individuals from the selected parents.
Recombination	The process of combining parts of two or more parent individuals to create new genetic material.
Crossover	A specific type of recombination where genetic material is swapped between parents at defined points to create offspring.
Mutation	A genetic operator that introduces random, small changes to an individual's genotype to maintain genetic diversity.
Offspring	The new individuals created through the processes of recombination, crossover, and mutation.

Table 4.2. Core Concepts of EAs

Evolutionary algorithms utilise the process of collective learning performed by a population of individuals (see Algorithm 1). The general workflow of an evolutionary algorithm starts with a randomly generated population of individuals. The population is evaluated by a fitness function, and the algorithm enters its main loop. On each iteration, the best individuals found so far are subjected to operations such as mutation and crossover. The termination criterion varies from a fixed number of iterations to more advanced hyper-parameter functions [11].

Algorithm 1 General workflow of an evolutionary algorithm

```

1:  $P \leftarrow$  a randomly generated population of individuals
2: evaluate every individual in  $P$  with the fitness function
3: while the termination criterion is not met do
4:   select the best individuals of  $P$  as parents
5:   create offspring by applying crossover and mutation to the parents
6:   evaluate the offspring with the fitness function
7:    $P \leftarrow$  the next population formed from  $P$  and the offspring
8: end while
9: return the best individual found

```

4.3.1. Algorithm families

A classical example of an evolutionary algorithm is a genetic algorithm (GA) [37]. GAs are part of a family of metaheuristics and they take inspiration from the process of natural selection. In GAs, the process of evolution is aiming to find as good as possible (preferably optimal) solution on the considered optimization problem. The evolution is an iterative process and starts with an initial population of individuals. The size of the population depends on the problem. A typical population is of fixed size. However, modern GAs often rely on parameter-less approaches, where the size of the population is not defined strictly [38, 39]. Every iteration is called a generation. Every generation is evaluated in terms of solving the given task by calculating the fitness function for each individual. The fitness function measures the quality of each generation and is the main indicator for comparing the quality of individuals.

GAs are only one branch of a broader family. EAs can be grouped into genetic algorithms (GAs), evolution strategies (ES), genetic programming (GP), and evolutionary programming (EP) [40]. GAs apply selection, crossover and mutation to strings that are usually of fixed length. Another type, ES, relies on mutation of vectors and self-adaptive mutation. In this case each individual encodes the standard deviations of its own Gaussian mutations and these evolve together with the solution. Covariance Matrix Adaptation Evolution Strategy (CMA-ES) [41] is a well-known variant that adapts the covariance matrix of its search distribution and has been used both to optimize hyper-parameters of deep networks [42] and to train deep neural network policies directly [43]. Evolutionary programming was first

designed to evolve finite-state machines and uses only mutation, with no recombination. Finally, GP uses a tree-like representation and evolves the structure of a program, which is why it appears in works that evolve full network topologies, such as Cartesian Genetic Programming for CNNs [44].

4.3.2. Linkage learning, optimal mixing, population pyramid

Linkage Tree Genetic Algorithm (LTGA) is a GA that identifies dependencies between variables and utilises this knowledge to improve its effectiveness and efficiency. The algorithm discovers which genes are codependent and should be inherited together [45]. Two variables x_1 and x_2 have dependency declared as a measure of distance:

$$D(x_1, x_2) = \frac{H(x_1, x_2) - I(x_1, x_2)}{H(x_1, x_2)}, \quad 0 \leq D(x_1, x_2) \leq 1 \quad (4.3)$$

where $I(x_1, x_2)$ is the *mutual information* between the two variables and $H(x_1, x_2)$ their joint entropy, defined as

$$H(x_1, x_2) = - \sum_{a \in \Omega_1} \sum_{b \in \Omega_2} p(a, b) \ln p(a, b). \quad (4.4)$$

The mutual information I (Equation 4.5) says how much knowing one variable reduces the uncertainty about the other. It is never negative, it equals zero when the two variables are independent ($p(a, b) = p(a)p(b)$), and grows as they become more dependent [46, 47].

$$I(x_i, x_j) = \sum_{a \in \Omega_i} \sum_{b \in \Omega_j} p(a, b) \ln \frac{p(a, b)}{p(a)p(b)} = H(x_i) + H(x_j) - H(x_i, x_j) \quad (4.5)$$

These pairwise dependence values must be organised into a representation that the algorithm can exploit. One such representation is a dependency structure matrix, a square matrix that records the pairwise interactions between the components of a system [48]. Each component is assigned one row and the corresponding column, and the diagonal has no interpretation. The matrix originates in project management and organisational design, where an off-diagonal entry is a single bit that equals one when two tasks or product components depend on one another and zero otherwise. In a genetic algorithm the components are the problem variables, and each entry represents the strength of the dependence. The mutual information of Equation 4.5 quantifies that strength, and LTGA converts it into the normalised distance of Equation 4.3, a value between zero and one that decreases as the dependence between two variables increases. The probabilities required by these formulas are not known in advance. The algorithm estimates them by counting how often each value and each pair of values occurs among the individuals used to construct

the matrix, so that $p(a)$ denotes the fraction of those individuals in which variable x_i takes value a , and $p(a, b)$ the fraction in which the pair (x_i, x_j) takes (a, b) . The matrix therefore reflects the dependence structure of the current population and is recomputed whenever that population changes.

LTGA organises the same information using hierarchical cluster tree of the problem variables. The algorithm rebuilds this tree in every generation from a subset of individuals chosen by tournament selection with size $s = 2$. It computes the frequencies above over this subset alone and uses the subset only to build the tree, while the core evolutionary process continues over the entire population. The construction begins with each problem variable in its own cluster, and the two closest clusters are merged recursively until a single cluster remains. For a problem with n genes, the resulting tree has n leaf nodes and $n - 1$ internal nodes, and every node identifies a group of variables and divides it into two mutually exclusive linkage sets.

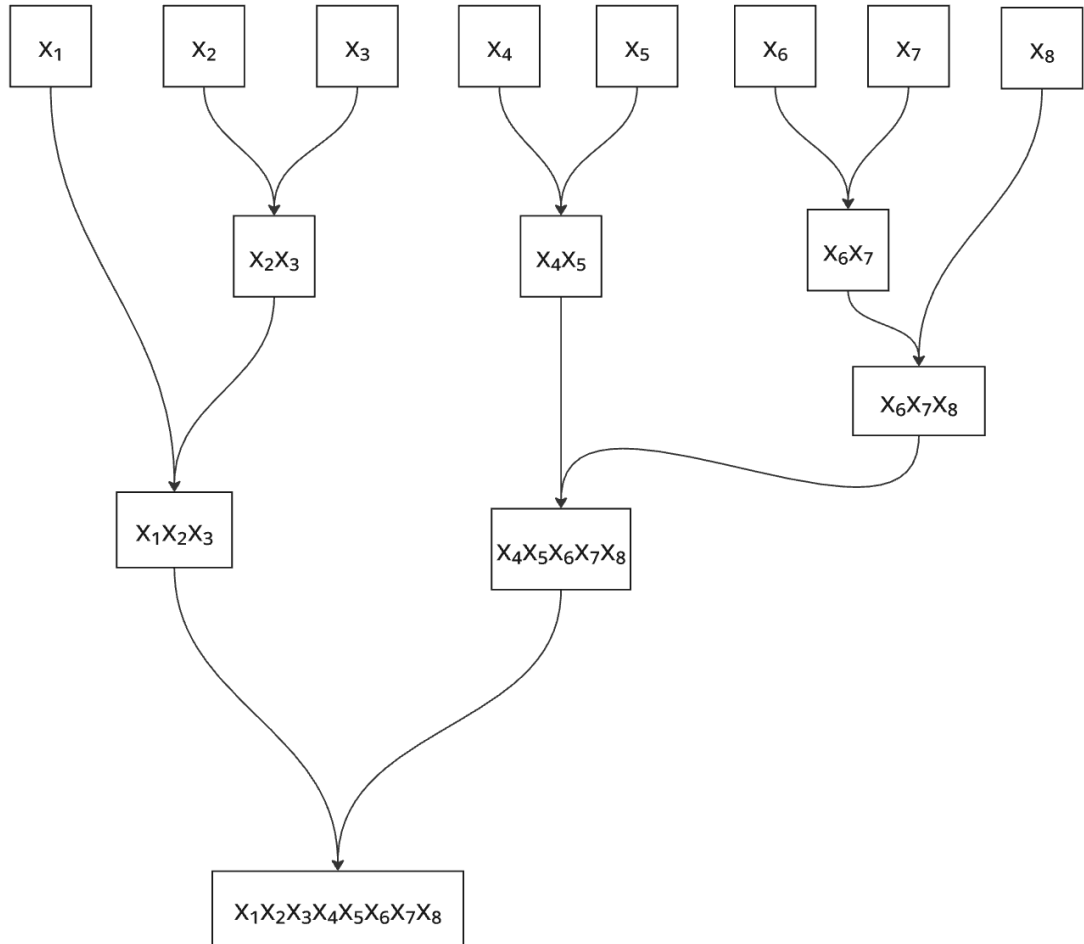


Fig. 4.2. Simplified representation of Linkage Tree. Each node represents gene. Genes are grouped into co-dependent clusters. Own work.

In LTGA, crossover, mutation, and selection are replaced by *Optimal Mixing* (OM) in-

roduced by Thierens and Bosman in 2011 [49]. Unlike the regular, GA-like crossover, OM evaluates partial changes as it makes them. In this operator, a random solution from the current population acts as a donor and its variable values for a specific linkage set are copied into the optimized solution denoted as *source*. Only the source is modified during OM. If the modification does not deteriorate the source’s fitness, then it is preserved. Otherwise it is rejected. The linkage tree decides which variables are copied together. New solutions are generated by greedily exploring the neighbourhood of each individual, where the structure of the linkage tree defines the variables clusters that are copied together. LTGA applies these clusters from the smallest to the largest. A modification is only accepted if the resulting offspring has a better or equal fitness value than the original parent [45].

The operator comes in recombinative and gene-pool variants. Recombinative OM uses one donor for the whole solution. Gene-pool OM (GOM) picks a new donor for each linkage set, so the copied values come from many solutions [49] (see Algorithm 2).

Algorithm 2 Gene-pool Optimal Mixing of one solution

```

1:  $o \leftarrow$  a copy of the parent solution
2: for all linkage sets  $F$  of the linkage tree do
3:   pick a random donor  $d$  from the population
4:    $o' \leftarrow o$  with the variables in  $F$  set to the donor’s values  $d_F$ 
5:   if  $o'$  differs from  $o$  then
6:     evaluate  $o'$ 
7:     if  $o'$  is not worse than  $o$  then
8:        $o \leftarrow o'$  ▷ accept the donated block
9:     end if
10:  end if
11: end for
12: return  $o$ 

```

LTGA was a precursor for Parameter-less Population Pyramid (P3). P3 is a concept originally presented in 2014 by Brian W. Goldman [39]. P3 modifies the concept of predefined size of population and empirical search for population size. It is based on pyramid structure of populations. Each level of the pyramid consists of sets of unique solutions with higher levels storing solutions that were improved a higher number of times and should fit better for the considered problem. In addition to keeping the populations, each level also stores the table of pairs (value of gene, frequency). Initial value of the frequencies is set to 0 and the table evolves when new individuals are added to the pyramid. Algorithm 3 presents one iteration of P3.

P3 is additionally improved by the local search mechanism called First Improvement Hill Climber (FIHC), shown in Algorithm 4. FIHC checks the variables in random order and

Algorithm 3 One iteration of P3

```
1:  $s \leftarrow$  a new random solution
2: improve  $s$  with FIHC ▷ Algorithm 4
3: if  $s \notin \text{hashset}$  then
4:   add  $s$  to level  $P_0$  and to hashset
5: end if
6: for all levels  $P_i$  of the pyramid, from the bottom up do
7:   mix  $s$  with the solutions of  $P_i$  ▷ Algorithm 5
8:   if the fitness of  $s$  improved and  $s \notin \text{hashset}$  then
9:     add  $s$  to level  $P_{i+1}$  and to hashset
10:  end if
11: end for
```

takes the first value for each of them that improves the fitness. This process is repeated until a single change does not generate a better solution.

Algorithm 4 First Improvement Hill Climber (FIHC)

```
1: repeat
2:   for all variables  $x_i$  of  $s$ , in random order do
3:     for all values  $v$  in the domain of  $x_i$  do
4:       if setting  $x_i \leftarrow v$  strictly improves the fitness of  $s$  then
5:         set  $x_i \leftarrow v$  and continue with the next variable
6:       end if
7:     end for
8:   end for
9: until no single-variable change improves  $s$ 
```

The evolution of the pyramid consists of checking the gene values for all pairs of positions. After that the table is updated by increasing the count for each observed pair [39].

At each level, P3 improves a solution with the GOM operator described above. Every level builds its own dependency matrix from its stored frequencies. From that matrix it constructs its own linkage tree, using the distance of Equation 4.3. Mixing applies the subsets of that tree to the solution smallest first, takes subsets of equal size in random order, and ignores the subsets of size one. For each subset a donor is drawn at random from the level's subpopulation, its values on the subset are copied into the solution, and the change is kept only when it does not lower the fitness (see Algorithm 5).

4.4. EVOLUTIONARY OPTIMIZATION OF NEURAL NETWORKS

In terms of evolutionary optimization of neural networks, EAs represent a promising approach to effectively searching a wide space and finding acceptable solutions. Broadly speaking, evolutionary optimization of deep learning (EDL) covers all methods of improving

Algorithm 5 Mixing solution s with level P_i

```
1: for all linkage subsets  $C$  of the level's tree, smallest first do
2:   for all donors  $d \in P_i$ , in random order do
3:     if  $d$  differs from  $s$  on at least one position of  $C$  then
4:        $f \leftarrow$  fitness of  $s$ 
5:       copy the values of  $d$  on the positions of  $C$  into  $s$ , and evaluate  $s$ 
6:       if the fitness of  $s$  is worse than  $f$  then
7:         revert the change
8:       end if
9:       break ▷ continue with the next subset
10:    end if
11:  end for
12: end for
```

DL models using EAs and evolutionary computation (EC). A 2022 survey [50] indicates that publications in EDL have increased threefold between 2010 and 2020, with most research focusing on the joint optimization of architecture and hyperparameters and an emphasis on CNNs. The most frequently discussed techniques are genetic algorithms and EAs more broadly.

Ma and Xie [51] divide evolutionary optimization of ANN into evolution of connection weights, evolution of architecture, and evolution of learning rules. The remainder of this section follows the first two axes, which are most relevant for this thesis.

4.4.1. Evolution of weights and hybrid methods

One of the optimization factors is the evolution of connection weights inside the network topology, which generally includes maximising network prediction accuracy. In this regard, EAs offer particular advantages over gradient-descent methods. Gradient-descent algorithms used to be the main technique when it comes to optimizing network weights. However, they are prone to be stuck in local optima. EAs on the other hand are more explorative. A modern optimization technique is a combination of both evolutionary algorithms for global search and gradient descent methods to find the best solution in the local neighbourhood. The state-of-the-art examples of so-called hybrid algorithms are NEAT [52] and CoDeepNEAT [53]. It is worth noticing that there are multiple strategies of encoding problem features. Direct encoding is a strategy where connections and nodes of ANNs are represented as a single gene, while indirect encoding is a technique where only specified features are encoded as genes. The direct encoding causes a massive search space and every weight must be evolved individually. Indirect encoding evolves a smaller mapping and is tractable for DNNs [51]. An interesting approach is presented by EvoCNN [54]. EvoCNN uses a GA to evolve both the architecture and the initial connection weight

distribution of a CNN. Thanks to variable-length gene encoding, network depth is treated as a search variable.

4.4.2. Neural architecture search

Neural architecture search (NAS) was originally proposed in 2017 [9]. It started as a gradient-based technique for finding neural network architectures from scratch. Through time, the idea of automated ANN design became an area of exploration for many researchers and evolved. Elsken, Metzen and Hutter (2019) [7] split NAS into three parts: the search space, the search strategy and the performance estimation strategy. The search space is the set of architectures the method can build, meaning which operations are available and how they connect. A space narrowed by prior knowledge is easier to search but may exclude good architectures. The search strategy chooses how to move through that space, balancing exploration and exploitation. Performance estimation scores a candidate without training it in full, for example by training it briefly or predicting its accuracy. Among dominant paradigms of NAS methodologies, Baymurzina, Golikov and Burtsev [8] include neuroevolution, which utilises mechanisms of EAs in designing well-fitted ANN architectures within a given task. The study also mentions that genetic operators tend to move candidate architectures into regions of the search space with higher fitness values.

NAS methods can be also classified into gradient-descent-based, reinforcement learning-based (RL-based) and EA-based [10]. The remainder of this section focuses on EA-based NAS, further referred to as ENAS. A 2023 survey on EA-based NAS [16] states that the number of publications on this topic increased 3 times in the span of 2017-2020, since LargeEvo [55] was presented. RL-based NAS often requires a huge amount of GPU-days and gradient-based methods require supernet, while EAs do not need any of that. The survey also divides EA-based NAS from neuroevolution, where neuroevolution searches for architecture and weights and ENAS focuses on architecture only with weights optimized by gradient.

4.4.2.1. Search space and encoding

Search spaces range from simple to highly structured, and the simplest is the chain-structured one. Here an architecture is a sequence of n layers in which each layer takes its input from the previous layer and passes its output to the next. The search space is the set of all such sequences, and it is defined by the number of layers, the operation available to each layer, and the hyperparameters of those operations.

NAS allows other operators to be used when designing the architecture, such as skip connections, summing previous layers (Residual Networks [56]) or concatenating previous layers (DenseNet [57]). The more general approach is searching for whole blocks of

cells or repetitive motifs instead of the whole architecture. A cell can be described as a computational graph smaller in size than a network. This approach was originally presented by Zoph et al. in 2018 [58]. The cells were divided into two kinds, normal and reduction cells, and the architecture was built by stacking them in a predefined pattern. Normal cells preserved the spatial dimensions of their input, while reduction cells reduced them.

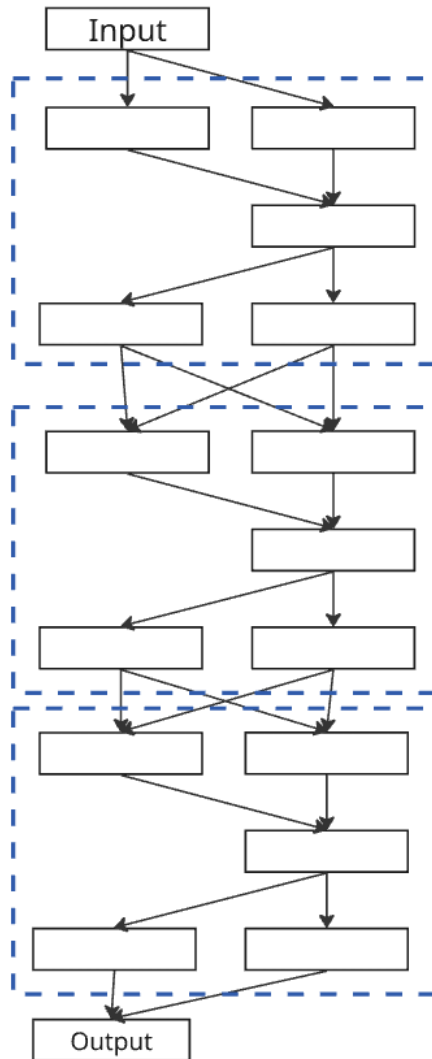


Fig. 4.3. Example of cell-based search space. This approach allows to reduce search space. Own preparation.

When it comes to utilising EAs in encoding, direct encoding produces a vast amount of parameters every iteration. This approach is computationally expensive, thus indirect encoding is used. An ANN architecture can be represented in many different ways. Among the most common in the literature are graphs with nodes and edges, where nodes are operations and edges are inputs or outputs. However, in this case the number of encoded results can become large. Thus, one of the ways of limiting the search space is to represent

it as cells repeated sequentially. This simplification can go further. For instance it can be assumed that the cell consists of a predefined number of edges and nodes. Mathematically, each node consists of outputs of previous nodes. The final output is a sum of previous nodes. This solution is used in popular NAS benchmarks such as NAS-Bench-101 [59] or NAS-Bench-201 [23].

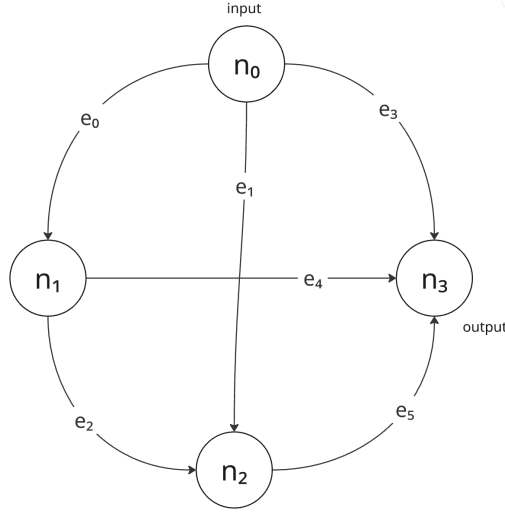


Fig. 4.4. Depiction of cell used in NAS-Bench-201. Own preparation.

4.4.2.2. Genetic operators in EA-based NAS

Elitism and tournament selection are the most popular ones [10]. Elitism promotes individuals with higher fitness and makes populations less diverse. Tournament selection equally samples individuals and chooses the best from them. Other selection strategies used were roulette wheel, which gives individuals a probability of being chosen according to their fitness value, and aging evolution, which focuses on discarding the oldest or worst individual [10].

In terms of mutation, several different approaches are presented. Sun et al. [54] used polynomial mutation, a method where parent solution's variables are perturbed by sampling from a polynomial probability distribution [60]. In this work, parameters of DNN layers were represented as real numbers. Another approach can be found in the Lorenzo and Nalepa study [61]. The researchers proposed mutation based on Gaussian regression, where it predicts which architecture is a promising candidate. *net2widernet* and *net2deepernet* [62] are two operators introducing network morphism, a method where a trained parent is transformed into a child network [63].

The crossover operator is the most controversial one. Some authors (*LargeEvo* [55], Wistuba et al. [64]) do not use it at all as they claim it does not suit the DNN paradigm and mutation is sufficient. However, Sun et al. [54] designed a crossover operator for variable-length

CNN encodings, the unit-alignment crossover, and used it as a core operator, which shows that crossover can suit CNN evolution.

4.4.3. Data processing and AutoML

Another key area is data processing, including data generation, imbalance of provided or generated datasets and pre-processing challenges. Research has shown that the use of EC to optimize data processing consistently boosts the performance of neural networks compared to when these methods are not applied [50].

A further survey [65] reviews potential issues and the optimization of DNN using EC, focusing on the concept of automated machine learning (AutoML). AutoML is a category of machine learning whose purpose is to automate the end-to-end process of applying ML algorithms to specific case studies [66]. The survey underlines that EDL is computationally expensive, requires substantial resources and often struggles with the trade-off between efficiency and accuracy. Furthermore, the majority of works concentrate on small datasets tested on a few algorithms. Despite these challenges, EC- and EA-based optimization of DNNs is consistently more effective than alternatives such as random search or local search.

4.5. MULTI-OBJECTIVE PROBLEMS

Many optimization-related studies focus on single objective. Single-objective optimization considers a problem defined by one objective function, also called the fitness function, $f(x)$ that assigns a single scalar quality value to every candidate solution x . Solving such a problem means finding one solution that optimizes (minimises or, equivalently, maximises) this function. However, in real-world engineering the goal is rarely reduced to a single quantity. For instance, ANNs can be optimized for accuracy together with other objectives: inference latency, parameter count, energy consumption etc. Improving one criterion may involve worsening other criteria. Such problems are called multi-objective (MO) problems and the task of finding the best solution responding to both objectives is called multi-objective optimization. A MO optimization can focus on m objective functions simultaneously, $f_i(x)$ with $i \in \{0, 1, \dots, m-1\}$, whose values form an objective value vector

$$f(x) = (f_0(x), f_1(x), \dots, f_{m-1}(x)), \quad (4.6)$$

where x is a candidate solution [67].

When the objectives conflict, no single solution is optimal for all of them at once, so the scalar comparison used in single-objective optimization is replaced by a dominance relation.

For a minimisation problem, a solution $x^{(1)}$ dominates a solution $x^{(2)}$ when it is not worse on any objective,

$$f_i(x^{(1)}) \leq f_i(x^{(2)}) \text{ for all } i \in \{0, 1, \dots, m-1\}, \quad (4.7)$$

and strictly better on at least one objective, which, given the inequalities above, is equivalent to requiring the two objective vectors to differ, $f(x^{(1)}) \neq f(x^{(2)})$.

4.5.1. Pareto front

Unlike single-objective problems, multi-objective problems do not have a single best solution. Instead, MO algorithms return a set of incomparable solutions. Each solution represents a different trade-off between objectives. This set of solutions is called the Pareto front.

Let $\mathcal{X}$ denote the set of feasible solutions, and write $x^{(1)} \prec x^{(2)}$ when $x^{(1)}$ dominates $x^{(2)}$ under the relation in Equation 4.7. A feasible solution x^* is Pareto-optimal when no feasible solution dominates it. The Pareto-optimal set, also called the Pareto set, is the set of all Pareto-optimal solutions in the search space:

$$\mathcal{P}_S = \{x^* \in \mathcal{X} : \nexists x \in \mathcal{X} \text{ such that } x \prec x^*\}. \quad (4.8)$$

The Pareto front is the image of the Pareto-optimal set in the objective space, that is, the set of objective vectors produced by the Pareto-optimal solutions:

$$\mathcal{P}_F = \{f(x^*) : x^* \in \mathcal{P}_S\}. \quad (4.9)$$

The Pareto-optimal set belongs to the search space, while the Pareto front is its representation in the objective space [67].

A solution can be added to the Pareto front if no other feasible solution dominates it. This is the most direct strategy of accepting new candidate to Pareto-front. However, almost every pair of solutions becomes mutually non-dominated and the front no longer separates good solutions from bad. There are other strategies to bypass this limitation.

The first is scalarisation, which lets candidates compete as in single-objective search. A weight vector $\lambda = (\lambda_0, \dots, \lambda_{m-1})$, with $\lambda_i \geq 0$ summing to one, fixes the trade-off, and a different weight vector shifts the search to another part of the front, so a spread of them, say $(0.5, 0.5)$ and $(0.8, 0.2)$ for two objectives, recovers the front piece by piece. Decomposition algorithms such as MOEA/D [68] are built on this idea. The weighted sum $\sum_i \lambda_i f_i(x)$ cannot reach solutions on a non-convex part of the front, so other aggregation functions are used instead. The Tchebycheff function minimises the largest weighted gap to a reference

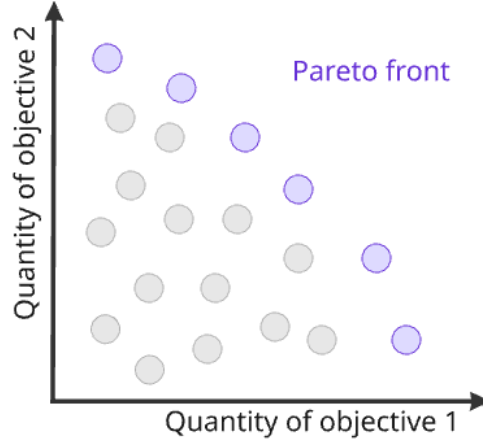


Fig. 4.5. Purple dots create a set of non-dominated solutions for trade-off between objectives. Own preparation.

point,

$$g^{te}(x \mid \lambda, z^*) = \max_{0 \leq i < m} \lambda_i |f_i(x) - z_i^*|, \quad (4.10)$$

where z^* is the reference point and z_i^* the best value reached for objective i [69]. Minimised once per weight vector, it draws the search towards a different region of the front each time. PBI keeps the same reference point but splits the gap into a distance d_1 along the weight vector and a distance d_2 away from it, then minimises $d_1 + \theta d_2$, where the penalty θ sets how much the off-axis distance counts [68].

Indicator-based selection scores the whole approximation set with a single indicator and removes the solution that contributes least to that score [12]. Common choices for indicator-based strategies are hypervolume and the two IGD variants.

Hypervolume (HV) is the volume of the region dominated by the Pareto front and bounded by a reference point [70]. Formally, consider an m -objective minimisation problem and let $A = \{a^{(1)}, \dots, a^{(|A|)}\} \subset \mathbb{R}^m$ be the approximation set (the obtained Pareto front) returned by an algorithm. The hypervolume indicator requires a reference point $r \in \mathbb{R}^m$ that is dominated by every member of A . It is defined as the Lebesgue measure Λ of the portion of objective space that is dominated by A and bounded from above by r :

$$HV(A, r) = \Lambda \left(\bigcup_{a \in A} \{z \in \mathbb{R}^m : a \preceq z \preceq r\} \right), \quad (4.11)$$

where $a \preceq z$ denotes that a weakly dominates z . Hypervolume jointly captures convergence to and spread along the true Pareto front.

The inverted generational distance (IGD) instead relies on a reference set $Z = \{z^{(1)}, \dots, z^{(|Z|)}\}$

that samples the true Pareto front. It averages, over the reference set, the Euclidean distance from each reference point to its nearest neighbour in the obtained set A :

$$IGD(A, Z) = \frac{1}{|Z|} \sum_{z \in Z} \min_{a \in A} \|z - a\|_2, \quad (4.12)$$

where $\|\cdot\|_2$ is the Euclidean norm in objective space.

IGD+ [71] is a weakly Pareto-compliant modification of IGD. For a minimisation problem it replaces the Euclidean distance with a one-sided distance d^+ that accumulates only the objectives on which an obtained point a is worse than the reference point z :

$$IGD^+(A, Z) = \frac{1}{|Z|} \sum_{z \in Z} \min_{a \in A} d^+(z, a), \quad (4.13)$$

$$d^+(z, a) = \sqrt{\sum_{j=1}^m (\max\{a_j - z_j, 0\})^2}. \quad (4.14)$$

The term $\max\{a_j - z_j, 0\}$ is nonzero only when a is worse than the reference on objective j , so objectives on which the obtained front already matches or exceeds the reference contribute nothing. Consequently a front that dominates another can never obtain a worse IGD+ value, and, as with IGD, smaller values are better.

4.5.2. EAs with multi-objective problems

Multi-objective optimization is one of the main areas of interest when it comes to EA-based optimization of DNNs. EAs are one of the methods used for multi-objective problems. The survey from 2022 [12] categorises multi-objective Evolutionary Algorithms (MOEAs) into dominance-based, indicator-based and decomposition-based.

Dominance-based MOEAs characterise themselves by utilising the principle of Pareto dominance. To apply this principle in a population, dominance-based MOEAs combine two mechanisms. Firstly, individuals are partitioned into successive fronts by non-dominated sorting. The first front contains all individuals non-dominated by anyone, the second front contains individuals dominated only by the first front and so on. Secondly, within a single front solutions are spread evenly along the front using a specified criterion.

For instance, NSGA-II [13] combines non-dominated sorting with a crowding distance to preserve diversity along the Pareto front. The crowding distance is a method of estimating how isolated a solution is from its neighbours in objective space. Let $\mathcal{F}$ be a non-dominated front and $d(x)$ the crowding distance of a solution $x \in \mathcal{F}$, initialised to zero. For every objective $i \in \{0, \dots, m-1\}$ the solutions in $\mathcal{F}$ are sorted by f_i , the two extreme solutions

of that order are assigned $d = \infty$ so that they are always kept, and every interior solution receives the contribution

$$\frac{f_i(x^+) - f_i(x^-)}{f_i^{\max} - f_i^{\min}}, \quad (4.15)$$

where x^+ and x^- are the neighbours of x in the sorted order and $f_i^{\max}$, $f_i^{\min}$ are the largest and smallest values of objective i on the front. The crowding distance $d(x)$ is the sum of these contributions over all objectives, an estimate of the perimeter of the cuboid formed by the nearest neighbours of x [13]. NSGA-II selector uses crowded comparison ($\prec_n$). A solution x precedes y when it has a lower front rank, or, for two solutions on the same front, when $d(x) > d(y)$. Selection favours solutions that are ranked better and from less crowded regions. Algorithm merges the parent and offspring populations and creates non-dominated fronts from them. After that the next population is filled with one front at a time. The first front that does not fit entirely is truncated. NSGA-II is a representative algorithm in the MOEA NAS literature as it returns the entire Pareto front in a single run (rather than a single trade-off point). NSGA-Net [14] adapts the NSGA-II algorithm to cell-based NAS for CNNs with two objectives: FLOPs and classification error.

LEMONADE [15] extends the same dominance principle but distinguishes objectives into a cheap and an expensive set and evaluates expensive accuracy less often than FLOPs. NSGANetV2 [72] merges NSGA-II with surrogate models that predict the expensive objective, and NAT [73] scales the framework to many-objective settings in which more than three objectives (accuracy, parameters, FLOPs, latency, memory) are optimized jointly. SMS-EMOA [74] is a hybrid algorithm, using Pareto-dominance strategy combined with hypervolume indicator in selection.

Decomposition-based MO algorithms take a different route. Instead of ranking a population by dominance, they break a multi-objective problem into a set of scalar subproblems using aggregation functions such as the weighted Tchebycheff method (Equation 4.10) [69], the weighted sum, or the weighted min-max formulation and solve them together. The canonical algorithm in this category is MOEA/D [68]. The algorithm defines N subproblems and then assigns one weight vector to each subproblem. All subproblems are optimized simultaneously in a single population by exchanging information between subproblems with similar weight vectors. Neighbourhoods are set by the Euclidean distance between weight vectors, and each subproblem is improved using only the current solutions of its neighbours, because neighbouring subproblems have similar optima. MOEA/D is widely used as a baseline against which the dominance-based MOEAs discussed below are compared.

Beyond the NSGA-II family, an alternative line of dominance-based MOEAs grows from the GOM Evolutionary Algorithm (GOMEA). The single-objective GOM, described previously

in Section 4.3.2, learns one linkage model, a family of subsets organised as a linkage tree, and improves each solution by accepting a change only when it does not worsen the fitness function. MO-GOMEA [75] also replaces traditional crossover with the OM operator. The general workflow of the algorithm is based on an elitist archive of the non-dominated solutions found during the computations. The archive stores current approximation of the Pareto front and provides elitism. In each iteration, based on the solution positions in the objective space, population is clustered into a fixed number of equally sized groups. A separate linkage model is learned for each group. A solution is then improved by mixing it using OM with donors taken from its own cluster, so different regions of the front are handled separately and the solutions stay spread along it. Acceptance of the new solution in OM in GOMEA is based on the dominance principle. A new solution is accepted when it dominates the archive, equals its objective values, or remains non-dominated with respect to the elitist solutions. Algorithm 6 states one generation of MO-GOMEA.

Algorithm 6 One generation of MO-GOMEA

```

1: partition the population into  $k$  equally sized clusters by proximity in objective space
2: for all clusters  $C$  do
3:   learn a linkage model  $\mathcal{F}_C$  from the solutions of  $C$ 
4: end for
5: for all solutions  $x$  of the population do
6:    $C \leftarrow$  the cluster of  $x$ ;  $o \leftarrow x$ 
7:   for all linkage sets  $S$  of  $\mathcal{F}_C$ , in random order do
8:      $b \leftarrow o$ 
9:     pick a random donor  $d$  from cluster  $C$ 
10:     $o \leftarrow o$  with the variables in  $S$  set to the donor's values  $d_S$ 
11:    if  $o$  differs from  $b$  then
12:      evaluate  $o$ 
13:      if  $o$  dominates  $b$ , equals its objective values, or is non-dominated with
        respect to the elitist archive then
14:        update the elitist archive with  $o$  ▷ accept the donated block
15:      else
16:         $o \leftarrow b$  ▷ revert
17:      end if
18:    end if
19:  end for
20:   $x \leftarrow o$ 
21: end for

```

MO-GOMEA was shown to scale better than NSGA-II and MOEA/D on problems where variables interact in non-trivial groups. The combination of MO-GOMEA with multiple pretrained supernetworks was demonstrated by ENCAS [21], which uses MO-GOMEA to jointly optimize accuracy and FLOPs of architecture cascades.

A related approach is taken by the Parameter-less Population Pyramid (P3) family. Its multi-objective variant MO-P3 [76] combines P3 and assigns a weight vector to each new individual, directing the search towards a specific part of the Pareto front. This method was shown to outperform NSGA-II and MOEA/D on the industrial production planning problem.

Similarly, the Pareto-wise ranking classifier was proposed to alleviate rank disorder in multi-objective NAS [77]. In this regard, the NAS task is reformulated as a classification problem by training a classifier to predict dominance relationships between architectures. It uses adaptive clustering for selecting reference architectures and balancing the sample distribution. Consequently, the Pareto-wise method reduces rank disorder and outperforms state-of-the-art approaches, efficiently discovering diverse architectures under various constraints.

Other dominance-based MOEAs have been proposed specifically for NAS. MOEA-PS [78] combines traditional sorting with queue-based evolution. It selects a fixed number of individuals from the initial population, trains them and then introduces mutations. This approach greatly reduces the error rate compared to other architectures tested on CIFAR-10 and CIFAR-100 datasets.

4.5.3. Surrogates

A surrogate-assisted (SA) optimization is a type of optimization where, instead of running expensive computations, a small, lightweight model is used to predict the outcome. SA optimization has proven to go well with EAs in solving computationally expensive problems, such as MO problems [17]. This section elaborates on the aspects of SA optimization.

Surrogates are commonly grouped into regression-based, classification-based and relation-based models [19]. Another type of division for surrogates is into deterministic and probabilistic. Deterministic surrogates return the predicted value only, while probabilistic surrogates also return how certain they are about the value they predicted.

Regression-based surrogate models predict objective values (e.g., fitness). Common surrogates using regression are the Gaussian Process, Radial Basis Functions (RBF), Neural Networks, Extreme Learning Machines, Multi-Layer Perceptrons (MLP) and Support Vector Regression (SVR). The Gaussian process (also known as Kriging) is a computationally heavy surrogate that proved its worth in MO problems. It returns both a mean prediction and a variance estimate. RBF networks offer strong local approximation at low computational cost, at the price of sensitivity to kernel parameters. Neural network surrogates model complex nonlinear landscapes but tend to overfit when only a few real evaluations are available. Ensemble surrogates combine homogeneous or heterogeneous base models to

improve robustness on irregular landscapes [19]. SVR extends the concept of the Support Vector Machine (described later in this section) to regression problems. It replaces the classification loss with a distance-based loss function [18]. Given a training set, SVR fits a linear regression function $f(x) = \langle w, x \rangle + b$, where the weight vector w is obtained by minimising

$$\phi(w, \xi) = \frac{1}{2} \|w\|^2 + C \sum_{i=1}^n (\xi_i^- + \xi_i^+), \quad (4.16)$$

with C a regularisation constant and ξ_i^-, ξ_i^+ slack variables bounding the residuals from below and above [18]. Non-linear regression is obtained by mapping the input to a higher-dimensional feature space via a kernel function $K(x, y)$ selected for the given problem [18]. Unlike the Gaussian process, SVR does not produce an uncertainty estimate.

Classification-based surrogates predict a categorical label such as dominance, Pareto rank, feasibility or membership of a promising set [19]. Common classifiers include Support Vector Machines, neural networks, Random Forests and K-Nearest Neighbours. The surrogate is then used in one of several ways: as a pre-screen that discards weak candidates before real evaluation, as a target of an active learning loop that selects informative samples near the decision boundary, or as an archive manager that decides which solutions are worth keeping [19]. Discrimination is cheaper than full regression, which is where the observed speed-up comes from. The corresponding weakness is that objective magnitudes are lost, and the classifier degrades quickly under label noise or class imbalance [19].

Relation-based surrogates shift from predicting objective values to learning relative relationships between candidate solutions, such as dominance hierarchies, Pareto rankings or pairwise preferences [19]. Typical models include ordinal regression, probabilistic ranking models and neural networks trained on pairwise inputs. They are used in two main ways: dominance prediction, where the model decides which of two candidates dominates the other and filters unpromising offspring on this basis and ranking approximation, where the model estimates a relative ordering on the population to guide selection [19]. Because the learning target is relational, these types of surrogates are rather insensitive to objective scaling and to irregularly shaped Pareto fronts, and they extend better to many-objective problems than regression-based alternatives. However, absolute objective values are unavailable for downstream use, and the reliance on pairwise training data makes the model prone to overfitting early in the search, when few labelled pairs exist [19].

4.5.4. EA-based SA optimization

Implementation of surrogates in optimization algorithms is one way of cutting the number of expensive evaluations. Surrogate-assisted EAs follow a common loop, where an initial dataset is built by sampling and real evaluation, a surrogate is trained on this dataset and

EA generates solutions whose fitness is estimated by the surrogate. Only candidates that meet the criteria are passed to real evaluation [19].

The training schedule of the surrogate varies between methods. Jin et al. [79] distinguish offline and online training methods. An offline surrogate is fitted once on a fixed set of evaluations and is not updated during the search. New solutions are evaluated, added to the training data and the model is refitted as the run proceeds. The online training allows the model to be refitted during the run. Typical configuration in EA-based SA optimization is refitting the model either every generation or every real-evaluated solution [80, 81, 20].

SA-P3-GOMEA [20] is an EA-based SA optimization method that uses individual-based control. This algorithm solves single-objective problem of partition-based ensemble learning. It uses a surrogate that estimates fitness on candidate solutions before any real evaluation is performed. The surrogate selects which solutions are worth a real evaluation by first scoring the candidate. If surrogate fitness exceeds the threshold T , real evaluation is performed. The threshold is the λ -quantile of the surrogate fitnesses of the solutions that already have a real fitness, so it depends on the solutions evaluated so far. λ controls how selective the gate is. Algorithm starts with the value of $\lambda = 1$. After every iteration in which no new best solution is found, it is assumed that the surrogate has become inaccurate, and λ is decreased by a factor $\eta = 0.999$. Finding new best solution sets λ back to value of 1. The surrogate is retrained after each real evaluation. The run starts with a warm-up of ℓ random solutions that are evaluated for real, where ℓ is the number of variables, and the first surrogate is trained on them. Hyperparameters of every surrogate are tuned only once, at the beginning of the experiments. Algorithm 7 states the full procedure.

SA-P3-GOMEA also uses OM and pyramid-based population management (described earlier in Section 4.3.2). In an algorithm, solutions are stored in layers of the pyramid. As in the regular P3, in each iteration new solution is created randomly and climbs the pyramid using OM.

Four regressors were considered in the original work: Multi-Layer Perceptron (MLP), Gradient Boosting, Support Vector Regression and Random Forest. The researchers decided to use SVR. On a partition-based ensemble learning problem with up to 500 variables, SA-P3-GOMEA found better solutions than P3-GOMEA without a surrogate and than the Bayesian optimization baselines COMBO [82], the Tree-structured Parzen Estimator [83] and SMAC [84], within a budget of a few thousand real evaluations.

SA-P3-GOMEA compares the surrogates once before the run. The gate decides whether to trust the surrogate, but not which surrogate to use. Not every surrogate is best on every problem [85], which turns the surrogate type into a parameter worth tuning [86]. One option keeps several models at once and combines them as a weighted ensemble [87, 88].

Algorithm 7 SA-P3-GOMEA, reproduced from [20]. Lines marked "surrogate" are the additions over plain P3-GOMEA.

```

1:  $R \leftarrow \emptyset$  ▷ solutions with a known real fitness
2: for  $i = 1$  to  $\ell$  do ▷ warm-up:  $\ell$  random real evaluations
3:   add a random, really evaluated solution to  $R$ 
4: end for
5: train the surrogate on  $R$  ▷ surrogate
6:  $F \leftarrow$  surrogate fitness of every  $x \in R$  ▷ surrogate
7:  $\lambda \leftarrow 1$ 
8:  $T \leftarrow$  the  $\lambda$ -quantile of  $F$  ▷ surrogate
9: create an empty pyramid
10: while the termination criterion is not met do
11:    $p \leftarrow$  a random solution; add it to pyramid level 0
12:    $elitistBefore \leftarrow elitist$ 
13:    $L \leftarrow 0$ 
14:   while  $L \leq$  highest pyramid level do
15:     learn the linkage tree of pyramid level  $L$ 
16:      $o \leftarrow \text{GOM}(p, \text{level } L)$  ▷ Algorithm 2; evaluation gated by Algorithm 8
17:     if  $\text{COMPARE}(o, p)$  prefers  $o$  then
18:       add  $o$  to pyramid level  $L + 1$ 
19:        $p \leftarrow o$ 
20:     end if
21:      $L \leftarrow L + 1$ 
22:   end while
23:   if  $elitist$  did not improve during this iteration then
24:      $\lambda \leftarrow \lambda \cdot \eta$  ▷ surrogate: relax the gate
25:   end if
26:   retrain the surrogate on  $R$  and recompute  $F$  ▷ surrogate
27:    $T \leftarrow$  the  $\lambda$ -quantile of  $F$  ▷ surrogate
28: end while

```

A different option keeps the pool but commits to one model at a time, re-chosen during the run from how well each model fits the data seen so far. Cross-validation error is the usual measure of fit [89]. In architecture search the surrogate only has to order candidates, so the rank correlation between predicted and real values says more about its usefulness than the absolute error [90]. Selection of this kind already appears in the literature. Lu, Sun and Tang switch between model types according to the quality of the solutions each one has produced [91], and NSGANetV2 chooses among accuracy predictors by Spearman rank correlation as evaluations accumulate [72].

Another MO SA algorithm is ENCAS [21]. It uses MO-GOMEA to search over pretrained supernetworks and treats the supernetwork itself as a surrogate, since weight sharing makes evaluating a candidate architecture orders of magnitude cheaper than training it from scratch. The resulting Pareto front of accuracy versus FLOPs cascades outperformed both NSGA-II-based and gradient-based NAS baselines.

Algorithm 8 Surrogate-gated evaluation and comparison used inside GOM, reproduced from [20].

```
1: function EVALUATE( $x$ )
2:    $x.surrogateFitness \leftarrow$  surrogate prediction for  $x$ 
3:   if  $x.surrogateFitness > T$  then     $\triangleright$  spend a real evaluation only on promising
      solutions
4:      $x.fitness \leftarrow$  real fitness of  $x$ ; mark  $x$  as really evaluated
5:      $R \leftarrow R \cup \{x\}$ 
6:     if  $x.fitness > elitist.fitness$  then
7:        $elitist \leftarrow x$ ;  $\lambda \leftarrow 1$      $\triangleright$  reset the gate on a new best solution
8:     end if
9:   end if
10: end function

11: function COMPARE( $x, y$ )
12:   if  $x$  and  $y$  both have a real fitness then
13:     return the comparison by real fitness
14:   else
15:     return the comparison by surrogate fitness
16:   end if
17: end function
```

4.6. SUMMARY

This section covered the foundations this thesis builds on, starting from the basics of ANNs and EAs, through the challenges of training networks and ending with optimization strategies.

Across the surveyed works, three observations emerge. Firstly, MO NAS has typically optimized proxy resource objectives such as FLOPs, parameters or latency instead of the measured energy. Secondly, EAs are a promising approach when it comes to optimization of DNNs, especially in MO. And lastly, surrogates scale well with EAs.

The following chapters address the issue of adapting SA-P3-GOMEA to multi-objective NAS with measured inference and training energy as a second objective to accuracy.

5. THE PROPOSED MULTI-OBJECTIVE SA-P3-GOMEA

This section describes MO-SA-P3-GOMEA, an algorithm which extends single-objective SA-P3-GOMEA proposed in [20] to solve multi-objective problems.

The SA-P3-GOMEA combines OM and parameterless-pyramid operators (both described in Section 4.3.2) with surrogate models to solve non-binary combinatorial problems of partitioning the datasets to achieve optimal accuracy. Surrogates are used for predicting expensive fitness. Real solutions are evaluated only if the surrogate predicts that the dataset is a strong candidate. This thesis proposes four modifications:

- MO-DSA-P3-GOMEA - modification of SA-P3-GOMEA addressing multi-objective problems with deterministic surrogates.
- MO-DSA-P3-GOMEA with online surrogate selection - modification of SA-P3-GOMEA addressing multi-objective problems with deterministic surrogates and online surrogate selection.
- MO-PSA-P3-GOMEA - modification of SA-P3-GOMEA addressing multi-objective problems with probabilistic surrogates.
- MO-PSA-P3-GOMEA with online surrogate selection - modification of SA-P3-GOMEA addressing multi-objective problems with probabilistic surrogates and online surrogate selection.

This work refers to the four variants collectively as MO-SA-P3-GOMEA, without distinguishing the surrogate type.

5.1. NAS

The main goal of proposed MO-SA-P3-GOMEA is to solve MO NAS problems. However, the proposed algorithm should score acceptable results on every combinatorial problem. MO-SA-P3-GOMEA adapts the SA-P3-GOMEA, which was originally applied to partition-based ensemble learning. In general, combinatorial problems share a similar structure: a fixed-length vector of discrete variables, each drawn from an alphabet, with the resulting solution evaluated through an expensive fitness function. The thesis adopts the cell-based encoding, where a small computational graph called a *cell* is the unit of search and the final network is built by stacking many copies of that cell. The chosen benchmark

is NAS-Bench-201 [23], which provides an established cell structure together with an alphabet of possible operators. This defines a problem of 6 variables with an alphabet of size 5, which makes a total search space of 15625 architectures. Each architecture in NAS-Bench-201 has been trained on CIFAR-10 and CIFAR-100, so the accuracy objective can be retrieved by lookup.

5.2. ADAPTING SA-P3-GOMEA TO MULTI-OBJECTIVE NAS PROBLEM

Original work focuses on single-objective problem of partitioning dataset. This work aims to adjust the SA-P3-GOMEA algorithm for multi-objective NAS with two objectives described later in this section. Two separate experiments are run for every architecture: inference energy on the held-out CIFAR-10 test set, and training energy for a short training schedule of one epoch. Inference energy is the per-query cost paid every time the trained model is used, and is the figure of merit when the architecture is meant for deployment. Training energy is the cost paid during development, and dominates the lifetime energy budget when the model is retrained often or when many candidate architectures are tried.

5.2.1. Objective 1 - accuracy

Accuracy is a standard performance measure for image classification, and it is the first of the two objectives. For a set of N labelled examples, top-1 accuracy is the fraction whose predicted class matches the true one [92]:

$$\text{acc} = \frac{1}{N} \sum_{i=1}^N \mathbf{1}[\hat{y}_i = y_i], \quad (5.1)$$

with y_i the true label of example i , $\hat{y}_i$ the class the network predicts, and $\mathbf{1}[\cdot]$ the indicator that equals one when its argument is true and zero otherwise. In this work the accuracy of each architecture is retrieved by lookup from NAS-Bench-201 [23]. NAS-Bench-201 provides accuracies for architectures trained on CIFAR-10, CIFAR-100 and ImageNet. However, only architectures trained on CIFAR-10 are considered. Using a benchmark lookup makes this objective deterministic and reproducible.

5.2.2. Objective 2 - usage of GPU power

The second objective is the resource cost of running a candidate architecture. Multi-objective NAS studies typically replace this cost with a proxy, most often FLOPs, parameter count or inference latency [14, 72, 73, 21]. Proxies of this kind are convenient because they can be read directly from the architecture description, but they correspond only loosely to the energy actually consumed on a given device [93], since they ignore memory traffic, hardware utilisation and the GPU power state. FLOPs, the most common choice, has a

further weakness shown by the pilot study below: it correlates so strongly with accuracy that pairing the two leaves almost no practical trade-off. The second objective in this work is therefore the energy drawn by the GPU during execution, measured directly on the target hardware.

To check whether accuracy and measured GPU energy form a multi-objective problem, a pilot study was conducted on the NAS-Bench-201 search space. A random sample of 1000 architectures was drawn. Each was executed on CIFAR-10 on the target hardware (NVIDIA RTX 5070 Ti) and the inference energy was recorded with pyNVML as the median over $N = 5$ runs, since measuring 1000 architectures on the hardware is time-consuming. The analytic FLOPs count and the CIFAR-10 test accuracy were obtained from NAS-Bench-201 for the same sample. The Spearman rank correlation with accuracy was $\rho = +0.749$ for FLOPs but only $\rho = +0.492$ for measured inference energy (Figure 5.1). FLOPs is thus too tightly coupled to accuracy to function as an independent objective: optimizing the two jointly reduces almost to the accuracy-only ranking. Measured inference energy, in contrast, keeps enough distance from accuracy to leave a genuine Pareto trade-off, and this is the justification for using GPU energy as the second objective in this work.

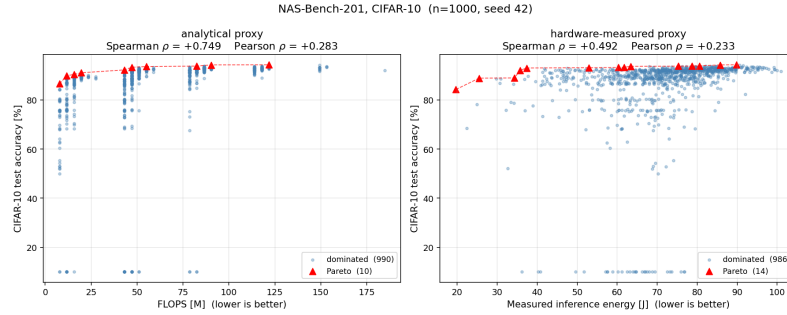


Fig. 5.1. FLOPs and measured inference energy and accuracy for 1000 random architectures from NAS-Bench-201 on CIFAR-10. Inference energy was measured with pyNVML on an NVIDIA RTX 5070 Ti as the median of $N = 5$ runs. Own pilot study.

The experiment measures both inference and training energy. Each result is stored in a JSON cache keyed by the NAS-Bench-201 architecture index. Previously evaluated architectures are read from the cache, which keeps the experiment reproducible across runs.

5.3. SURROGATES

This work proposes two multi-objective approaches to the SA-P3-GOMEA. The first variant uses a single surrogate that predicts the pair (accuracy, energy) from a shared input. The second approach is a per-objective variant that trains two surrogates, one for each objective. The idea behind comparison between these two approaches is that with the joint setup the model can pick up correlations between accuracy and energy, but both predictions have

to share one hyperparameter choice. With the per-objective setup the two predictions are independent, but there are now two models to retrain and no way for one objective to inform the other.

This work also revisits the retraining approach used in SOTA. In the original work, the surrogate and its hyperparameters are chosen once at the start of the run. The surrogate itself is retrained on the growing archive after each real evaluation. This assumes that one regression model stays accurate for the whole search. The distribution of evaluated solutions changes as the algorithm progresses, and a model that fits the early data may stop fitting later data. The proposed schedule keeps the per-evaluation retraining and adds two further steps. At fixed intervals the hyperparameters are re-tuned with a small grid search, so the model can adjust to the new data. At the same intervals, the algorithm picks the surrogate class from a pool of candidates, using the one with the highest cross-validated rank correlation on the archive. Both steps are described in detail in Section 6.4.3.

5.4. ACCEPTING NEW SOLUTION AND DYNAMIC THRESHOLD

A new solution is accepted by a probability of Pareto-dominance. The candidate is admitted when the probability that any archive member dominates it stays below a threshold τ . The threshold is adaptive and parameter-less. It is the λ -quantile of all dominance probabilities observed so far, reusing the percentile rule of the original surrogate gate. The relaxation λ is inherited from SA-P3-GOMEA. It starts at 1, is multiplied by $\eta = 0.999$ after each generation that leaves the non-dominated archive unchanged, and resets to 1 when the archive changes. Because the bar is read from the observed probabilities and from λ , the threshold introduces no constant of its own. Details were previously described in Section 6.4.2.

5.5. SUMMARY

The proposed MO-SA-P3-GOMEA is based on the original algorithm of SA-P3-GOMEA and adds only necessary modifications to adapt it to multi-objective, energy-aware NAS. The MO-SA-P3-GOMEA uses same mechanisms as original SA-P3-GOMEA. The parameterless pyramid operator manages the population and gene-pool OM recombines solutions. Proposed algorithm experiments with deterministic and probabilistic surrogates with two configurations (one surrogate per both objectives and two surrogates, one per objective). This setup was created to compare how hyperparameter choice, one systematic error, retraining, and connection between objectives affect final result. The probabilistic surrogates repeat this pattern to test whether it shapes the uncertainty as well as the point prediction. Finally, the work examines whether the type of surrogate should be fixed. The algorithm keeps a pool and swaps the surrogate at set of intervals. Overall, MO-SA-P3-GOMEA

is specifically designed for NAS with solution encoding inherited from NAS-Bench-201, however it works on any combinatorial problem.

6. METHODS

This section describes and justifies methods used in experiments.

6.1. SOLUTION ENCODING

The thesis uses NAS-Bench-201 [23] as a benchmark set. The 15625 measured architectures NAS-Bench-201 provides are the search space for modified SA-P3-GOMEA. NAS-Bench-201 is a cell-based benchmark. The skeleton of the network is fixed, containing a convolutional stem followed by three stages of N stacked copies of the same cell separated by residual reduction blocks, while the search itself is restricted to the internal structure of the cell. Every candidate architecture therefore corresponds to one cell, and the size of the search space is fully determined by how the cell can be built.

The cell is a directed acyclic graph with four nodes V_0, V_1, V_2, V_3 and six edges. Each edge (i, j) with $i < j$ is assigned exactly one operation drawn from a fixed five-element set. The output of node V_j is the sum of the operation outputs along all incoming edges, and the cell output is taken at node V_3 (cell topology shown in Figure 4.4). The five operations available on every edge are listed in Table 6.1.

Code	Operation	Description
0	none	Zero operation; the edge is suppressed and contributes a constant zero to its target node.
1	skip_connect	Identity; the input of the edge is forwarded unchanged.
2	nor_conv_1x1	ReLU $\rightarrow$ 1×1 convolution $\rightarrow$ BatchNorm.
3	nor_conv_3x3	ReLU $\rightarrow$ 3×3 convolution $\rightarrow$ BatchNorm.
4	avg_pool_3x3	3×3 average pooling with stride 1.

Table 6.1. Operations available on each edge of a NAS-Bench-201 cell [23]. The code column gives the integer used in the encoding of architectures throughout this work.

An architecture is encoded as a vector of six discrete variables, with five values available for each. The full search space therefore contains $5^6 = 15625$ distinct architectures. SA-P3-GOMEA operates directly on this encoding: each candidate solution is a string of six integers from $\{0, 1, 2, 3, 4\}$ and is treated by linkage learning and OM as a generic discrete combinatorial problem.

Every architecture in this space has been trained from scratch by the authors of NAS-Bench-201 on CIFAR-10, CIFAR-100 and ImageNet-16-120. The accuracy objective for any candidate solution is therefore obtained by indexing into this table.

6.2. MEASURING POWER

The second objective requires a numerical value for the energy spent by a candidate architecture. Three families of options were considered before deciding how to obtain it.

The first family is analytic proxies, most often FLOPs and parameter count. Their main appeal is reproducibility, since both quantities are derived directly from the architecture description and do not depend on hardware. However, as confirmed by the pilot study summarised in Figure 5.1, FLOPs correlate too strongly with accuracy on NAS-Bench-201 to act as an independent second objective, and they ignore memory traffic, hardware utilisation and the GPU power state [93]. They were therefore rejected.

The second family is hardware-aware lookup tables, of which HW-NAS-Bench [94] is the closest fit to the search space used here, as it provides precomputed energy and latency values for NAS-Bench-201 architectures. However, the values are reported for IoT and edge accelerators (Edge GPU Jetson, Edge TPU, Eyeriss, FPGA, Raspberry Pi 4), which do not represent the workstation-class GPU on which the CNNs in this work are actually trained and deployed. The second family was therefore also rejected, on the grounds that the reported numbers would describe a different device.

The third family is direct hardware measurement on the target machine. This is the option adopted in this work. Energy is measured through the NVIDIA Management Library (NVML), accessed from Python via the pyNVML bindings [95]. NVML exposes the on-board power sensor of the GPU and is therefore restricted to GPU draw. CPU and DRAM contributions and platform losses (PSU, fans) are not included. This restriction is intentional. The search varies only the cell architecture, and that affects the GPU workload while leaving the rest of the system unchanged. Measuring whole-system energy would introduce variation from components the search does not control.

The measurement protocol is the same for every evaluated architecture. A background thread polls `nvmlDeviceGetPowerUsage` at a fixed interval of 50 ms while the workload runs on the GPU. The energy of a single run is obtained as the trapezoid integral of instantaneous power over wall-clock time. To suppress GPU clock and CUDA initialisation transients, N_{warmup} untimed runs precede every measurement, followed by N_{samples} timed runs. The reported value for the architecture is the median of these samples and the noise level is reported as the interquartile range. Median and IQR are preferred over mean and

standard deviation because pyNVML readings are occasionally affected by outliers from background activity on the GPU. The choice of N_{warmup} and N_{samples} is studied in Section 6.2.1.

To keep measurements reproducible, the full benchmark environment is containerised with Docker and only one measurement process is allowed to use the GPU at a time. The cache entry for an architecture is keyed by the NAS-Bench-201 architecture index, the experiment type, the GPU model returned by NVML, and a protocol version string encoding $(N_{\text{warmup}}, N_{\text{samples}}, \text{sample interval})$. A change in any of these invalidates prior measurements automatically, which prevents accidental mixing of incompatible runs.

6.2.1. Convergence study of the measurement protocol

A typical measure of consumed energy combines warm up series and sample series. A convergence study was performed to determine what number of warmups and what number of samples gives the best results between stability and measurement cost. 5 architectures were selected from pretrained set of 100 architectures by taking 1 architecture from each of 5 equally spaced quantiles of validation accuracy. The worst performing architecture accuracy was 0.10 and best performing accuracy was 0.94. For each architecture the grid $N_{\text{warmup}} \in \{1, 3, 5, 10, 20\}$ and $N_{\text{samples}} \in \{3, 5, 10\}$ was evaluated. The (20, 10) configuration serves as the reference, because current hardware resources do not allow such exhaustive experiments. Each combination was repeated 3 times under identical conditions to estimate the level of noise for pyNVML readings for inference. Results are summarised in Table 6.2.1 and Table 6.2.1.

Results in Table 6.2.1 reveal that monotonic improvement with respect to N_{warmup} does not appear. Combinations with $N_{\text{warmup}} = 3$ or 5 achieve lower deviation than $N_{\text{warmup}} = 10$. Moreover, deviation remains in the range of 1.5-3.6 Joules, suggesting that the number of warmups does not significantly affect pyNVML readings. Given the observations, the configuration $(N_{\text{warmup}} = 3, N_{\text{samples}} = 3)$ was selected for the main experiments as it helps to reduce time running experiments and achieve a mean deviation of 2.4%.

6.3. DETERMINISTIC SURROGATES

The deterministic surrogates studied in this work are the four surrogates originally considered in SA-P3-GOMEA: SVR, MLP, random forest [96] and gradient boosting [97]. Each one is examined in two configurations. The per-objective configuration trains two independent models, one for accuracy and one for GPU energy. The joint configuration trains a single model that predicts both objectives from a shared input. Every surrogate is warmed up on the same set of 100 architectures sampled uniformly from the search space,

$(N_{\text{warmup}}, N_{\text{samples}})$	A1	A2	A3	A4	A5
(1, 3)	103.65	129.11	133.94	159.19	158.97
(1, 5)	104.51	127.90	123.32	161.31	157.48
(1, 10)	113.64	130.99	135.40	154.40	156.40
(3, 3)	115.71	128.50	125.83	154.90	158.55
(3, 5)	115.62	123.27	122.71	154.56	163.59
(3, 10)	108.26	129.07	122.81	162.20	156.08
(5, 3)	109.31	135.70	122.26	160.67	156.83
(5, 5)	105.69	123.12	121.14	161.32	164.85
(5, 10)	115.52	134.67	121.72	162.25	157.81
(10, 3)	105.08	135.40	135.25	163.68	158.94
(10, 5)	109.46	129.25	131.68	154.71	157.34
(10, 10)	109.47	123.67	120.81	154.23	156.74
(20, 3)	109.34	122.65	120.54	161.35	161.83
(20, 5)	108.58	130.05	120.27	155.25	158.94
(20, 10)	116.12	130.99	121.57	161.36	155.13

Table 6.2. Median inference energy in joules for five architectures spanning the NAS-Bench-201 accuracy spectrum, measured under all combinations of $(N_{\text{warmup}}, N_{\text{samples}})$ in the convergence study. Each cell is the median of three independent replicates. The last row (20, 10) corresponds to the most comprehensive protocol available and serves as the reference configuration for the analysis. Architecture labels and validation accuracies (CIFAR-10): A1 = 0_1_0_0_4_0 (acc 0.1000), A2 = 1_1_0_1_4_3 (acc 0.8801), A3 = 1_0_2_1_2_2 (acc 0.9076), A4 = 2_4_0_3_0_3 (acc 0.9213), A5 = 0_3_3_1_4_3 (acc 0.9386). (Table 6.1)

$(N_{\text{warmup}}, N_{\text{samples}})$	mean $\Delta\%$	max $\Delta\%$	mean Δ [J]	rep. std [J]	passes
(20, 10)	0.00	0.00	0.00	3.56	30
(5, 10)	1.15	2.81	1.60	2.61	15
(3, 10)	2.08	6.77	2.56	2.22	13
(5, 3)	2.31	5.86	2.92	2.10	8
(3, 3)	2.40	4.00	3.41	2.36	6
(20, 5)	2.90	6.49	3.94	1.74	25
(1, 5)	3.07	10.00	3.77	1.98	6
(3, 5)	3.39	5.90	4.93	2.76	8
(20, 3)	3.48	6.37	4.58	2.40	23
(10, 10)	3.48	5.73	4.69	2.17	20
(1, 10)	3.73	11.38	4.91	2.28	11
(10, 5)	4.19	8.32	5.47	3.53	15
(5, 5)	4.33	8.98	5.70	1.51	10
(1, 3)	5.24	10.74	6.55	2.34	4
(10, 3)	5.60	11.26	7.05	3.35	13

Table 6.3. Deviation of per-cell median energy from the reference cell ($N_{\text{warmup}} = 20, N_{\text{samples}} = 10$), averaged over five architectures spanning the NAS-Bench-201 accuracy spectrum.

which corresponds to 0.64% of the 15625 candidates. Using a fixed warm-up set across surrogate types removes one source of variance between configurations.

Each configuration is run for 10 random seeds, matching the seeding protocol of the original work. A run terminates on the first of two conditions. The first is a budget of 500 real

evaluations. The second triggers if no new real evaluation is recorded over 1000 consecutive outer iterations of the pyramid loop, which is treated as exploration collapse. The original SA-P3-GOMEA does not need this second guard, because its search space is large enough for the OM operator to keep producing unseen genotypes throughout the run. On the much smaller NAS-Bench-201 space the population in the pyramid converges to a small region. With so little diversity left among the available donors, OM produces only recombinations whose genotypes have already been evaluated, even though most of the 15625-element search space remains unvisited. The count of real evaluations therefore stops growing while the outer loop continues iterating. The second condition makes that situation observable and terminates the run with a defined cause.

A candidate enters the Pareto archive under a strict Pareto-dominance rule (Equation 4.7). It is accepted when no current archive member dominates it, and any archive members it dominates are removed at the same time.

6.4. SURROGATES WITH PROBABILISTIC DISTRIBUTIONS

A deterministic surrogate returns only the value. A probabilistic surrogate also reports how uncertain that prediction is, as a standard deviation σ around the predicted mean μ . This allows the acceptance gate for a new solutions to be adjusted. When the model is confident and σ is small, the gate stays strict. A large σ indicates the model had little data and could not provide a reliable prediction, so it is better to evaluate such a candidate directly.

The probabilistic pool keeps the same families as the deterministic pool of Section 6.3, with a single substitution. Three of the four deterministic families are retained, because each can provide a measure of predictive spread in addition to a point estimate. SVR is removed and replaced by the Gaussian process [98]. SVR fits a single function and provides no estimate of variance. An uncertainty estimate could be obtained externally, for instance by bootstrapping an ensemble of SVR models, but this multiplies the training cost by the number of resamples while still yielding a spread that the model itself does not represent. The Gaussian process belongs to the same kernel-based family as SVR and produces a posterior variance analytically, so it occupies the kernel-based role that SVR previously filled, and unlike SVR it yields an uncertainty estimate.

Each surrogate creates σ through a different construction. A random forest is an ensemble of decision trees, each fitted to a bootstrap resample of the training data, and its prediction is the mean over the trees. The trees disagree most where the training data is sparse, and the standard deviation of their predictions, the inter-tree standard deviation, is taken as σ [96]. Gradient boosting offers no such ensemble and must be extended with two further models under a quantile loss [99], one trained on the 16th percentile of the target and one on the

84th. The width of that interval determines σ , since the two percentiles of a Gaussian fall one standard deviation on either side of the mean. The MLP relies on its dropout layers. When they are left active at prediction time, repeated forward passes over the same input yield different outputs, and σ is the standard deviation of those outputs. This procedure is known as Monte Carlo dropout [100]. For the Gaussian, its posterior already includes a standard deviation for every prediction.

Output configurations from the deterministic study are kept. Random forest, gradient boosting and MLP are each evaluated in the joint and the per-objective configuration. The Gaussian process is evaluated in the per-objective configuration only. A standard Gaussian process predicts a single scalar, and the natural choice is one model per objective, one for accuracy and one for energy. A joint variant would require a multi-output, coregionalised kernel [101]. Such a kernel adds further cost to the most computationally heavy surrogate in the pool and offers no advantage over the per-objective form at this problem scale. The joint Gaussian process is therefore excluded.

6.4.1. Probabilistic acceptance

Once the surrogate returns mean and standard deviation for each objective, the comparison between two solutions becomes a matter of probability. The probability that a new candidate dominates an existing solution is presented in the equation below.

$$P(\text{new} \prec \text{old}) = \Phi \left(\frac{\mu_a^{\text{new}} - \mu_a^{\text{old}}}{\sqrt{(\sigma_a^{\text{new}})^2 + (\sigma_a^{\text{old}})^2}} \right) \cdot \Phi \left(\frac{\mu_e^{\text{old}} - \mu_e^{\text{new}}}{\sqrt{(\sigma_e^{\text{new}})^2 + (\sigma_e^{\text{old}})^2}} \right), \quad (6.1)$$

a is accuracy, e is energy, and Φ is the standard normal cumulative distribution function.

Each factor gives the probability that the candidate is the better of the two predictions, modelled as two independent Gaussians. Accuracy is maximised, energy is minimised. The result is a probability of being better on both objectives at once. A candidate is accepted once this probability reaches the threshold τ (described in Section 6.4.2).

This probability acceptance rule is used in two points where a solution can be accepted. During the OM, a partial change is kept unless the unchanged solution dominates the changed one with probability at least τ (see Algorithm 9).

Same probability acceptance is applied to Pareto archive. Archive members have real objective values, so their standard deviation is zero.

When the predictions are exact, standard deviation equals zero. Each factor then reduces to a step that equals 1 when the candidate is strictly better on an objective and 0 when it is worse.

Algorithm 9 Probabilistic acceptance of a donated block during OM, given the threshold τ

```
1: for all linkage sets  $F$  of the FOS, in random order do
2:    $b \leftarrow o$  ▷ back up the unchanged solution  $o$ 
3:   pick a random donor  $d$  from the current pyramid level that differs from  $o$  on  $F$ 
4:   copy the donor's values  $d_F$  into  $o$  on the positions of  $F$ 
5:   if  $o$  differs from  $b$  then
6:     evaluate  $o$  with the surrogate, obtaining  $\mu$  and  $\sigma$  for each objective
7:      $p_{\text{keep}} \leftarrow P(o \prec b)$  ▷ changed dominates unchanged, Equation 6.1
8:      $p_{\text{revert}} \leftarrow P(b \prec o)$  ▷ unchanged dominates changed
9:     append  $\max(p_{\text{keep}}, p_{\text{revert}})$  to the dominance-probability history
10:    if  $p_{\text{revert}} \geq \tau$  and  $p_{\text{revert}} > p_{\text{keep}}$  then
11:       $o \leftarrow b$  ▷ revert the donated block
12:    end if
13:  end if
14: end for
15: return  $o$ 
```

6.4.2. Threshold

The threshold τ is the probability a candidate must reach to be accepted. A fixed value would work, but it would not reflect how decisively the surrogate separates solutions at a given point in the run, so τ is made adaptive.

Original SA-P3-GOMEA introduces the relaxation parameter λ . It states how strict the surrogate's real-evaluation gate is and begins at its strictest value of 1. Every generation that leaves the non-dominated archive unchanged multiplies it by $\eta = 0.999$, and every changed archive returns it to 1. The original algorithm tied this reset to the improvement of a single elitist solution. A multi-objective search keeps no single elitist, so the change of the Pareto archive takes that role. In SA-P3-GOMEA the η parameter was set to 0.999. This work experimented with a set of three different values (per objective and per surrogate): 0.999, 0.99, 0.9.

Given λ , the threshold is read directly from the dominance probabilities the search has already produced. Every comparison of two surrogate predictions contributes one such probability, collected over the whole run, and τ is their λ -quantile,

$$\tau = Q_\lambda(\mathcal{P}), \quad (6.2)$$

where $\mathcal{P}$ is the set of all observed dominance probabilities and Q_λ their empirical λ -quantile. This is the percentile rule of the original real-evaluation gate, applied to the dominance probabilities instead of the predicted objectives. At $\lambda = 1$ the quantile is the largest probability seen, so only an equally decisive candidate is accepted. As λ relaxes towards 0, the quantile decreases and a less decisive candidate is accepted. The threshold depends

only on λ and the observed probabilities. For a sensitivity analysis τ can instead be set to a fixed constant.

6.4.3. Dynamic surrogate selection and hyperparameter tuning

The variants described so far fix the surrogate type at the start of the run and keep it to the end, as the original algorithm does. This assumes that one type is the best choice. This study examines whether this state is true or not. The set of evaluated architectures shifts as the search concentrates on the promising region, so a model that fits the early, scattered sample need not fit the later, clustered one. A type that ranks accuracy well may rank energy poorly.

For both deterministic and probabilistic configurations the pool of candidate surrogates is kept. The deterministic pool contains eight models in total and the probabilistic pool contains seven models. Re-selection runs on a fixed schedule, every 25, 50, or 100 real evaluations across the experiments, and starts once the archive has reached 30 entries.

The surrogate is only ever consulted to order solutions. The acceptance gate keeps the better of two predictions, and the quantile threshold selects a position in the sorted predictions. A model that shifts or compresses the fitness scale, yet keeps every pair of architectures in the right order, still guides the search correctly, even though a squared-error score such as R^2 would rate it poorly. This work therefore scores surrogate quality by the Spearman rank correlation on unseen solutions. Writing R_i^f and $R_i^{\hat{f}}$ for the ranks of solution i under the true fitness f and the prediction $\hat{f}$ across that set,

$$r_s = \frac{\text{cov}(R^f, R^{\hat{f}})}{s_{R^f} s_{R^{\hat{f}}}}, \quad (6.3)$$

the Pearson correlation of the two rank vectors. It lies in $[-1, 1]$, is invariant to any strictly monotone rescaling of the surrogate output, and equals 1 when the predicted ordering matches the true one. Computing it out of sample, over cross-validation folds or a held-out split, measures how far that ordering generalises to architectures the model has not seen, which is what decides the surrogate's value mid-run.

At each re-selection every model in the pool is trained on the current data and used to predict a holdout of the last fifth of the archive (at least 10 points). Each model is scored by the rank correlation of Equation 6.3, computed separately for accuracy and energy and averaged into one number. The models are sorted by this score, and the top fraction, between 12 and 100 percent of the pool depending on the configuration, becomes the new active subset, never fewer than one model.

Before the first re-selection the whole pool is active and its predictions are averaged. The

search therefore opens with the full ensemble and narrows to the stronger types only once there are enough points to rank them. The rank correlation needs a holdout to be meaningful, and the early archive is too small to separate the types reliably. Committing to one type at that stage would risk locking onto a model that only fits the first few samples.

Each candidate also tunes its own hyperparameters. The first time a type is trained, a grid search fixes its hyperparameters, and that configuration is reused for the rest of the run. This is the tuning step of the original algorithm, applied to every member of the pool instead of to a single fixed model. Grid configurations are ranked by the coefficient of determination R^2 , estimated under 3-fold cross-validation [102]. For solutions with true fitness y_i and prediction $\hat{y}_i$,

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}, \quad (6.4)$$

with $\bar{y}$ the mean of the true values. The denominator is the error of the constant predictor that always returns that mean, so $R^2 = 1$ is an exact fit, $R^2 = 0$ matches the mean baseline, and a negative value lies below it. The cross-validation is what makes the number an estimate for architectures the model has not seen. The training points are shuffled and split into three equal folds. Each fold is predicted by a model fitted on the other two, so every point is scored without having trained on it, and one R^2 is computed over the pooled held-out predictions. A single train-test split would couple the estimate to one arbitrary partition, whereas three rotating folds use every point in both roles and average out that dependence.

6.4.4. Comparison

The proposed MO-SA-P3-GOMEA is compared with two baselines. The first is a multi-objective local search (MO-LS). The second is MO-P3-GOMEA, which evaluates every candidate directly, with no surrogate prediction. Both baselines come from the implementations distributed with the original SA-P3-GOMEA code release [24] and run under the same evaluation budget, the same seeds and the same termination conditions as the surrogate-assisted variants.

Two standard multi-objective metrics are used: hypervolume (HV) [70] and the modified inverted generational distance (IGD+) [71]. HV is the volume of objective space dominated by the archive relative to a fixed reference point (in this case: the worst accuracy and worst energy seen across the search space). IGD+ averages, over points of a reference Pareto front, the distance to the nearest archive member, using a distance metric that ignores axes on which the archive is already at least as good as the reference. IGD+ is used instead of the original IGD [103] because an archive that strictly dominates another can never score worse on IGD+. The reference Pareto front is computed by enumeration over the measurement cache.

6.4.5. Description of proposed sequential modification

The table below shows proposed process of adjusting original SA-P3-GOMEA.

Variant	Description of Modification	Surrogate pool
MO-DSA-P3-GOMEA	Adjusting original SA-P3-GOMEA to MO and NAS	SVR, Random Forest, MLP, Gradient Boosting
MO-DSA-P3-GOMEA with online selection	Online switching between types of surrogates and hyperparameter tuning during the run	SVR, Random Forest, MLP, Gradient Boosting
MO-PSA-P3-GOMEA	Updating multi-objective SA-P3-GOMEA to use probabilistic surrogates	Random Forest with inter-tree deviation, Gradient Boosting with quantile loss, MLP with dropout, Gaussian Process
MO-PSA-P3-GOMEA with online selection	Online switching between types of surrogates and hyperparameter tuning during the run	Random Forest with inter-tree deviation, Gradient Boosting with quantile loss, MLP with dropout, Gaussian Process

Table 6.4. Proposed modifications sequentially.

7. EXPERIMENTS

7.1. EXPERIMENTAL SETUP

7.1.1. Research protocol

The experiments cover the two energy problems from Section 6.2: inference energy on the held-out CIFAR-10 test set, and training energy over a one-epoch schedule. Candidate architectures come from the NAS-Bench-201 search space of 15625 designs. Every variant starts from the same warm-up set of 100 architectures, sampled uniformly, and runs across 10 seeds, 42 to 51. Each evaluation goes into a cache keyed by architecture index, so nothing is measured twice.

7.1.2. Measures

Each run is described by HV and IGD+ (both defined in Section 6.4.4). IGD+ is measured against a reference Pareto front built by enumerating the measurement cache. Hypervolume has a reference pointset to the worst accuracy and worst energy in the cache plus a small margin. For inference energy that point is $(0.0900, 182.8698)$, and its oracle front contains 24 architectures. For training energy it is $(0.0900, 575.2847)$, and the front contains 36. Energy is given in joules.

7.1.3. Stopping criteria

A run ends when it reaches a budget of 500 real evaluations, or sooner if exploration collapses, that is, when the pyramid loop completes 1000 consecutive outer iterations without a new real evaluation. The gate relaxation rate stays at its inherited value $\eta = 0.999$ in every run, and the sections below examine its effect for each surrogate family.

7.1.4. Optimizers

The deterministic variants are the four surrogate types (SVR, MLP, random forest, gradient boosting), each in a joint and a per-objective configuration. The probabilistic variants are the seven described in Section 6.4. Dynamic selection runs over each pool in twelve schedules, one for each pairing of the kept fraction $k \in \{12, 25, 50, 100\}$ percent with the re-selection interval $\{25, 50, 100\}$ real evaluations. Results are compared against multi-objective local search (MO-LS) and multi-objective GOMEA.

7.2. OVERALL COMPARISON AT $\eta = 0.999$

Table 7.1 compares the deterministic surrogates, the probabilistic surrogates, the best dynamic-selection schedule from each pool, and the two baselines side by side at the inherited relaxation rate $\eta = 0.999$. The gap between results is the widest on training energy, where the probabilistic variants reach 351 to 354 hypervolume, apart from the joint MLP at 346, while both baselines stay below 295. The best training hypervolume, 353.98, comes from dynamic selection over the probabilistic pool. The best inference hypervolume and IGD+ come from the probabilistic MLP.

Surrogate	Inference		Training	
	HV	IGD+	HV	IGD+
<i>MO-DSA-P3-GOMEA</i>				
SVR joint	139.48	1.231	294.51	21.776
SVR per-obj	139.62	1.352	299.00	17.582
MLP joint	142.40	0.338	347.84	0.898
MLP per-obj	141.47	0.463	350.29	0.605
RF joint	139.73	1.247	297.18	20.613
RF per-obj	139.89	1.296	305.80	16.189
GBoost joint	139.07	1.489	305.18	17.056
GBoost per-obj	139.03	1.451	332.86	5.297
<i>MO-PSA-P3-GOMEA</i>				
RF joint	142.91	0.253	351.80	0.348
RF per-obj	142.53	0.418	353.72	0.106
GBoost joint	142.03	0.479	351.46	0.392
GBoost per-obj	142.45	0.317	353.75	0.010
MLP joint	143.03	0.211	345.96	1.450
MLP per-obj	143.06	0.212	351.52	0.464
Gaussian process per-obj	142.02	0.278	352.51	0.073
<i>Online selection, best schedule</i>				
MO-DSA-P3-GOMEA ($k = 100$, interval 50)	139.17	1.400	342.69	1.347
MO-PSA-P3-GOMEA ($k = 12$, interval 25)	142.07	0.350	353.98	0.090
<i>Baselines</i>				
MO-LS	140.23	1.113	294.23	16.155
MO-P3-GOMEA	138.85	1.522	280.29	16.141

Table 7.1. Overall comparison at the inherited relaxation rate $\eta = 0.999$, mean over 10 seeds (best per column in bold).

Table 7.2 reports the statistical significance of the differences in Table 7.1.

Method	Inference		Training	
	HV	IGD+	HV	IGD+
<i>MO-DSA-P3-GOMEA</i>				
SVR joint	0.002	0.002	0.002	0.002
SVR per-obj	0.002	0.002	0.002	0.002
MLP joint	0.027	0.105	0.014	0.002
MLP per-obj	0.010	0.037	0.492	0.084
RF joint	0.002	0.002	0.002	0.002
RF per-obj	0.002	0.002	0.002	0.002
GBoost joint	0.002	0.002	0.002	0.002
GBoost per-obj	0.002	0.002	0.002	0.002
<i>MO-PSA-P3-GOMEA</i>				
RF joint	0.557	0.695	0.020	0.004
RF per-obj	0.131	0.020	0.492	0.002
GBoost joint	0.004	0.020	0.020	0.002
GBoost per-obj	0.027	0.105	0.922	–
MLP joint	1.000	–	0.002	0.002
MLP per-obj	–	0.695	0.232	0.004
Gaussian process per-obj	0.010	0.695	0.049	0.006
<i>Online selection</i>				
MO-DSA ($k = 100$, interval 50)	0.002	0.002	0.002	0.002
MO-PSA ($k = 12$, interval 25)	0.020	0.105	–	0.014
<i>Baselines</i>				
MO-LS	0.002	0.002	0.002	0.002
MO-P3-GOMEA	0.002	0.002	0.002	0.002

Table 7.2. Wilcoxon signed-rank p-values for the differences in Table 7.1, comparing each method against a control and paired over the 10 seeds 42 to 51. Each problem has one control per indicator, marked with a dash ((probabilistic MLP per-objective on inference, dynamic selection over the probabilistic pool on training for HV), (probabilistic joint MLP on inference, probabilistic per-objective gradient boosting on training for IGD+)). A value below 0.05, in bold, means the two medians differ.

7.3. MO-DSA-P3-GOMEA

The MO-DSA-P3-GOMEA variants are evaluated on the training-energy and inference-energy problems. Each of the four surrogates (SVR, MLP, random forest, gradient boosting) runs in a joint and a per-objective configuration, which gives eight variants. The results below report each variant separately for the two problems, over 10 seeds from 42 to 51, for 160 runs in total.

Surrogate	Inference		Training	
	HV	IGD+	HV	IGD+
SVR joint	139.48 ± 1.28	1.231 ± 0.610	294.51 ± 32.42	21.776 ± 15.445
SVR per-obj	139.62 ± 1.07	1.352 ± 0.574	299.00 ± 30.01	17.582 ± 14.902
MLP joint	142.40 ± 0.66	0.338 ± 0.180	347.84 ± 6.80	0.898 ± 0.942
MLP per-obj	141.47 ± 1.12	0.463 ± 0.452	350.29 ± 8.46	0.605 ± 1.354
RF joint	139.73 ± 0.95	1.247 ± 0.482	297.18 ± 30.69	20.613 ± 14.330
RF per-obj	139.89 ± 0.93	1.296 ± 0.621	305.80 ± 28.46	16.189 ± 13.110
GBoost joint	139.07 ± 1.42	1.489 ± 0.623	305.18 ± 34.91	17.056 ± 16.142
GBoost per-obj	139.03 ± 1.62	1.451 ± 0.635	332.86 ± 17.08	5.297 ± 5.690
MO-LS	140.23 ± 1.19	1.113 ± 0.573	294.23 ± 27.40	16.155 ± 12.713
MO-P3-GOMEA	138.85 ± 1.09	1.522 ± 0.708	280.29 ± 35.87	16.141 ± 11.183

Table 7.3. Deterministic surrogates, mean $\pm$ deviation over 10 seeds. Best HV and IGD+ per problem is bold. Reference points (0.0900, 182.8698) on inference and (0.0900, 575.2847) on training, with oracle Pareto front sizes 24 and 36.

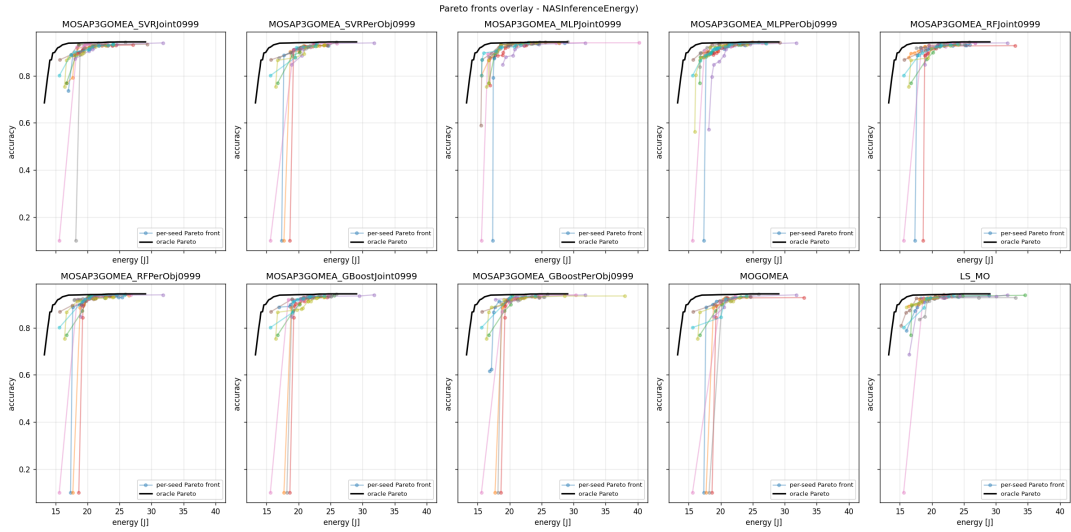


Fig. 7.1. Pareto-front overlay across algorithms for the inference-energy problem.

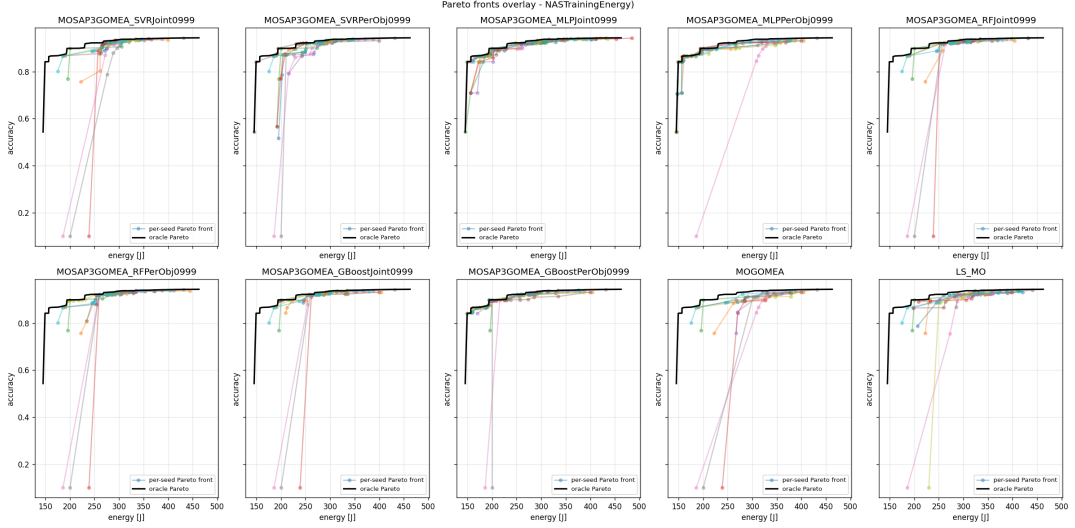


Fig. 7.2. Pareto-front overlay across algorithms for the training-energy problem.

On inference MLP joint has both the highest hypervolume and the lowest IGD+, 142.40 and 0.338. On training per-objective MLP reaches 350.29 hypervolume against 347.84 for the joint variant and 0.605 IGD+ against 0.898, though the two MLP variants are placed within each other's hypervolume deviation. MLP joint has the smallest training hypervolume deviation of any algorithm, 6.8.

On inference the eight surrogate variants span 3.4 hypervolume. On training the per-objective configuration is the higher of the pair for every surrogate type, by 27.7 for gradient boosting, 8.6 for random forest, 4.5 for SVR, and 2.5 for MLP.

MO-P3-GOMEA has the lowest hypervolume on both problems, 138.85 on inference and 280.29 on training, and the smallest archive, 5.2 and 5.5. MO-LS reaches 140.23 inference hypervolume, 2.2 below MLP joint, and 294.23 on training, close to SVR joint.

7.3.1. Sensitivity to the gate relaxation rate

The relaxation-rate study is repeated for the deterministic surrogates, over $\eta \in \{0.9, 0.99, 0.999\}$ and the eight variants above. Table 7.4 reports the result.

For the deterministic surrogates the loosest rate $\eta = 0.9$ is best, or close to it, for almost every variant on both problems, and the inherited $\eta = 0.999$ is consistently the worst. On inference, moving to $\eta = 0.9$ lifts SVR joint from 139.48 to 142.27 HV and cuts its IGD+ from 1.231 to 0.171. Across the eight variants HV runs from 139.0 to 141.2 at $\eta = 0.999$ and up to 142.7 at $\eta = 0.9$. On training per-objective SVR spans from 299.00 to 324.95 HV, and per-objective random forest from 305.80 to 332.42.

The deterministic tables, and the comparison with the probabilistic surrogates, are reported at $\eta = 0.999$, the worst rate for the deterministic surrogates. Tuning to $\eta = 0.9$ closes the inference gap for them. The best deterministic variant, MLP joint at 142.74 HV, reaches the probabilistic range. On training the best probabilistic variant is placed at 355.51 HV against 341.60 for the best deterministic one.

Table 7.4: Gate relaxation rate for the deterministic surrogates on the per-objective-gate campaign, mean over 10 seeds. Each surrogate runs under 12 gate configurations, the single-rate joint gate (joint rows) and the nine per-axis accuracy and energy rate pairs (η_a/η_e). Best HV and lowest IGD+ per surrogate and problem in bold. The joint 0.999 rows match Table 7.3 for the SVR, random forest, and gradient boosting surrogates. The MLP rows differ, at 140.79 and 332.38 hypervolume here against 142.40 and 347.84 there, from the stochastic MLP training. MO-LS and MO-P3-GOMEA are surrogate-free baselines and do not depend on the gate. Reference points (0.0900, 182.8698) on inference and (0.0900, 575.2847) on training.

Surrogate	Gate (η_a/η_e)	Inference		Training	
		HV	IGD+	HV	IGD+
SVR joint	joint 0.9	142.27	0.171	296.41	20.861
SVR joint	joint 0.99	142.22	0.116	293.87	21.939
SVR joint	joint 0.999	139.48	1.231	294.50	21.776
SVR joint	0.9/0.9	142.83	0.175	309.36	14.813
SVR joint	0.9/0.99	142.61	0.076	350.75	0.309
SVR joint	0.9/0.999	141.92	0.180	348.51	0.609
SVR joint	0.99/0.9	142.79	0.376	292.89	21.945
SVR joint	0.99/0.99	142.80	0.165	293.93	21.940
SVR joint	0.99/0.999	141.87	0.167	349.90	0.466
SVR joint	0.999/0.9	142.68	0.559	292.22	21.947
SVR joint	0.999/0.99	142.66	0.539	292.31	21.947
SVR joint	0.999/0.999	142.15	0.372	293.86	21.830
SVR per-obj	joint 0.9	142.03	0.673	324.95	7.725
SVR per-obj	joint 0.99	141.76	0.694	313.10	12.860
SVR per-obj	joint 0.999	139.62	1.352	299.00	17.582
SVR per-obj	0.9/0.9	142.13	0.824	341.14	2.428
SVR per-obj	0.9/0.99	142.53	0.382	353.44	0.131
SVR per-obj	0.9/0.999	141.99	0.300	354.50	0.008
SVR per-obj	0.99/0.9	141.34	1.396	298.41	19.548
SVR per-obj	0.99/0.99	141.95	0.959	330.98	7.068
SVR per-obj	0.99/0.999	141.24	0.534	354.36	0.008

Surrogate	Gate (η_a/η_e)	Inference		Training	
		HV	IGD+	HV	IGD+
SVR per-obj	0.999/0.9	140.88	1.547	296.12	19.213
SVR per-obj	0.999/0.99	140.90	1.547	296.51	18.702
SVR per-obj	0.999/0.999	141.38	1.101	316.38	10.284
MLP joint	joint 0.9	142.74	0.282	341.60	2.225
MLP joint	joint 0.99	141.15	0.743	337.33	2.925
MLP joint	joint 0.999	140.79	0.849	332.38	4.261
MLP joint	0.9/0.9	142.51	0.387	341.97	2.086
MLP joint	0.9/0.99	141.89	0.451	343.74	1.863
MLP joint	0.9/0.999	141.51	0.439	342.43	1.989
MLP joint	0.99/0.9	142.42	0.474	340.21	2.150
MLP joint	0.99/0.99	141.71	0.521	338.77	2.429
MLP joint	0.99/0.999	140.97	0.837	344.25	1.440
MLP joint	0.999/0.9	142.35	0.429	336.07	3.300
MLP joint	0.999/0.99	142.01	0.597	324.84	7.157
MLP joint	0.999/0.999	140.06	1.157	332.17	3.349
MLP per-obj	joint 0.9	139.40	1.325	319.07	10.767
MLP per-obj	joint 0.99	138.77	1.550	297.17	19.447
MLP per-obj	joint 0.999	139.97	1.072	289.71	21.949
MLP per-obj	0.9/0.9	139.37	1.414	332.24	5.310
MLP per-obj	0.9/0.99	139.18	1.227	305.58	14.571
MLP per-obj	0.9/0.999	139.95	0.918	289.71	21.949
MLP per-obj	0.99/0.9	139.53	1.338	313.18	12.478
MLP per-obj	0.99/0.99	138.77	1.550	295.21	19.961
MLP per-obj	0.99/0.999	140.67	0.724	289.71	21.949
MLP per-obj	0.999/0.9	139.41	1.395	308.59	14.551
MLP per-obj	0.999/0.99	138.77	1.550	290.05	21.948
MLP per-obj	0.999/0.999	140.90	0.862	289.71	21.949
RF joint	joint 0.9	142.05	0.498	306.22	16.961
RF joint	joint 0.99	142.02	0.550	300.46	19.374
RF joint	joint 0.999	139.73	1.247	297.18	20.613
RF joint	0.9/0.9	142.69	0.267	343.21	2.430
RF joint	0.9/0.99	142.22	0.432	351.74	0.342
RF joint	0.9/0.999	142.12	0.497	354.35	0.028
RF joint	0.99/0.9	141.58	0.794	299.05	19.305
RF joint	0.99/0.99	141.97	0.567	309.02	15.760
RF joint	0.99/0.999	141.97	0.539	354.10	0.033
RF joint	0.999/0.9	141.43	0.934	293.68	21.943
RF joint	0.999/0.99	141.52	0.831	293.75	21.878
RF joint	0.999/0.999	141.84	0.613	296.14	21.641

Surrogate	Gate (η_a/η_e)	Inference		Training	
		HV	IGD+	HV	IGD+
RF per-obj	joint 0.9	142.17	0.484	332.42	5.829
RF per-obj	joint 0.99	141.84	0.628	315.41	12.401
RF per-obj	joint 0.999	139.89	1.296	305.80	16.189
RF per-obj	0.9/0.9	142.70	0.281	344.91	1.663
RF per-obj	0.9/0.99	142.25	0.426	352.52	0.222
RF per-obj	0.9/0.999	141.35	0.742	354.48	0.022
RF per-obj	0.99/0.9	142.22	0.640	320.18	11.703
RF per-obj	0.99/0.99	141.89	0.673	333.41	6.638
RF per-obj	0.99/0.999	141.72	0.750	354.43	0.009
RF per-obj	0.999/0.9	141.69	1.041	297.55	20.436
RF per-obj	0.999/0.99	141.95	0.757	296.20	20.903
RF per-obj	0.999/0.999	141.71	0.782	323.83	9.806
GBoost joint	joint 0.9	142.21	0.472	323.50	10.024
GBoost joint	joint 0.99	141.77	0.566	304.94	17.904
GBoost joint	joint 0.999	139.07	1.489	305.18	17.056
GBoost joint	0.9/0.9	142.13	0.573	339.46	3.038
GBoost joint	0.9/0.99	142.22	0.470	351.12	0.444
GBoost joint	0.9/0.999	140.19	0.972	352.46	0.131
GBoost joint	0.99/0.9	141.70	0.801	297.30	20.256
GBoost joint	0.99/0.99	141.72	0.736	316.10	14.540
GBoost joint	0.99/0.999	140.21	0.963	352.90	0.107
GBoost joint	0.999/0.9	141.39	1.109	294.07	21.857
GBoost joint	0.999/0.99	141.54	0.931	293.84	21.943
GBoost joint	0.999/0.999	140.56	0.933	314.33	14.677
GBoost per-obj	joint 0.9	142.15	0.446	336.35	3.971
GBoost per-obj	joint 0.99	142.16	0.379	327.83	8.595
GBoost per-obj	joint 0.999	139.03	1.451	332.86	5.297
GBoost per-obj	0.9/0.9	142.70	0.311	346.20	1.381
GBoost per-obj	0.9/0.99	142.29	0.449	353.25	0.128
GBoost per-obj	0.9/0.999	141.58	0.549	353.24	0.012
GBoost per-obj	0.99/0.9	141.81	0.836	317.33	12.455
GBoost per-obj	0.99/0.99	142.01	0.634	342.16	2.959
GBoost per-obj	0.99/0.999	141.19	0.664	353.46	0.012
GBoost per-obj	0.999/0.9	141.57	0.943	294.62	21.318
GBoost per-obj	0.999/0.99	141.57	0.852	297.80	20.767
GBoost per-obj	0.999/0.999	141.30	0.749	330.89	9.093
MO-LS	n/a	140.23	1.113	294.23	16.155
MO-P3-GOMEA	n/a	138.85	1.522	280.29	16.141

Figures 7.3 and 7.4 present the per-axis grid as heatmaps. On training the strongest cells are located at $\eta_a = 0.9$ and $\eta_e = 0.999$, reaching the range of the probabilistic surrogates, while inference shows little variation across the grid.

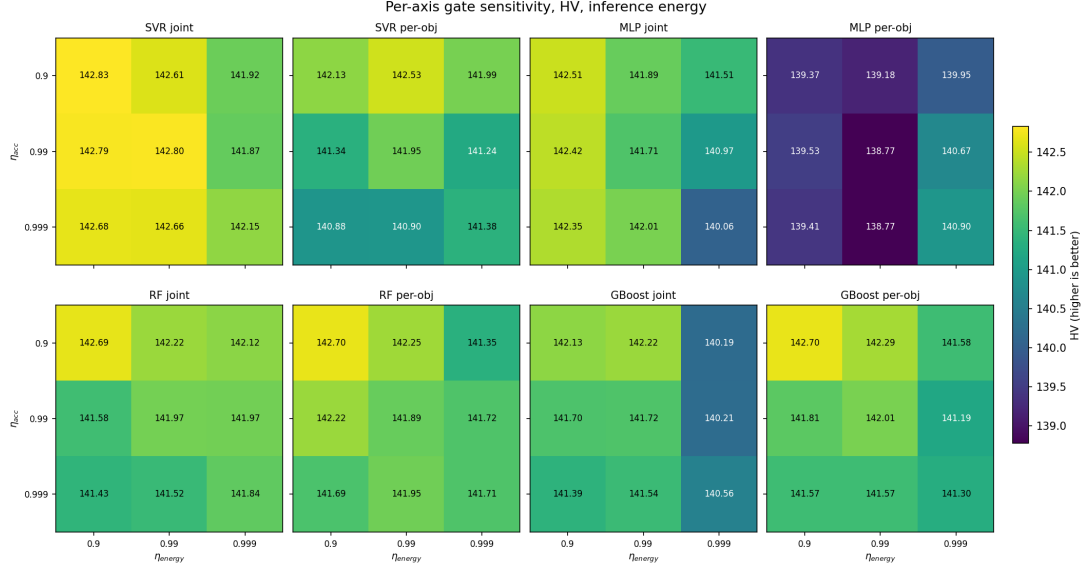


Fig. 7.3. Per-axis gate sensitivity for the deterministic surrogates on inference energy. Mean hypervolume over 10 seeds across the nine per-axis pairs (η_a, η_e) .

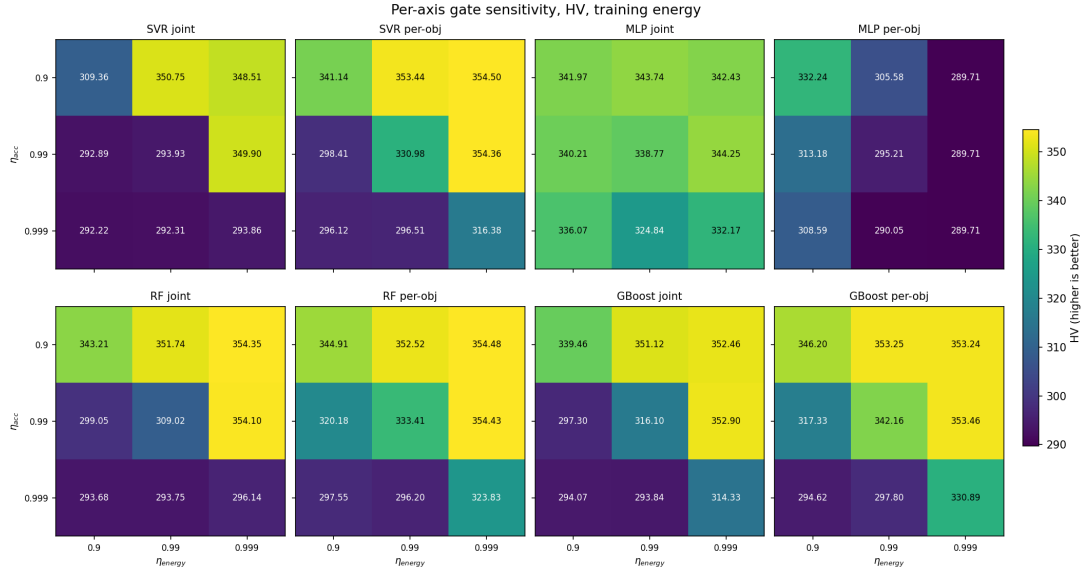


Fig. 7.4. Per-axis gate sensitivity for the deterministic surrogates on training energy. Mean hypervolume over 10 seeds across the nine per-axis pairs (η_a, η_e) .

7.3.2. MO-DSA-P3-GOMEA with online surrogate selection

Table 7.5 reports dynamic selection over the deterministic pool, across the twelve schedules formed by the kept fraction $k \in \{12, 25, 50, 100\}$ percent and the re-selection interval

$\{25, 50, 100\}$ real evaluations. HV and IGD+ use the same reference points and oracle fronts as the fixed-surrogate tables.

On inference every schedule falls between 139.05 and 139.81 HV, close to the fixed deterministic surrogates and below the best of them, the joint MLP at 141.16. On training the kept fraction determines the outcome. At $k = 100$ the variant reaches 342.69 HV with a small spread. At smaller fractions it falls to between 314 and 339, with standard deviations as large as 33. On inference no schedule reaches MO-LS (140.23 against the best schedule’s 139.81). On training every schedule surpasses both baselines (MO-P3-GOMEA 280.29, MO-LS 294.23) by a wide margin. Figures 7.5 and 7.6 show the schedule overlays.

Schedule		Inference		Training	
k [%]	interval	HV	IGD+	HV	IGD+
12	25	139.81 ± 1.22	1.229 ± 0.564	330.03 ± 24.10	6.992 ± 10.140
12	50	139.69 ± 1.33	1.321 ± 0.532	336.25 ± 14.24	4.135 ± 4.785
12	100	139.35 ± 1.32	1.287 ± 0.709	339.18 ± 13.86	3.218 ± 4.501
25	25	139.23 ± 1.45	1.460 ± 0.706	319.00 ± 29.88	11.485 ± 13.396
25	50	139.44 ± 1.60	1.325 ± 0.690	325.35 ± 32.44	9.804 ± 13.593
25	100	139.05 ± 1.52	1.463 ± 0.678	331.82 ± 16.80	5.461 ± 6.208
50	25	139.23 ± 1.34	1.408 ± 0.624	314.57 ± 32.82	13.426 ± 14.669
50	50	139.26 ± 1.41	1.369 ± 0.637	328.69 ± 23.13	7.073 ± 9.728
50	100	139.35 ± 1.31	1.398 ± 0.627	329.53 ± 22.90	6.787 ± 9.932
100	25	139.59 ± 1.16	1.243 ± 0.598	341.18 ± 7.09	2.087 ± 2.059
100	50	139.17 ± 1.59	1.400 ± 0.732	342.69 ± 5.77	1.347 ± 1.080
100	100	139.44 ± 1.14	1.303 ± 0.641	340.71 ± 14.96	2.999 ± 5.097
MO-P3-GOMEA		138.85 ± 1.09	1.522 ± 0.708	280.29 ± 35.87	16.141 ± 11.183
MO-LS		140.23 ± 1.19	1.113 ± 0.573	294.23 ± 27.40	16.155 ± 12.713

Table 7.5. Dynamic surrogate selection over the deterministic pool, mean $\pm$ std over 10 seeds. Best HV per problem in bold.

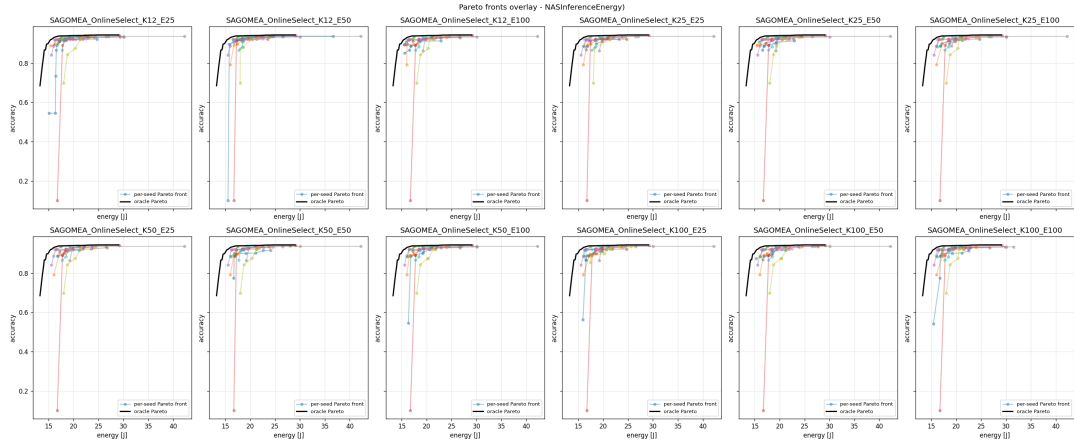


Fig. 7.5. Pareto-front overlay across the online selection runs over the deterministic pool for the inference-energy problem.

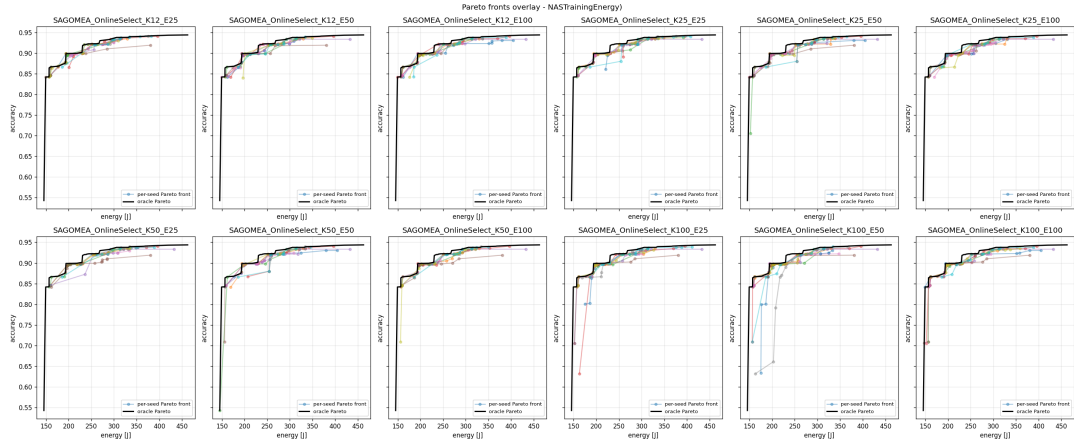


Fig. 7.6. Pareto-front overlay across the online selection runs over the deterministic pool for the training-energy problem.

7.4. MO-PSA-P3-GOMEA

Table 7.6 shows an improvement over the deterministic surrogates. On inference every probabilistic variant outperforms every deterministic one. On training, per-objective gradient boosting attains the highest hypervolume, 353.75, and the lowest IGD+, 0.010, against 347.59 HV and 0.822 for the best deterministic surrogate. The joint MLP is the weakest probabilistic variant here, at 345.96 HV and 1.450 IGD+, so its Monte Carlo dropout uncertainty is unreliable on this problem. Every probabilistic variant still outperforms both baselines (MO-P3-GOMEA at 138.85 and 280.29, MO-LS at 140.23 and 294.23). Their archives are also larger, 10 to 14 points on inference and 14 to 20 on training against 8 to 11 for the deterministic surrogates. Figures 7.7 and 7.8 show the per-surrogate overlays.

The full per-algorithm report for both families and the two baselines, with overlays, HV and IGD+ boxplots over the seeds, and summary tables, is in Appendix A.

Surrogate	Inference		Training	
	HV	IGD+	HV	IGD+
RF joint	142.91 ± 0.66	0.253 ± 0.188	351.80 ± 2.49	0.348 ± 0.275
RF per-obj	142.53 ± 0.59	0.418 ± 0.215	353.72 ± 1.31	0.106 ± 0.050
GBoost joint	142.03 ± 0.75	0.479 ± 0.284	351.46 ± 2.93	0.392 ± 0.361
GBoost per-obj	142.45 ± 0.64	0.317 ± 0.187	353.75 ± 1.64	0.010 ± 0.003
MLP joint	143.03 ± 0.41	0.211 ± 0.093	345.96 ± 4.23	1.450 ± 0.981
MLP per-obj	143.06 ± 0.29	0.212 ± 0.084	351.52 ± 3.50	0.464 ± 0.466
Gaussian process per-obj	142.02 ± 0.89	0.278 ± 0.266	352.51 ± 2.03	0.073 ± 0.070
MO-P3-GOMEA	138.85 ± 1.09	1.522 ± 0.708	280.29 ± 35.87	16.141 ± 11.183
MO-LS	140.23 ± 1.19	1.113 ± 0.573	294.23 ± 27.40	16.155 ± 12.713

Table 7.6. Probabilistic surrogates, mean $\pm$ deviation over 10 seeds. Best HV and IGD+ per problem is bold.

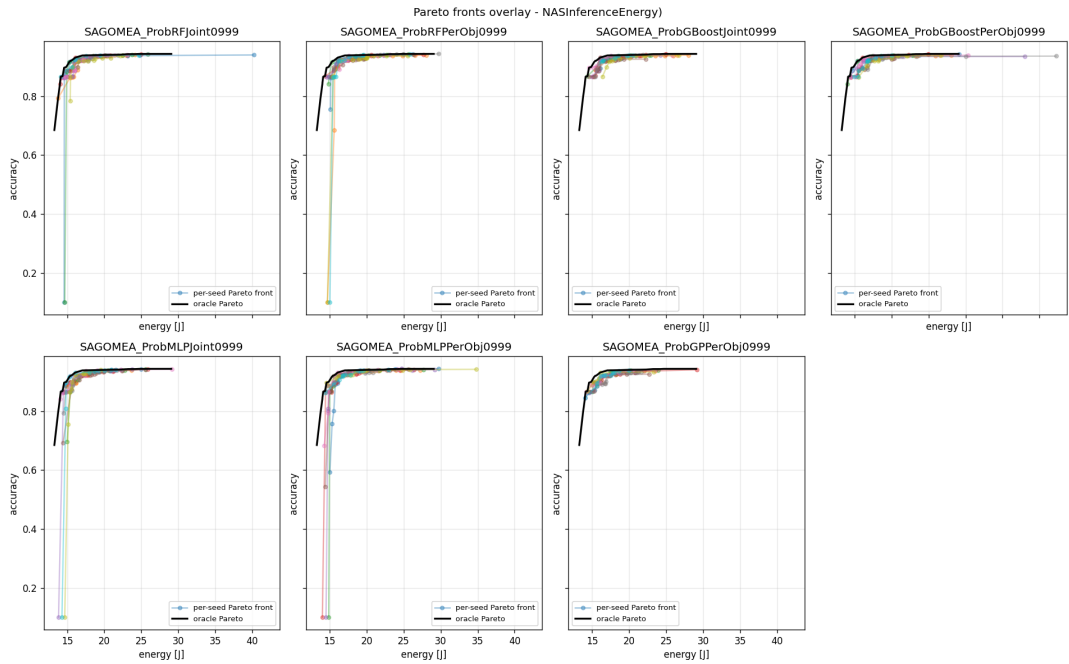


Fig. 7.7. Pareto-front overlay across the probabilistic surrogates for the inference-energy problem.

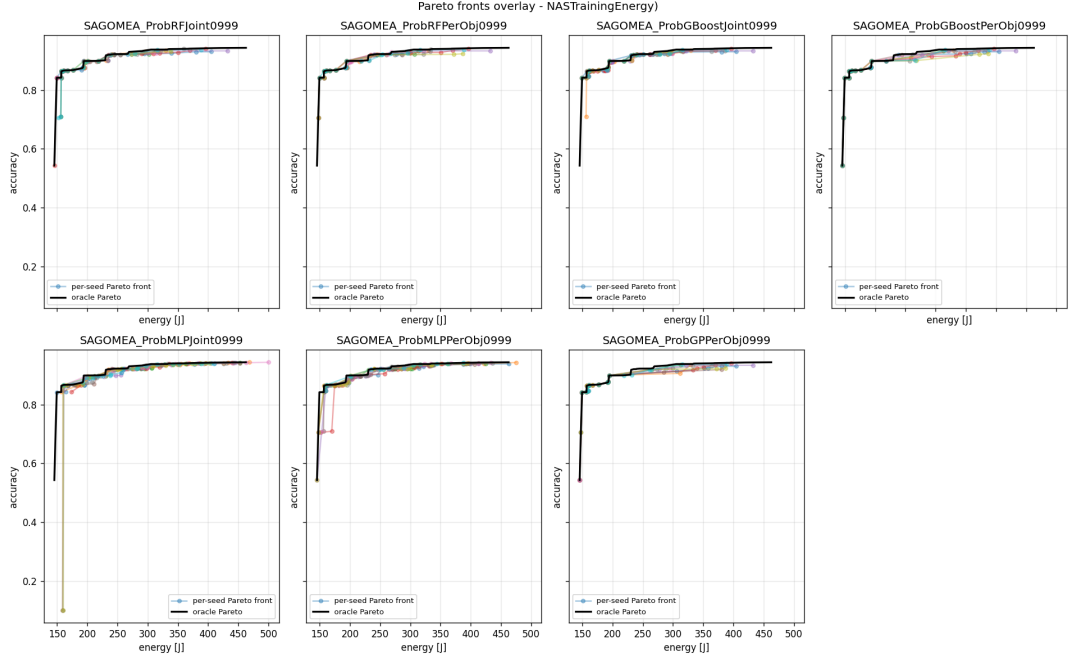


Fig. 7.8. Pareto-front overlay across the probabilistic surrogates for the training-energy problem.

7.4.1. Sensitivity to the gate relaxation rate

The study above fixes the gate relaxation rate at $\eta = 0.999$, inherited from SA-P3-GOMEA. Probabilistic acceptance already loosens the gate where the surrogate is uncertain, so the relaxation rate interacts with the acceptance rule and deserves a separate look.

The relaxation rate matters mainly on training energy. There, $\eta = 0.99$ and $\eta = 0.999$ give higher hypervolume and lower IGD+ than $\eta = 0.9$. Joint gradient boosting goes from 0.392 IGD+ at $\eta = 0.999$ to 1.168 at $\eta = 0.9$, and the two strongest training surrogates, per-objective gradient boosting and per-objective MLP, reach their lowest IGD+, 0.010 and 0.464, at $\eta = 0.999$. On inference the quantile threshold removes most of this dependence, and no rate is uniformly best. The highest hypervolume falls to $\eta = 0.9$ for four of the seven surrogates and to $\eta = 0.99$ or $\eta = 0.999$ for the others, yet every inference value is placed between 141.9 and 143.3 HV.

Table 7.7: Gate relaxation rate for the probabilistic surrogates on the per-axis-gate campaign, mean over 10 seeds. Each surrogate runs under the nine per-axis accuracy and energy rate pairs (η_a/η_e). Best HV and lowest IGD+ per surrogate and problem in bold. Reference points (0.0900, 182.8698) on inference and (0.0900, 575.2847) on training.

Surrogate	Gate (η_a/η_e)	Inference		Training	
		HV	IGD+	HV	IGD+
RF joint	0.9/0.9	142.70	0.293	340.87	2.725
	0.9/0.99	142.87	0.226	351.71	0.324
	0.9/0.999	143.13	0.091	354.12	0.073
	0.99/0.9	142.91	0.205	337.51	3.383
	0.99/0.99	142.87	0.188	344.06	1.698
	0.99/0.999	142.62	0.254	354.02	0.148
	0.999/0.9	143.07	0.226	345.66	1.628
	0.999/0.99	142.68	0.330	344.58	1.435
	0.999/0.999	142.58	0.353	353.23	0.145
RF per-obj	0.9/0.9	142.88	0.239	345.48	1.546
	0.9/0.99	142.80	0.222	351.22	0.490
	0.9/0.999	142.86	0.191	354.21	0.032
	0.99/0.9	142.75	0.270	342.22	2.132
	0.99/0.99	142.88	0.259	349.12	0.766
	0.99/0.999	142.92	0.191	354.61	0.021
	0.999/0.9	143.33	0.208	341.18	2.249
	0.999/0.99	142.77	0.375	343.59	1.980
	0.999/0.999	142.59	0.394	353.06	0.186
GBoost joint	0.9/0.9	142.74	0.268	344.44	1.955
	0.9/0.99	142.94	0.182	352.43	0.266
	0.9/0.999	142.45	0.271	354.40	0.026
	0.99/0.9	142.21	0.448	331.14	5.297
	0.99/0.99	142.49	0.344	348.61	1.014
	0.99/0.999	142.29	0.277	354.47	0.020
	0.999/0.9	142.44	0.452	333.20	5.290
	0.999/0.99	142.47	0.424	325.45	7.009
	0.999/0.999	142.05	0.498	350.09	0.544
GBoost per-obj	0.9/0.9	142.54	0.314	345.09	1.791
	0.9/0.99	142.42	0.244	353.45	0.131
	0.9/0.999	142.64	0.152	353.21	0.013
	0.99/0.9	142.73	0.308	338.73	2.715
	0.99/0.99	142.51	0.404	351.77	0.420

Surrogate	Gate (η_a/η_e)	Inference		Training	
		HV	IGD+	HV	IGD+
	0.99/0.999	142.38	0.232	353.48	0.012
	0.999/0.9	142.62	0.398	340.76	2.550
	0.999/0.99	142.48	0.390	341.44	2.731
	0.999/0.999	142.48	0.298	354.01	0.016
MLP joint	0.9/0.9	143.28	0.158	345.58	1.267
	0.9/0.99	143.20	0.185	345.55	1.319
	0.9/0.999	143.29	0.158	340.52	2.812
	0.99/0.9	143.08	0.218	343.08	1.461
	0.99/0.99	143.19	0.187	342.32	2.211
	0.99/0.999	143.00	0.188	343.43	2.122
	0.999/0.9	143.17	0.145	343.62	1.651
	0.999/0.99	142.86	0.270	335.25	3.882
	0.999/0.999	143.08	0.165	340.18	2.382
MLP per-obj	0.9/0.9	143.08	0.242	348.30	1.135
	0.9/0.99	143.13	0.201	346.10	1.365
	0.9/0.999	142.93	0.177	347.59	1.213
	0.99/0.9	143.27	0.185	342.78	2.081
	0.99/0.99	143.17	0.139	349.21	0.722
	0.99/0.999	143.11	0.192	349.69	0.702
	0.999/0.9	143.11	0.242	342.87	2.028
	0.999/0.99	143.19	0.187	345.09	1.487
	0.999/0.999	142.85	0.257	349.96	0.692
Gaussian process per-obj	0.9/0.9	142.69	0.181	346.45	1.204
	0.9/0.99	142.80	0.131	353.55	0.153
	0.9/0.999	142.49	0.171	353.28	0.012
	0.99/0.9	142.58	0.291	339.78	2.772
	0.99/0.99	142.64	0.145	351.52	0.310
	0.99/0.999	142.72	0.101	353.29	0.012
	0.999/0.9	142.58	0.362	336.20	3.542
	0.999/0.99	142.44	0.336	345.95	1.277
	0.999/0.999	142.74	0.216	352.68	0.046

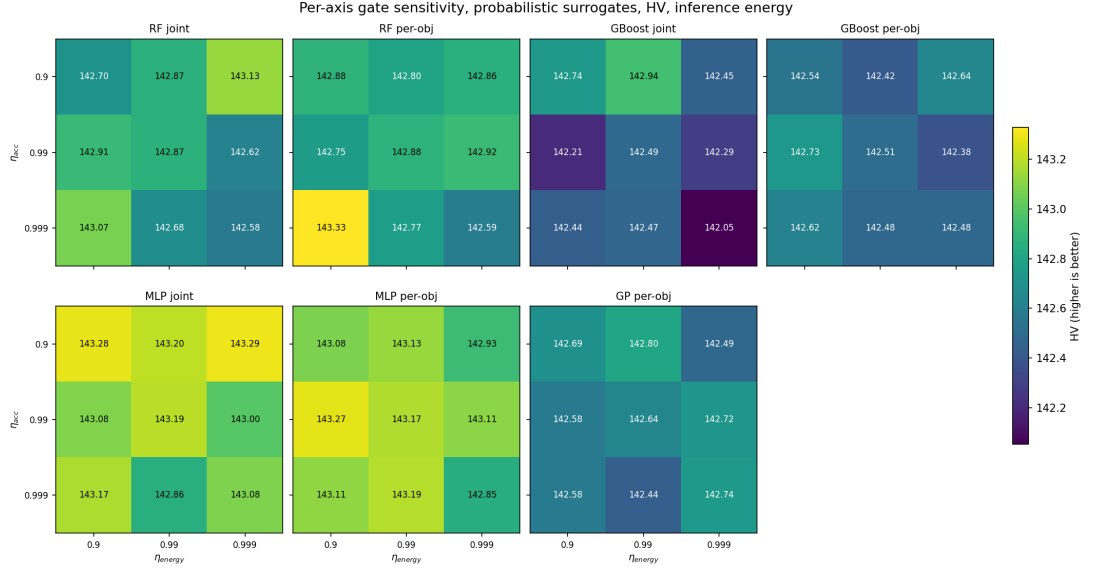


Fig. 7.9. Per-axis gate sensitivity for the probabilistic surrogates on inference energy. Mean hyper-volume over 10 seeds across the nine per-axis pairs (η_a, η_e) .

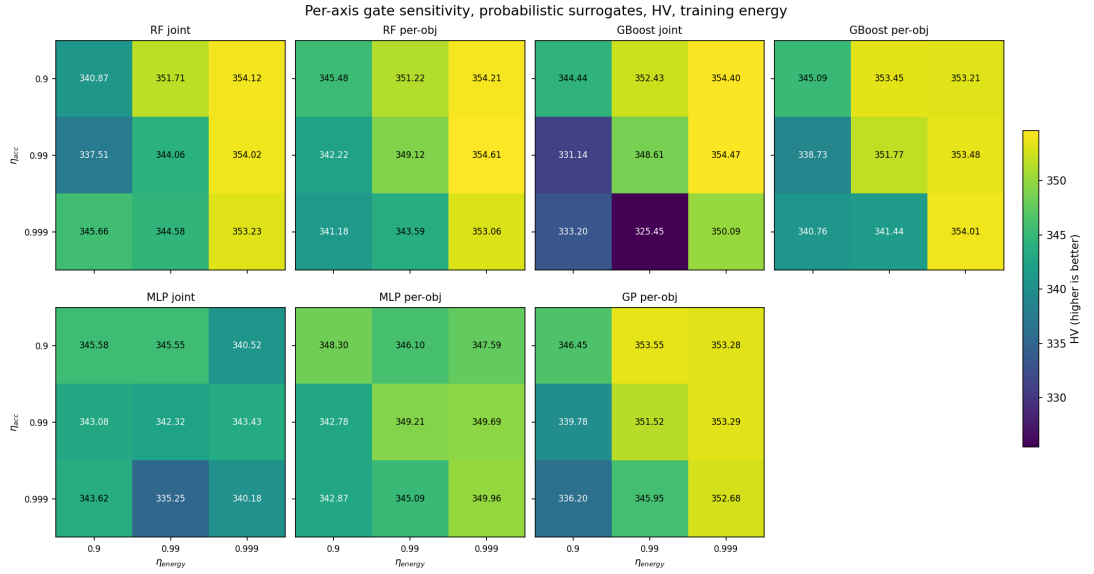


Fig. 7.10. Per-axis gate sensitivity for the probabilistic surrogates on training energy. Mean hyper-volume over 10 seeds across the nine per-axis pairs (η_a, η_e) .

7.4.2. MO-PSA-P3-GOMEA with online surrogate selection

Table 7.8 reports dynamic selection over the probabilistic pool under the same twelve schedules as before. On training the top 12 percent reaches 353.98 HV at the shortest interval, every schedule with $k \leq 50$ is between 352.7 and 354.0 HV, and the full ensemble at $k = 100$ is between 347.7 and 349.9 HV, with IGD+ between 0.49 and 0.69. On inference every schedule is between 142.0 and 142.5 HV, the highest 142.49 ($k = 12$, longest interval). The best training result is 353.98 HV, against 353.75 for per-objective gradient boosting,

and the best inference result is 142.49 HV, against 143.06 for per-objective MLP. For $k \leq 50$ the training standard deviation is near 1.5, against up to 33 for the deterministic pool. Every schedule exceeds both baselines on both problems (MO-P3-GOMEA 138.85 and 280.29, MO-LS 140.23 and 294.23). Figures 7.11 and 7.12 show the schedule overlays.

Schedule		Inference		Training	
k [%]	interval	HV	IGD+	HV	IGD+
12	25	142.07 $\pm$ 1.09	0.350 $\pm$ 0.216	353.98 $\pm$ 1.31	0.090 $\pm$ 0.067
12	50	142.16 $\pm$ 0.76	0.309 $\pm$ 0.165	353.13 $\pm$ 1.88	0.174 $\pm$ 0.192
12	100	142.49 $\pm$ 0.54	0.228 $\pm$ 0.148	353.95 $\pm$ 1.08	0.101 $\pm$ 0.057
25	25	142.25 $\pm$ 0.87	0.321 $\pm$ 0.146	352.88 $\pm$ 2.08	0.128 $\pm$ 0.065
25	50	142.31 $\pm$ 0.38	0.419 $\pm$ 0.170	353.19 $\pm$ 1.14	0.149 $\pm$ 0.042
25	100	142.33 $\pm$ 0.53	0.403 $\pm$ 0.142	353.47 $\pm$ 1.60	0.086 $\pm$ 0.058
50	25	142.44 $\pm$ 0.78	0.412 $\pm$ 0.271	353.32 $\pm$ 1.24	0.138 $\pm$ 0.048
50	50	142.23 $\pm$ 0.50	0.446 $\pm$ 0.204	353.33 $\pm$ 1.71	0.091 $\pm$ 0.055
50	100	142.39 $\pm$ 0.56	0.357 $\pm$ 0.188	352.71 $\pm$ 1.77	0.144 $\pm$ 0.054
100	25	142.01 $\pm$ 0.73	0.361 $\pm$ 0.242	349.86 $\pm$ 3.09	0.494 $\pm$ 0.378
100	50	142.08 $\pm$ 0.67	0.397 $\pm$ 0.289	349.19 $\pm$ 3.29	0.515 $\pm$ 0.471
100	100	142.12 $\pm$ 0.74	0.278 $\pm$ 0.192	347.68 $\pm$ 3.67	0.692 $\pm$ 0.418
MO-P3-GOMEA		138.85 $\pm$ 1.09	1.522 $\pm$ 0.708	280.29 $\pm$ 35.87	16.141 $\pm$ 11.183
MO-LS		140.23 $\pm$ 1.19	1.113 $\pm$ 0.573	294.23 $\pm$ 27.40	16.155 $\pm$ 12.713

Table 7.8. Dynamic surrogate selection over the probabilistic pool, mean $\pm$ std over 10 seeds. Best HV per problem in bold.

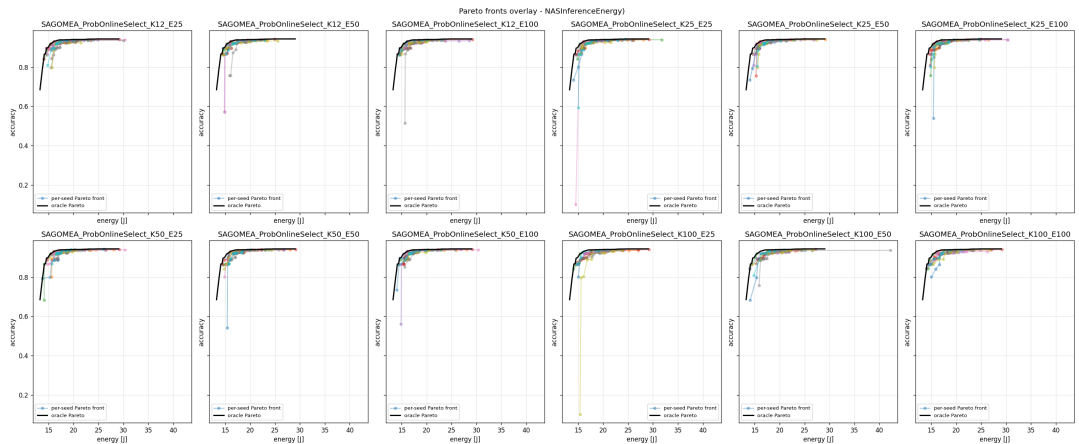


Fig. 7.11. Pareto-front overlay across the dynamic-selection schedules over the probabilistic pool for the inference-energy problem.

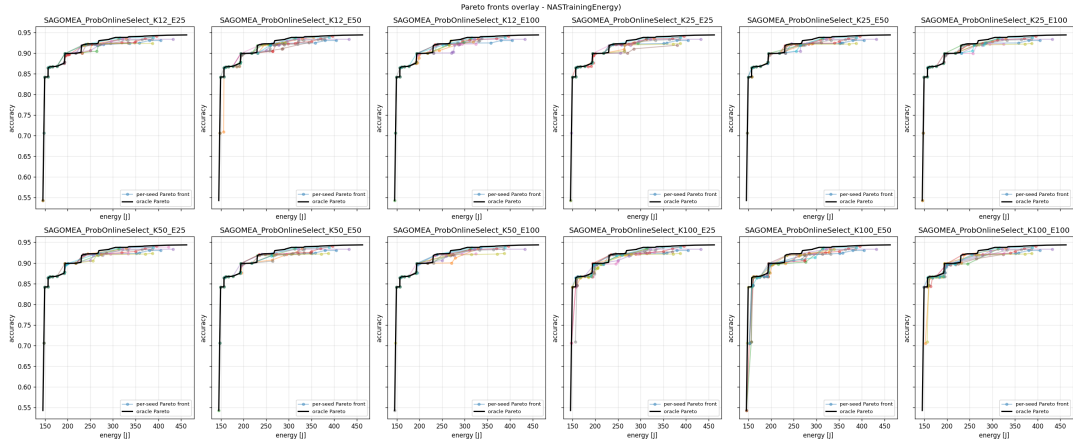


Fig. 7.12. Pareto-front overlay across the dynamic-selection schedules over the probabilistic pool for the training-energy problem.

7.5. FINDINGS

The aim of the experiments was to answer the questions set in Objectives section. The first asked whether a surrogate-assisted parameterless GOMEA can approximate the accuracy and energy Pareto front at a lower evaluation cost than the same search without a surrogate. Within the fixed budget of 500 real evaluations the surrogate achieves this on training energy, where the strongest variants reach about 354 hypervolume against 294.23 for multi-objective local search and 280.29 for MO-P3-GOMEA. On inference energy local search reaches 140.23, and only the two MLP surrogates exceed it.

The second question asked whether probabilistic surrogates, dynamic surrogate selection and automated hyperparameter tuning improve on a fixed deterministic surrogate. The probabilistic surrogates outperform the deterministic ones on training energy, where they reach 351.5 to 353.8 hypervolume with IGD+ as low as 0.010, while the best deterministic surrogate reaches 350.29 and the rest fall between 294 and 333. On inference the probabilistic surrogates lead only on IGD+, and the best deterministic surrogate reaches their hypervolume once its relaxation rate is tuned. Modelling each objective with its own surrogate is consistently better than a single joint model on training energy, across every surrogate type.

Dynamic selection of the surrogate type matches the best fixed surrogate without choosing it in advance, but only over the probabilistic pool, where a small kept fraction reaches 353.98 hypervolume against 353.75 for the best fixed type. Over the deterministic pool the same mechanism does not improve committing to the single best type. Automated tuning of the relaxation rate matters most for the deterministic surrogates and least for the probabilistic ones on inference.

$\eta = 0.999$ is the worst of the three rates tested for the deterministic surrogates, and the loosest rate $\eta = 0.9$ ranks best or nearly best for almost every variant on both problems. The overall comparison uses $\eta = 0.999$ and understates the deterministic surrogates on inference energy. Loosening the rate raises the strongest variant to 142.74 HV, into the range of the probabilistic surrogates. The probabilistic surrogates loosen the gate through their quantile acceptance wherever the model is uncertain. On inference energy the fixed rate has little effect, and all values remain between 141.9 and 143.3 HV. On training energy the tighter rates perform better, and the inherited $\eta = 0.999$ yields the lowest IGD+ for the two strongest training surrogates, down to 0.010 for per-objective gradient boosting. The deterministic surrogate must re-tune the relaxation rate for the multi-objective problem, while the probabilistic surrogate performs well at the rate inherited from the single-objective method.

8. CONCLUSION

8.1. WORK

This thesis adapted the SA-P3-GOMEA of Dushatskiy et al. [20], built for the single-objective partition-based ensemble learning, into a MO algorithm for NAS. Each architecture in NAS-Bench-201 [23] search space was encoded as six decision variables over an alphabet of five operators, which gives 15625 candidate networks. The first objective was CIFAR-10 classification accuracy taken from the benchmark. The second was energy consumption measured on the GPU that ran the evaluation. Thesis studied the problems of inference and training energy consumption against accuracy.

Thesis added three surrogate mechanisms. Firstly, the mechanism of online surrogate selection answered the question whether it is worth keeping the fixed surrogate at all. Online selection keeps a pool of surrogate types and scores each type by rank correlation on a fixed number of last entries to the archive at fixed intervals. Moreover, the surrogate hyperparameters are tuned during the run with a small grid search. Finally, the thesis introduces probabilistic surrogate prediction of distribution and acceptance adjusted to utilise uncertainty too. All proposed algorithm - MO-DSA-P3-GOMEA, MO-PSA-P3-GOMEA and variants with online selection - were compared against surrogate-free baselines, MO-GOMEA and MO-LS. Baselines were adapted from code provided by authors of the original work.

8.2. DISCUSSION

The surrogate increases hypervolume on training energy and has a marginal effect on inference. The size of the search space can be one explanation of this difference. The search space contains 15625 architectures. The budget of 500 real evaluations is 3.2 percent of the space, and the offline pretraining set of 100 is 0.64 percent. The surrogate-free local search reaches 140.23 HV on inference energy, and the best deterministic surrogate reaches 142.40. The eight deterministic variants span 3.4 HV. The baselines reach 294.23 and 280.29 on training energy, and the surrogate variants exceed 350. The size also requires a termination rule the single-objective SA-P3-GOMEA did not need. The OM operator continues to produce unseen genotypes in a large space, but the pyramid converges to a small region in the NAS-Bench-201 space. The remaining donors recombine into architectures already evaluated, and the number of real evaluations remains constant while the outer loop continues. The run therefore terminates after 1000 outer iterations without a new

evaluation, before it reaches the evaluation budget. Enumeration over the whole space yields an exact reference front, so the hypervolume and IGD+ values are exact.

Moreover, the choice of surrogate and configuration determines the outcome on training energy. One model per objective outperforms a single joint model on training energy for every family, by 27.7 HV for gradient boosting and 2.5 HV for the MLP. Two independent models each fit their own target, while a joint model compromises between two objectives of different shape. The MLP is affected least in its joint form. It shares two hidden layers across the objectives and has a separate output for each, so the joint MLP performs almost as well as the per-objective form. The MLP also reaches the highest inference hypervolume in both families, though the deterministic variants differ by 3.4 HV on inference. Its higher training result at the inherited rate is misleading. That rate is the worst setting for the other surrogates, and under a per-axis gate the tree and kernel surrogates reach 353 to 354 HV and exceed the MLP at about 344. Two results limit any conclusion based on the MLP alone. The joint probabilistic MLP is the weakest probabilistic variant on training energy, at 345.96 HV, and its Monte Carlo dropout spread does not track the real error. Its inference hypervolume also differs between two campaigns, 142.40 against 140.79, from the run-to-run noise of network training.

The relaxation rate was inherited as $\eta = 0.999$ from the original SA-P3-GOMEA, and the results show it is not a neutral default. At a single shared rate the loosest value $\eta = 0.9$ is the best of the three for the deterministic surrogates on both problems, which suggests that a liberal gate suits the energy problem. Setting a separate rate on each axis refines this result. A strict rate on an axis makes the gate rely on that objective’s prediction when it selects candidates for real evaluation, and the two predictions are not equally reliable. A strict accuracy rate lowers the result. The training hypervolume falls to 292 to 296 for the deterministic surrogates at $\eta_a = 0.999$, for every energy rate. The accuracy surrogate is unreliable on this space and directs the real evaluations to a region it has misjudged. A strict energy rate raises the result once the accuracy rate is relaxed. A tighter energy rate at $\eta_a = 0.9$ raises SVR joint from 309.36 HV to 350.75 and random forest joint to 354.35, and the best configurations use a strict energy rate. The energy objective therefore benefits from a strict gate, and the accuracy objective from a liberal one. A single shared rate applies the same reliance to both objectives and cannot separate them, so only the per-axis gate reaches the training front. The source does not measure which objective is easier to predict, so this reading rests on the gate results. The per-objective rank-correlation scores the surrogate already computes would confirm it directly.

The inherited rate has the opposite effect for the probabilistic surrogates, for which $\eta = 0.999$ is the best or near-best setting on training energy. A comparison that fixes it overstates their advantage. Their advantage is clear on training energy and narrower on inference once

the deterministic surrogates use their preferred rate. Their behaviour follows from how the acceptance rule uses uncertainty. The rule admits a candidate when its predicted spread makes it difficult to reject, so a wider spread admits more candidates. These surrogates therefore maintain larger and better-spread archives, 14 to 20 points on training against 8 to 11 for the deterministic surrogates, and the same margin explains why the relaxation rate matters less for them on inference. The rule treats the two objectives as independent and multiplies one Gaussian term per objective. Correlated prediction errors make the product understate the true dominance probability and leave the gate more conservative than intended. This conservatism is the likely reason these surrogates gain more from a larger archive than from a higher acceptance rate. An unreliable spread degrades the result, as the joint MLP shows on training energy. A spread of zero reduces the rule to strict Pareto dominance.

Several limits bound how far these results generalise. They rest on a single benchmark, NAS-Bench-201, and a single dataset, with CIFAR-10 accuracy taken from the benchmark table and not from retraining, so the conclusions describe this search space and not Neural Architecture Search in general. Energy is measured only at the GPU through NVML, which excludes CPU, memory and platform draw, so the second objective is the architecture’s GPU energy and not the whole systems. The inference-energy problem is also easy. The baselines reach the front without a surrogate, so this problem serves as a contrast case.

8.3. FUTURE WORK

The per-axis gate reached the training front. The two relaxation rates were determined by testing three values on each axis, and the algorithm did not choose them. These results indicate a selection rule the algorithm could apply during the run. The gate should be strict on the objective the surrogate ranks well and loose on the objective the surrogate ranks poorly. Online selection already computes the rank correlation of each surrogate on a held-out part of the archive, separately for accuracy and energy. These two scores could set the two axis rates during the run. This removes the manual choice of rates and keeps the method parameterless. The rank correlation could set a continuous rate, so the gate follows the surrogate’s reliability as it shifts during the run. The same rule would extend to more than two objectives, with one rate per axis.

The probabilistic acceptance rule multiplies one Gaussian term per objective. It treats the accuracy and energy prediction errors as independent. Correlated errors make the product understate the true dominance probability, and the gate becomes too conservative. A rule that estimates the correlation and forms a bivariate dominance probability removes this bias. A surrogate that models the two objectives jointly can supply the correlation directly. The thesis excluded such a surrogate. The joint Gaussian process required a multi-output,

coregionalised kernel that was too costly at this problem scale. A study that accepts this cost can test whether the independence assumption limits the probabilistic surrogates. A less costly path would estimate the correlation from the residuals the run already collects, so the full joint model is not required. The measured correlation would also show whether independence applies to other search spaces.

The MLP surrogate produced the least reliable uncertainty estimate. Its joint probabilistic form was the weakest variant on training energy. The Monte Carlo dropout spread it reports did not track the real prediction error, and the acceptance rule that relies on this spread performs poorly. Dropout is an inexpensive approximation of the posterior. Deep ensembles and evidential regression estimate predictive uncertainty more accurately. Both provide the acceptance rule with a spread that follows the true error more closely. The uncertainty should also be evaluated by its calibration, not only by the resulting hypervolume. A surrogate can reach a good front while remaining poorly calibrated. The acceptance rule could then weight each surrogate by its measured calibration, so an unreliable spread has less effect. Calibration could also join the rank correlation as a second criterion in the online selection.

BIBLIOGRAPHY

- [1] W.S. McCulloch, W. Pitts, *A logical calculus of the ideas immanent in nervous activity*, The Bulletin of Mathematical Biophysics. 1943, volume 5, 4, pages 115–133.
- [2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A.N. Gomez, Ł. Kaiser, I. Polosukhin, *Attention is all you need*, in: *Advances in Neural Information Processing Systems 30 (NIPS 2017)* (Curran Associates, Inc., Red Hook, NY, USA, 2017), pages 5998–6008.
- [3] A.S. Luccioni, S. Viguier, A.L. Ligozat, *Estimating the carbon footprint of BLOOM, a 176B parameter language model*, Journal of Machine Learning Research. 2023, volume 24, 253, pages 1–15.
- [4] S. Luccioni, Y. Jernite, E. Strubell, *Power hungry processing: Watts driving the cost of ai deployment?*, in: *Proceedings of the 2024 ACM Conference on Fairness, Accountability, and Transparency*, FAccT '24 (Association for Computing Machinery, New York, NY, USA, 2024), page 85–99.
- [5] Y. Cheng, D. Wang, P. Zhou, T. Zhang, *Model compression and acceleration for deep neural networks: The principles, progress, and challenges*, IEEE Signal Processing Magazine. 2018, volume 35, 1, pages 126–136.
- [6] J. Tmamna, E.B. Ayed, R. Fourati, M. Gogate, T. Arslan, A. Hussain, M.B. Ayed, *Pruning deep neural networks for green energy-efficient models: A survey*, Cognitive Computation. 2024, volume 16, 6, pages 2931–2952.
- [7] T. Elsken, J.H. Metzen, F. Hutter, *Neural architecture search: A survey*, Journal of Machine Learning Research. 2019, volume 20, 55, pages 1–21.
- [8] D. Baymurzina, E. Golikov, M. Burtsev, *A review of neural architecture search*, Neurocomputing. 2022, volume 474, pages 82–93.
- [9] B. Zoph, Q.V. Le, *Neural architecture search with reinforcement learning*, in: *5th International Conference on Learning Representations (ICLR)* (2017).
- [10] E. Real, A. Aggarwal, Y. Huang, Q.V. Le, *Regularized evolution for image classifier architecture search*, Proceedings of the AAAI Conference on Artificial Intelligence. 2019, volume 33, 01, pages 4780–4789.
- [11] T. Baeck, D. Fogel, Z. Michalewicz, *Evolutionary Computation 1: Basic Algorithms and Operators*, Basic algorithms and operators (Institute of Physics Publishing, Bristol, United Kingdom, 2000).
- [12] S. Sharma, V. Kumar, *A comprehensive review on multi-objective optimization techniques: Past, present and future*, Archives of Computational Methods in Engineering. 2022, volume 29, 7, pages 5605–5633.
- [13] K. Deb, A. Pratap, S. Agarwal, T. Meyarivan, *A fast and elitist multiobjective genetic*

- algorithm: NSGA-II*, IEEE Transactions on Evolutionary Computation. 2002, volume 6, 2, pages 182–197.
- [14] Z. Lu, I. Whalen, V. Boddeti, Y. Dhebar, K. Deb, E. Goodman, W. Banzhaf, *NSGA-Net: Neural architecture search using multi-objective genetic algorithm*, in: *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO '19* (ACM, New York, NY, USA, 2019), pages 419–427.
 - [15] T. Elsken, J.H. Metzen, F. Hutter, *Efficient multi-objective neural architecture search via Lamarckian evolution*, in: *International Conference on Learning Representations, ICLR '19* (2019).
 - [16] Y. Liu, Y. Sun, B. Xue, M. Zhang, G.G. Yen, K.C. Tan, *A survey on evolutionary neural architecture search*, IEEE Transactions on Neural Networks and Learning Systems. 2023, volume 34, 2, pages 550–570.
 - [17] T. Chugh, K. Sindhya, J. Hakanen, K. Miettinen, *A survey on handling computationally expensive multiobjective optimization problems with evolutionary algorithms*, Soft Computing. 2019, volume 23, 9, pages 3137–3166.
 - [18] A. Díaz-Manríquez, G. Toscano, J.H. Barron-Zambrano, E. Tello-Leal, *A review of surrogate assisted multiobjective evolutionary algorithms*, Computational Intelligence and Neuroscience. 2016, volume 2016, page 9420460.
 - [19] Q. Wang, Y. Zhang, D. Gong, X. Song, C. He, X. Ji, F. Chu, *A review of surrogate-assisted evolutionary multi-objective optimization*, IEEE Transactions on Emerging Topics in Computational Intelligence. 2026, pages 1–21.
 - [20] A. Dushatskiy, T. Alderliesten, P.A.N. Bosman, *A novel surrogate-assisted evolutionary algorithm applied to partition-based ensemble learning*, in: *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '21)* (ACM, New York, NY, USA, 2021), pages 583–591.
 - [21] A. Chebykin, T. Alderliesten, P.A.N. Bosman, *Evolutionary neural cascade search across supernetworks*, in: *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '22)* (ACM, New York, NY, USA, 2022), pages 1038–1047.
 - [22] A. Krizhevsky, *Learning Multiple Layers of Features from Tiny Images*, Technical report, University of Toronto. 2009.
 - [23] X. Dong, Y. Yang, *NAS-Bench-201: Extending the scope of reproducible neural architecture search*, in: *International Conference on Learning Representations, ICLR '20* (2020).
 - [24] A. Dushatskiy, *SAGOMEA: Surrogate-assisted P3-GOMEA reference implementation*, <https://github.com/ArkadiyD/SAGOMEA>. 2021. Reference C++/Python implementation accompanying Dushatskiy et al., GECCO 2021.
 - [25] J.D. Kelleher, *Deep learning* (MIT Press, Cambridge, MA, USA, 2019).
 - [26] K. O'Shea, *An introduction to convolutional neural networks*, arXiv preprint arXiv:1511.08458. 2015.
 - [27] V. Nair, G.E. Hinton, *Rectified linear units improve restricted Boltzmann machines*, in: *Proceedings of the 27th International Conference on Machine Learning (ICML)* (Omnipress, Madison, WI, USA, 2010), pages 807–814.

- [28] D.E. Rumelhart, G.E. Hinton, R.J. Williams, *Learning representations by back-propagating errors*, Nature. 1986, volume 323, 6088, pages 533–536.
- [29] M.H. Mohd Noor, A.O. Ige, *A survey on state-of-the-art deep learning applications and challenges*, Engineering Applications of Artificial Intelligence. 2025, volume 159, page 111225.
- [30] A. Younesi, M. Ansari, M. Fazli, A. Ejlali, M. Shafique, J. Henkel, *A comprehensive survey of convolutions in deep learning: Applications, challenges, and future trends*, IEEE Access. 2024, volume 12, pages 41180–41218.
- [31] T. Talaei Khoei, H. Ould Slimane, N. Kaabouch, *Deep learning: systematic review, models, challenges, and research directions*, Neural Computing and Applications. 2023, volume 35, 31, pages 23103–23124.
- [32] W. Jiang, V. Banna, N. Vivek, A. Goel, N. Synovic, G.K. Thiruvathukal, J.C. Davis, *Challenges and practices of deep learning model reengineering: A case study on computer vision*, Empirical Software Engineering. 2024, volume 29, 6, page 142.
- [33] M. Dorigo, T. Stützle, *Ant Colony Optimization* (MIT Press, Cambridge, MA, 2004).
- [34] J. Kennedy, R. Eberhart, *Particle swarm optimization*, in: *Proceedings of the IEEE International Conference on Neural Networks*, volume 4 (IEEE, Piscataway, NJ, USA, 1995), pages 1942–1948.
- [35] C.L. Camacho-Villalón, M. Dorigo, T. Stützle, *Exposing the grey wolf, moth-flame, whale, firefly, bat, and antlion algorithms: six misleading optimization techniques inspired by bestial metaphors*, International Transactions in Operational Research. 2023, volume 30, 6, pages 2945–2971.
- [36] C. Aranha, C.L. Camacho-Villalón, F. Campelo, M. Dorigo, R. Ruiz, M. Sevaux, K. Sörensen, T. Stützle, *Metaphor-based metaheuristics, a call for action: the elephant in the room*, Swarm Intelligence. 2022, volume 16, 1, pages 1–6.
- [37] J.H. Holland, *Adaptation in Natural and Artificial Systems: An Introductory Analysis with Applications to Biology, Control, and Artificial Intelligence* (University of Michigan Press, Ann Arbor, MI, 1975). Reprinted by MIT Press, 1992.
- [38] P.A. Bosman, N.H. Luong, D. Thierens, *Expanding from discrete cartesian to permutation gene-pool optimal mixing evolutionary algorithms*, in: *Proceedings of the Genetic and Evolutionary Computation Conference 2016*, GECCO '16 (Association for Computing Machinery, New York, NY, USA, 2016), page 637–644.
- [39] B.W. Goldman, W.F. Punch, *Parameter-less population pyramid*, in: *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation*, GECCO '14 (Association for Computing Machinery, New York, NY, USA, 2014), page 785–792.
- [40] Z. Michalewicz, M. Schoenauer, *Evolutionary algorithms*, in: *Encyclopedia of Information Systems*, edited by H. Bidgoli, volume 2 (Academic Press, San Diego, CA, 2003), pages 259–267.
- [41] N. Hansen, A. Ostermeier, *Completely derandomized self-adaptation in evolution strategies*, Evolutionary Computation. 2001, volume 9, 2, pages 159–195.

- [42] I. Loshchilov, F. Hutter, *CMA-ES for hyperparameter optimization of deep neural networks*, in: *International Conference on Learning Representations (ICLR), Workshop Track* (2016).
- [43] P. Chrabąszcz, I. Loshchilov, F. Hutter, *Back to basics: Benchmarking canonical evolution strategies for playing Atari*, in: *Proceedings of the 27th International Joint Conference on Artificial Intelligence (IJCAI)* (International Joint Conferences on Artificial Intelligence Organization, 2018), pages 1419–1426.
- [44] M. Suganuma, S. Shirakawa, T. Nagao, *A genetic programming approach to designing convolutional neural network architectures*, in: *Proceedings of the Genetic and Evolutionary Computation Conference (GECCO '17)* (ACM, New York, NY, USA, 2017), pages 497–504.
- [45] D. Thierens, P.A.N. Bosman, *Hierarchical problem solving with the linkage tree genetic algorithm*, in: *Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation, GECCO '11* (ACM, New York, NY, USA, 2011), pages 877–884.
- [46] T.M. Cover, J.A. Thomas, *Elements of Information Theory*, 2 edition (Wiley-Interscience, Hoboken, NJ, USA, 2006).
- [47] S.H. Hsu, T.L. Yu, *Optimization by pairwise linkage detection, incremental linkage set, and restricted / back mixing: Dsmga-ii*, in: *Proceedings of the 2015 Annual Conference on Genetic and Evolutionary Computation, GECCO '15* (Association for Computing Machinery, New York, NY, USA, 2015), page 519–526.
- [48] T.L. Yu, D.E. Goldberg, K. Sastry, C.F. Lima, M. Pelikan, *Dependency structure matrix, genetic algorithms, and effective recombination*, *Evolutionary Computation*. 2009, volume 17, 4, pages 595–626.
- [49] D. Thierens, P.A.N. Bosman, *Optimal mixing evolutionary algorithms*, in: *Proceedings of the 13th Annual Conference on Genetic and Evolutionary Computation, GECCO '11* (ACM, New York, NY, USA, 2011), pages 617–624.
- [50] Z.H. Zhan, J.Y. Li, J. Zhang, *Evolutionary deep learning: A survey*, *Neurocomputing*. 2022, volume 483, pages 42–58.
- [51] Y. Ma, Y. Xie, *Evolutionary neural networks for deep learning: a review*, *International Journal of Machine Learning and Cybernetics*. 2022, volume 13, 10, pages 3001–3018.
- [52] K.O. Stanley, R. Miikkulainen, *Evolving neural networks through augmenting topologies*, *Evolutionary Computation*. 2002, volume 10, 2, pages 99–127.
- [53] R. Miikkulainen, J. Liang, E. Meyerson, A. Rawal, D. Fink, O. Francon, B. Raju, H. Shahrzad, A. Navruzian, N. Duffy, B. Hodjat, *Evolving deep neural networks*, in: *Artificial Intelligence in the Age of Neural Networks and Brain Computing*, edited by R. Kozma, C. Alippi, Y. Choe, F.C. Morabito (Elsevier, London, United Kingdom, 2019), pages 293–312.
- [54] Y. Sun, B. Xue, M. Zhang, G.G. Yen, *Evolving deep convolutional neural networks for image classification*, *IEEE Transactions on Evolutionary Computation*. 2020, volume 24, 2, pages 394–407.
- [55] E. Real, S. MOre, A. Selle, S. Saxena, Y.L. Suematsu, J. Tan, Q.V. Le, A. Kurakin, *Large-scale evolution of image classifiers*, in: *Proceedings of the 34th International Conference on Machine Learning, ICML '17*, volume 70 (PMLR, 2017), pages 2902–2911.
- [56] K. He, X. Zhang, S. Ren, J. Sun, *Deep residual learning for image recognition*, in: *Proceed-*

- ings of the *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (IEEE, Piscataway, NJ, USA, 2016), pages 770–778.
- [57] G. Huang, Z. Liu, L. van der Maaten, K.Q. Weinberger, *Densely connected convolutional networks*, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (IEEE, Piscataway, NJ, USA, 2017), pages 2261–2269.
 - [58] B. Zoph, V. Vasudevan, J. Shlens, Q.V. Le, *Learning transferable architectures for scalable image recognition*, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (IEEE, Piscataway, NJ, USA, 2018), pages 8697–8710.
 - [59] C. Ying, A. Klein, E. Christiansen, E. Real, K. Murphy, F. Hutter, *NAS-Bench-101: Towards reproducible neural architecture search*, in: *Proceedings of the 36th International Conference on Machine Learning, ICML '19*, volume 97 (PMLR, 2019), pages 7105–7114.
 - [60] K. Deb, M. Goyal, *A combined genetic adaptive search (GeneAS) for engineering design*, *Computer Science and Informatics*. 1996, volume 26, 4, pages 30–45.
 - [61] P.R. Lorenzo, J. Nalepa, *Memetic evolution of deep neural networks*, in: *Proceedings of the Genetic and Evolutionary Computation Conference, GECCO '18* (ACM, New York, NY, USA, 2018), pages 505–512.
 - [62] T. Chen, I. Goodfellow, J. Shlens, *Net2Net: Accelerating learning via knowledge transfer*, in: *International Conference on Learning Representations, ICLR '16* (2016).
 - [63] T. Wei, C. Wang, Y. Rui, C.W. Chen, *Network morphism*, in: *Proceedings of the 33rd International Conference on Machine Learning, ICML '16*, volume 48 (PMLR, 2016), pages 564–572.
 - [64] M. Wistuba, A. Rawat, T. Pedapati, *A survey on neural architecture search*, arXiv preprint arXiv:1905.01392. 2019.
 - [65] N. Li, L. Ma, G. Yu, B. Xue, M. Zhang, Y. Jin, *Survey on evolutionary deep learning: Principles, algorithms, applications, and open issues*, *ACM Comput. Surv.* 2023, volume 56, 2.
 - [66] K. Chauhan, S. Jani, D. Thakkar, R. Dave, J. Bhatia, S. Tanwar, M.S. Obaidat, *Automated machine learning: The new wave of machine learning*, in: *2020 2nd International Conference on Innovative Mechanisms for Industry Applications (ICIMIA)* (IEEE, Piscataway, NJ, USA, 2020), pages 205–212.
 - [67] K. Deb, *Multi-Objective Optimization Using Evolutionary Algorithms* (John Wiley & Sons, Chichester, 2001).
 - [68] Q. Zhang, H. Li, *Moea/d: A multiobjective evolutionary algorithm based on decomposition*, *IEEE Transactions on Evolutionary Computation*. 2007, volume 11, 6, pages 712–731.
 - [69] Q. Zhang, H. Li, D. Maringer, E. Tsang, *Moea/d with nbi-style tchebycheff approach for portfolio management*, in: *IEEE Congress on Evolutionary Computation* (IEEE, Piscataway, NJ, USA, 2010), pages 1–8.
 - [70] E. Zitzler, L. Thiele, *Multiobjective evolutionary algorithms: A comparative case study and the strength Pareto approach*, *IEEE Transactions on Evolutionary Computation*. 1999, volume 3, 4, pages 257–271.
 - [71] H. Ishibuchi, H. Masuda, Y. Tanigaki, Y. Nojima, *Modified distance calculation in gen-*

- erational distance and inverted generational distance*, in: *Evolutionary Multi-Criterion Optimization (EMO 2015)*, *Lecture Notes in Computer Science*, volume 9019 (Springer, Cham, Switzerland, 2015), pages 110–125.
- [72] Z. Lu, K. Deb, E. Goodman, W. Banzhaf, V.N. Boddeti, *NSGANetV2: Evolutionary multi-objective surrogate-assisted neural architecture search*, in: *Computer Vision – ECCV 2020*, *Lecture Notes in Computer Science*, volume 12346 (Springer, Cham, Switzerland, 2020), pages 35–51.
 - [73] Z. Lu, G. Sreekumar, E. Goodman, W. Banzhaf, K. Deb, V.N. Boddeti, *Neural architecture transfer*, *IEEE Transactions on Pattern Analysis and Machine Intelligence*. 2021, volume 43, 9, pages 2971–2989.
 - [74] D. Brockhoff, T. Wagner, H. Trautmann, *2 indicator-based multiobjective search*, *Evolutionary Computation*. 2015, volume 23, 3, pages 369–395.
 - [75] N.H. Luong, H. La Poutré, P.A.N. Bosman, *Multi-objective gene-pool optimal mixing evolutionary algorithms*, in: *Proceedings of the 2014 Annual Conference on Genetic and Evolutionary Computation (GECCO '14)* (ACM, New York, NY, USA, 2014), pages 357–364.
 - [76] M.W. Przewoźniczek, P. Dziurzynski, S. Zhao, L.S. Indrusiak, *Multi-objective parameter-less population pyramid for solving industrial process planning problems*, *Swarm and Evolutionary Computation*. 2021, volume 60, page 100773.
 - [77] L. Ma, N. Li, G. Yu, X. Geng, S. Cheng, X. Wang, M. Huang, Y. Jin, *Pareto-wise ranking classifier for multiobjective evolutionary neural architecture search*, *IEEE Transactions on Evolutionary Computation*. 2024, volume 28, 3, pages 570–581.
 - [78] Y. Xue, C. Chen, A. Słowik, *Neural architecture search based on a multi-objective evolutionary algorithm with probability stack*, *IEEE Transactions on Evolutionary Computation*. 2023, volume 27, 4, pages 778–786.
 - [79] Y. Jin, H. Wang, T. Chugh, D. Guo, K. Miettinen, *Data-driven evolutionary optimization: An overview and case studies*, *IEEE Transactions on Evolutionary Computation*. 2019, volume 23, 3, pages 442–458.
 - [80] Y. Jin, M. Olhofer, B. Sendhoff, *A framework for evolutionary optimization with approximate fitness functions*, *IEEE Transactions on Evolutionary Computation*. 2002, volume 6, 5, pages 481–494.
 - [81] J. Knowles, *ParEGO: A hybrid algorithm with on-line landscape approximation for expensive multiobjective optimization problems*, *IEEE Transactions on Evolutionary Computation*. 2006, volume 10, 1, pages 50–66.
 - [82] C. Oh, J.M. Tomczak, E. Gavves, M. Welling, *Combinatorial Bayesian optimization using the graph Cartesian product*, in: *Advances in Neural Information Processing Systems*, volume 32 (Curran Associates, Inc., Red Hook, NY, USA, 2019), pages 2910–2920.
 - [83] J. Bergstra, R. Bardenet, Y. Bengio, B. Kégl, *Algorithms for hyper-parameter optimization*, in: *Advances in Neural Information Processing Systems*, volume 24 (Curran Associates, Inc., Red Hook, NY, USA, 2011), pages 2546–2554.
 - [84] F. Hutter, H.H. Hoos, K. Leyton-Brown, *Sequential model-based optimization for general algorithm configuration*, in: *Learning and Intelligent Optimization (LION 5)*, *Lecture Notes*

- in *Computer Science*, volume 6683 (Springer, Berlin, Heidelberg, Germany, 2011), pages 507–523.
- [85] R. Jin, W. Chen, T.W. Simpson, *Comparative studies of metamodelling techniques under multiple modelling criteria*, Structural and Multidisciplinary Optimization. 2001, volume 23, 1, pages 1–13.
 - [86] A. Díaz-Manríquez, G. Toscano-Pulido, W. Gómez-Flores, *On the selection of surrogate models in evolutionary optimization algorithms*, in: *2011 IEEE Congress of Evolutionary Computation (CEC)* (IEEE, Piscataway, NJ, USA, 2011), pages 2155–2162.
 - [87] T. Goel, R.T. Haftka, W. Shyy, N.V. Queipo, *Ensemble of surrogates*, Structural and Multidisciplinary Optimization. 2007, volume 33, 3, pages 199–216.
 - [88] D. Guo, Y. Jin, J. Ding, T. Chai, *Heterogeneous ensemble-based infill criterion for evolutionary multiobjective optimization of expensive problems*, IEEE Transactions on Cybernetics. 2019, volume 49, 3, pages 1012–1025.
 - [89] F.A.C. Viana, R.T. Haftka, V. Steffen, *Multiple surrogates: How cross-validation errors can help us to obtain the best predictor*, Structural and Multidisciplinary Optimization. 2009, volume 39, 4, pages 439–457.
 - [90] C. White, A. Zela, B. Ru, Y. Liu, F. Hutter, *How powerful are performance predictors in neural architecture search?*, in: *Advances in Neural Information Processing Systems (NeurIPS)*, volume 34 (Curran Associates, Inc., Red Hook, NY, USA, 2021), pages 28454–28469.
 - [91] X. Lu, T. Sun, K. Tang, *Evolutionary optimization with hierarchical surrogates*, Swarm and Evolutionary Computation. 2019, volume 46, pages 69–83.
 - [92] M. Sokolova, G. Lapalme, *A systematic analysis of performance measures for classification tasks*, Information Processing & Management. 2009, volume 45, 4, pages 427–437.
 - [93] P. Henderson, J. Hu, J. Romoff, E. Brunskill, D. Jurafsky, J. Pineau, *Towards the systematic reporting of the energy and carbon footprints of machine learning*, Journal of Machine Learning Research. 2020, volume 21, 248, pages 1–43.
 - [94] C. Li, Z. Yu, Y. Fu, Y. Zhang, Y. Zhao, H. You, Q. Yu, Y. Wang, Y. Lin, *HW-NAS-Bench: Hardware-aware neural architecture search benchmark*, in: *9th International Conference on Learning Representations (ICLR)* (2021).
 - [95] NVIDIA Corporation, *NVML api reference guide / pyNVML python bindings*, <https://developer.nvidia.com/nvidia-management-library-nvml>. 2024. Accessed: 2026-05-12.
 - [96] L. Breiman, *Random forests*, Machine Learning. 2001, volume 45, 1, pages 5–32.
 - [97] J.H. Friedman, *Greedy function approximation: A gradient boosting machine*, The Annals of Statistics. 2001, volume 29, 5, pages 1189–1232.
 - [98] C.E. Rasmussen, C.K.I. Williams, *Gaussian Processes for Machine Learning*, Adaptive Computation and Machine Learning (MIT Press, Cambridge, MA, 2006).
 - [99] R. Koenker, G. Bassett, *Regression quantiles*, Econometrica. 1978, volume 46, 1, pages 33–50.
 - [100] Y. Gal, Z. Ghahramani, *Dropout as a Bayesian approximation: Representing model uncertainty in deep learning*, in: *Proceedings of the 33rd International Conference on Machine*

- Learning (ICML), Proceedings of Machine Learning Research*, volume 48 (PMLR, 2016), pages 1050–1059.
- [101] E.V. Bonilla, K.M.A. Chai, C.K.I. Williams, *Multi-task Gaussian process prediction*, in: *Advances in Neural Information Processing Systems (NeurIPS)*, volume 20 (Curran Associates, Inc., Red Hook, NY, USA, 2008), pages 153–160.
 - [102] T. Hastie, R. Tibshirani, J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*, 2 edition, Springer Series in Statistics (Springer, New York, NY, 2009).
 - [103] C.A. Coello Coello, M. Reyes Sierra, *A study of the parallelization of a coevolutionary multi-objective evolutionary algorithm*, in: *MICAI 2004: Advances in Artificial Intelligence, Lecture Notes in Computer Science*, volume 2972 (Springer, Berlin, Heidelberg, Germany, 2004), pages 688–697.

LIST OF FIGURES

4.1.	Example of ANN layers. Own work.	12
4.2.	Simplified representation of Linkage Tree. Each node represents gene. Genes are grouped into co-dependent clusters. Own work.	18
4.3.	Example of cell-based search space. This approach allows to reduce search space. Own preparation.	23
4.4.	Depiction of cell used in NAS-Bench-201. Own preparation.	24
4.5.	Purple dots create a set of non-dominated solutions for trade-off between objectives. Own preparation.	27
5.1.	FLOPs and measured inference energy and accuracy for 1000 random architectures from NAS-Bench-201 on CIFAR-10. Inference energy was measured with pyNVML on an NVIDIA RTX 5070 Ti as the median of $N = 5$ runs. Own pilot study.	38
7.1.	Pareto-front overlay across algorithms for the inference-energy problem.	54
7.2.	Pareto-front overlay across algorithms for the training-energy problem.	55
7.3.	Per-axis gate sensitivity for the deterministic surrogates on inference energy. Mean hypervolume over 10 seeds across the nine per-axis pairs (η_a, η_e)	59
7.4.	Per-axis gate sensitivity for the deterministic surrogates on training energy. Mean hypervolume over 10 seeds across the nine per-axis pairs (η_a, η_e)	59
7.5.	Pareto-front overlay across the online selection runs over the deterministic pool for the inference-energy problem.	61
7.6.	Pareto-front overlay across the online selection runs over the deterministic pool for the training-energy problem.	61
7.7.	Pareto-front overlay across the probabilistic surrogates for the inference-energy problem.	62
7.8.	Pareto-front overlay across the probabilistic surrogates for the training-energy problem.	63
7.9.	Per-axis gate sensitivity for the probabilistic surrogates on inference energy. Mean hypervolume over 10 seeds across the nine per-axis pairs (η_a, η_e)	66
7.10.	Per-axis gate sensitivity for the probabilistic surrogates on training energy. Mean hypervolume over 10 seeds across the nine per-axis pairs (η_a, η_e)	66
7.11.	Pareto-front overlay across the dynamic-selection schedules over the probabilistic pool for the inference-energy problem.	67
7.12.	Pareto-front overlay across the dynamic-selection schedules over the probabilistic pool for the training-energy problem.	68

LIST OF ALGORITHMS

1	General workflow of an evolutionary algorithm	16
2	Gene-pool Optimal Mixing of one solution	19
3	One iteration of P3	20
4	First Improvement Hill Climber (FIHC)	20
5	Mixing solution s with level P_i	21
6	One generation of MO-GOMEA	30
7	SA-P3-GOMEA, reproduced from [20]. Lines marked "surrogate" are the additions over plain P3-GOMEA.	34
8	Surrogate-gated evaluation and comparison used inside GOM, reproduced from [20].	35
9	Probabilistic acceptance of a donated block during OM, given the threshold τ	47

LIST OF TABLES

4.1.	Training, finetuning, and inference energy for the BLOOMz model family. Cost parity shows how many inferences are needed to equal the combined training and finetuning energy. Adapted from Luccioni et al. [4].	14
4.2.	Core Concepts of EAs	15
6.1.	Operations available on each edge of a NAS-Bench-201 cell [23]. The code column gives the integer used in the encoding of architectures throughout this work.	41
6.2.	Median inference energy in joules for five architectures spanning the NAS-Bench-201 accuracy spectrum, measured under all combinations of $(N_{\text{warmup}}, N_{\text{samples}})$ in the convergence study. Each cell is the median of three independent replicates. The last row (20, 10) corresponds to the most comprehensive protocol available and serves as the reference configuration for the analysis. Architecture labels and validation accuracies (CIFAR-10): A1 = 0_1_0_0_4_0 (acc 0.1000), A2 = 1_1_0_1_4_3 (acc 0.8801), A3 = 1_0_2_1_2_2 (acc 0.9076), A4 = 2_4_0_3_0_3 (acc 0.9213), A5 = 0_3_3_1_4_3 (acc 0.9386). (Table 6.1)	44
6.3.	Deviation of per-cell median energy from the reference cell ($N_{\text{warmup}} = 20, N_{\text{samples}} = 10$), averaged over five architectures spanning the NAS-Bench-201 accuracy spectrum.	44
6.4.	Proposed modifications sequentially.	50
7.1.	Overall comparison at the inherited relaxation rate $\eta = 0.999$, mean over 10 seeds (best per column in bold).	52
7.2.	Wilcoxon signed-rank p-values for the differences in Table 7.1, comparing each method against a control and paired over the 10 seeds 42 to 51. Each problem has one control per indicator, marked with a dash ((probabilistic MLP per-objective on inference, dynamic selection over the probabilistic pool on training for HV), (probabilistic joint MLP on inference, probabilistic per-objective gradient boosting on training for IGD+)). A value below 0.05, in bold, means the two medians differ. . . .	53
7.3.	Deterministic surrogates, mean $\pm$ deviation over 10 seeds. Best HV and IGD+ per problem is bold. Reference points (0.0900, 182.8698) on inference and (0.0900, 575.2847) on training, with oracle Pareto front sizes 24 and 36.	54

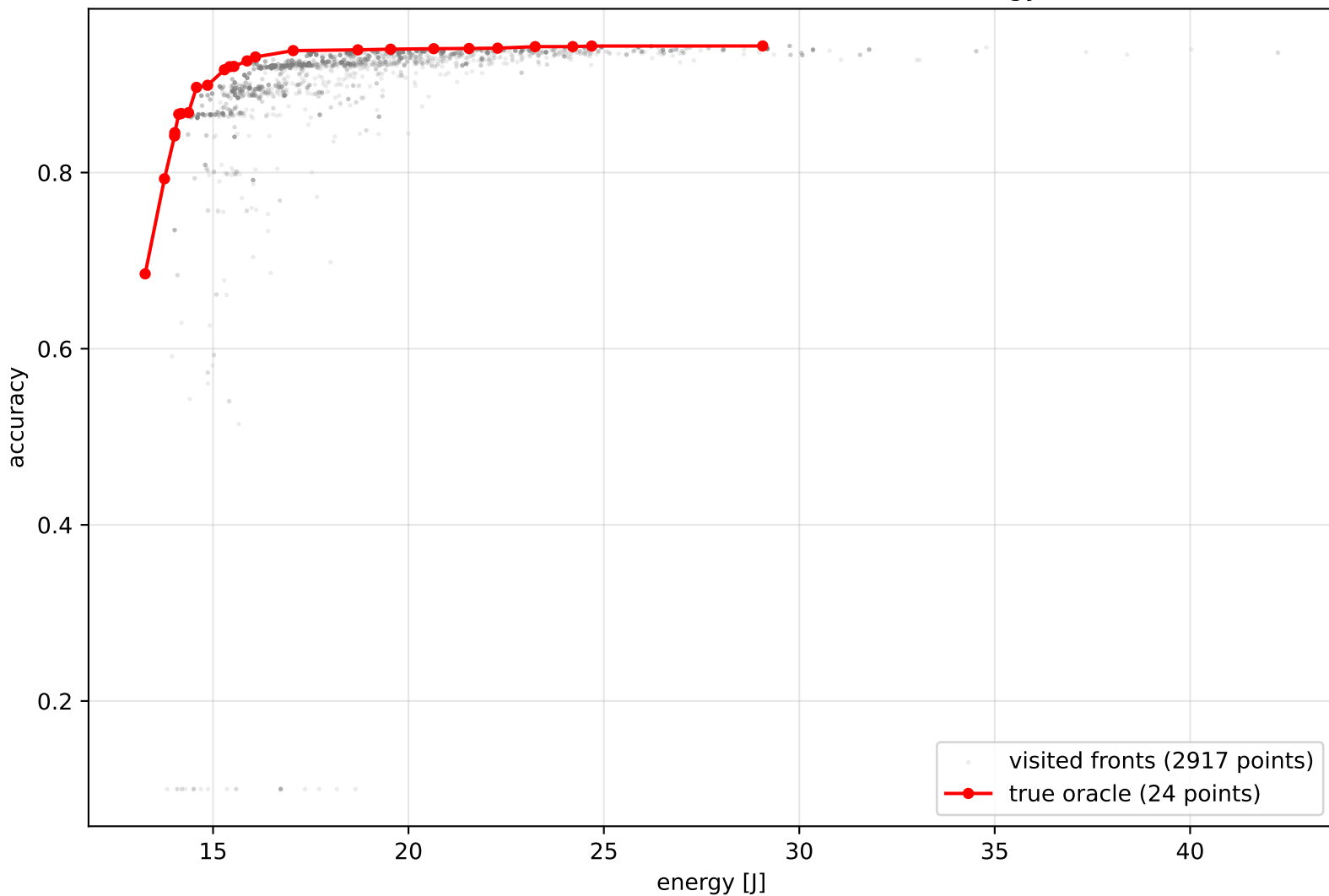
7.4	Gate relaxation rate for the deterministic surrogates on the per-objective-gate campaign, mean over 10 seeds. Each surrogate runs under 12 gate configurations, the single-rate joint gate (joint rows) and the nine per-axis accuracy and energy rate pairs (η_a/η_e). Best HV and lowest IGD+ per surrogate and problem in bold. The joint 0.999 rows match Table 7.3 for the SVR, random forest, and gradient boosting surrogates. The MLP rows differ, at 140.79 and 332.38 hypervolume here against 142.40 and 347.84 there, from the stochastic MLP training. MO-LS and MO-P3-GOMEA are surrogate-free baselines and do not depend on the gate. Reference points (0.0900, 182.8698) on inference and (0.0900, 575.2847) on training.	56
7.5.	Dynamic surrogate selection over the deterministic pool, mean $\pm$ std over 10 seeds. Best HV per problem in bold.	60
7.6.	Probabilistic surrogates, mean $\pm$ deviation over 10 seeds. Best HV and IGD+ per problem is bold.	62
7.7	Gate relaxation rate for the probabilistic surrogates on the per-axis-gate campaign, mean over 10 seeds. Each surrogate runs under the nine per-axis accuracy and energy rate pairs (η_a/η_e). Best HV and lowest IGD+ per surrogate and problem in bold. Reference points (0.0900, 182.8698) on inference and (0.0900, 575.2847) on training.	64
7.8.	Dynamic surrogate selection over the probabilistic pool, mean $\pm$ std over 10 seeds. Best HV per problem in bold.	67

Appendix

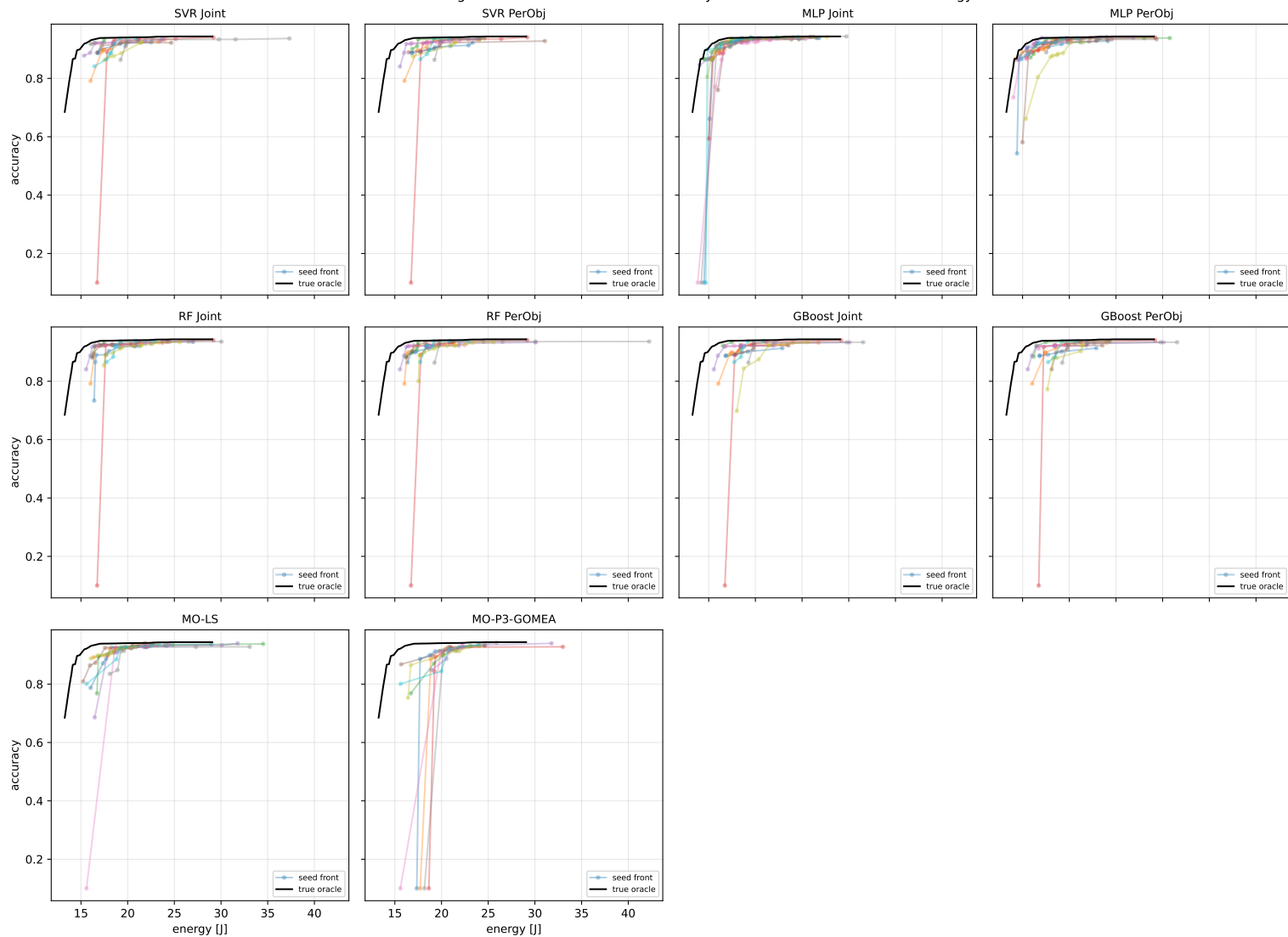
A. FULL CAMPAIGN REPORT

This appendix reproduces the per-algorithm report for the full campaign on both energy problems. It covers the deterministic surrogates, the probabilistic surrogates, and the two surrogate-free baselines. For each it returns a Pareto-front overlay against the reference Pareto front, hypervolume and IGD+ boxplots over the 10 seeds, and a summary table. Hypervolume and IGD+ are computed against the same enumerated reference front and reference points used by the result tables in the main text.

True Pareto front (oracle) — NASInferenceEnergy

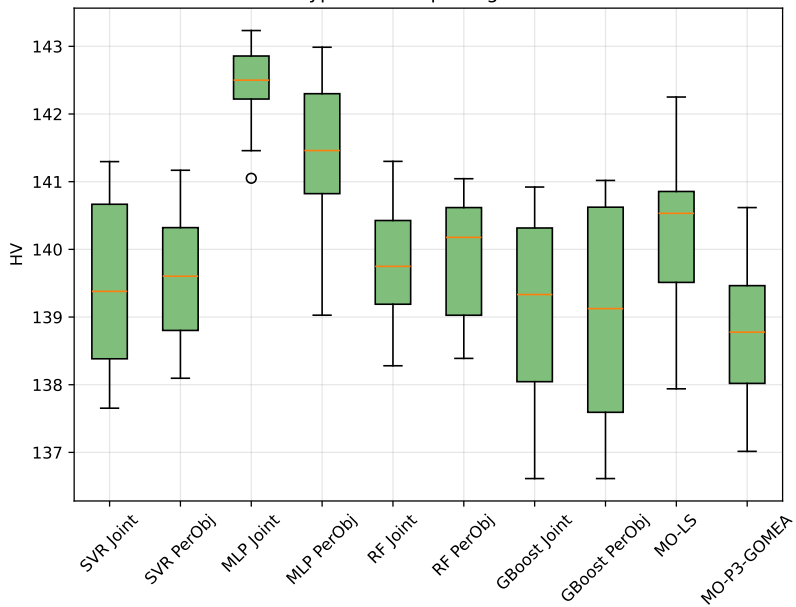


Deterministic surrogates and baselines — Pareto overlay vs true oracle — NASInferenceEnergy

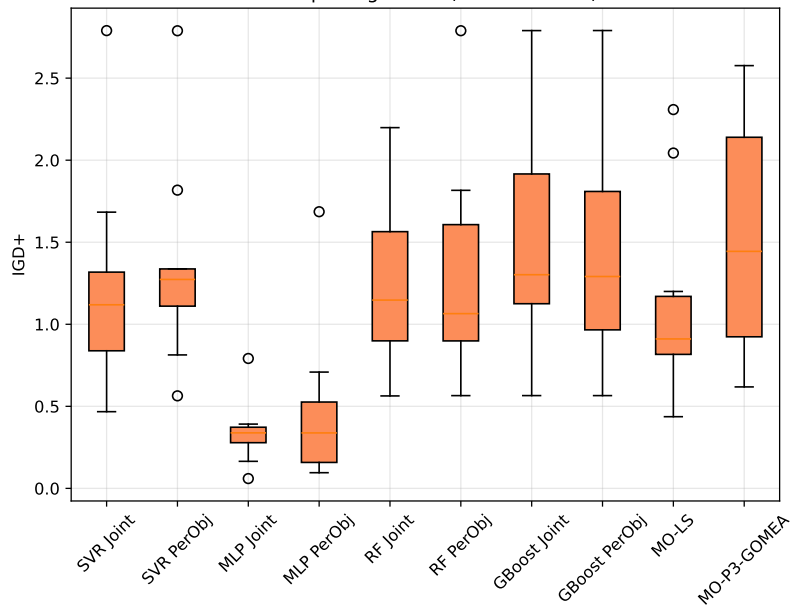


Deterministic surrogates and baselines — performance across 10 seeds — NASInferenceEnergy

Hypervolume per algorithm



IGD+ per algorithm (vs true oracle)



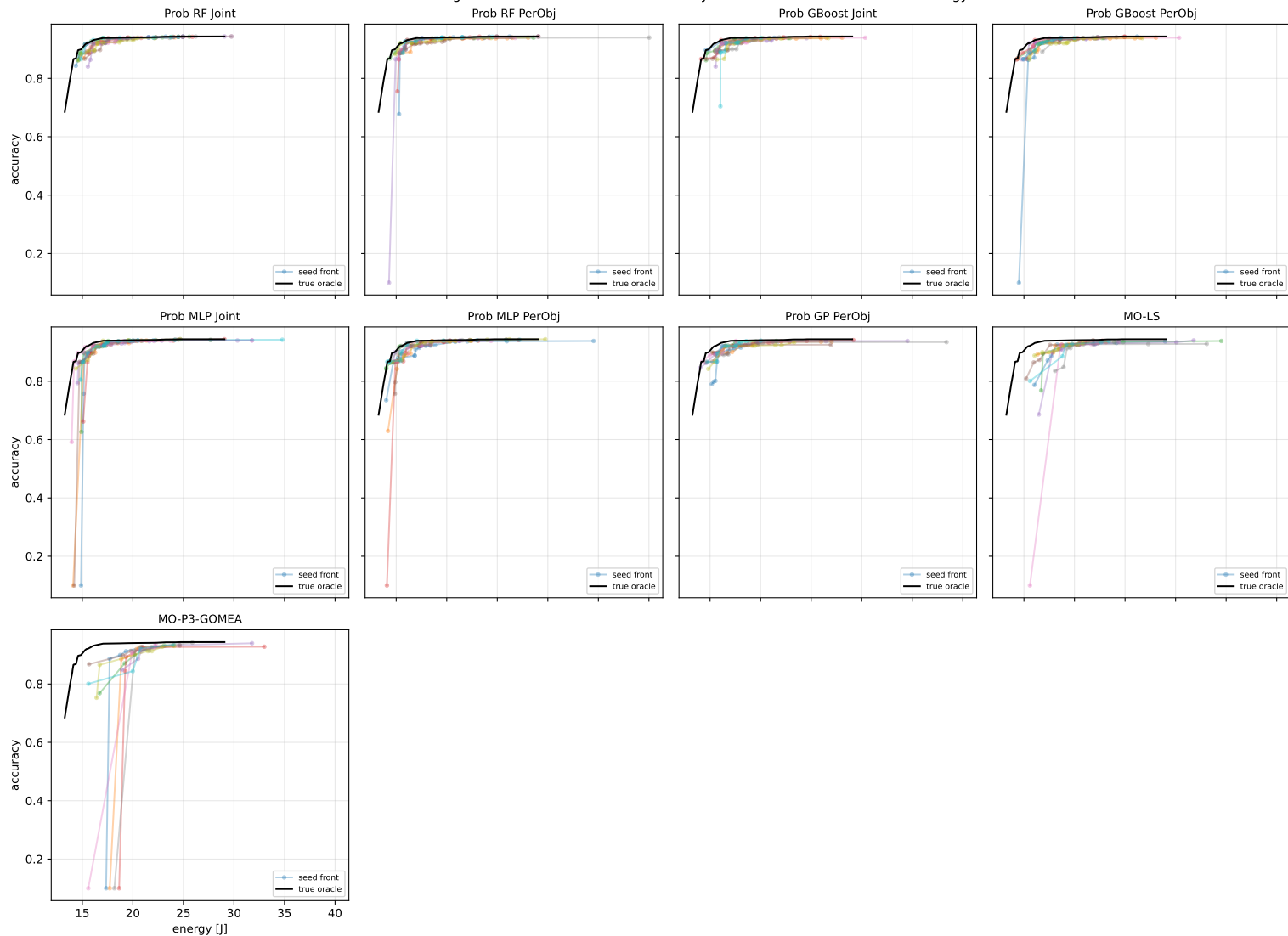
Deterministic surrogates and baselines — NASInferenceEnergy — true oracle Pareto size: 24

algorithm	HV	IGD+ (true oracle)	Pareto size
SVR Joint	139.481 $\pm$ 1.279	1.2307 $\pm$ 0.6102	6.4 $\pm$ 2.1
SVR PerObj	139.623 $\pm$ 1.067	1.3524 $\pm$ 0.5743	5.9 $\pm$ 2.1
MLP Joint	142.404 $\pm$ 0.658	0.3383 $\pm$ 0.1804	12.5 $\pm$ 2.3
MLP PerObj	141.466 $\pm$ 1.124	0.4629 $\pm$ 0.4522	10.1 $\pm$ 2.9
RF Joint	139.729 $\pm$ 0.951	1.2475 $\pm$ 0.4823	7.5 $\pm$ 2.1
RF PerObj	139.892 $\pm$ 0.930	1.2961 $\pm$ 0.6206	7.7 $\pm$ 2.5
GBoost Joint	139.070 $\pm$ 1.423	1.4887 $\pm$ 0.6228	6.0 $\pm$ 2.4
GBoost PerObj	139.031 $\pm$ 1.624	1.4507 $\pm$ 0.6353	5.5 $\pm$ 1.3
MO-LS	140.226 $\pm$ 1.193	1.1132 $\pm$ 0.5731	8.2 $\pm$ 1.7
MO-P3-GOMEA	138.846 $\pm$ 1.088	1.5223 $\pm$ 0.7079	5.2 $\pm$ 0.9

Wilcoxon signed-rank test (Joint vs PerObj, paired by seed):

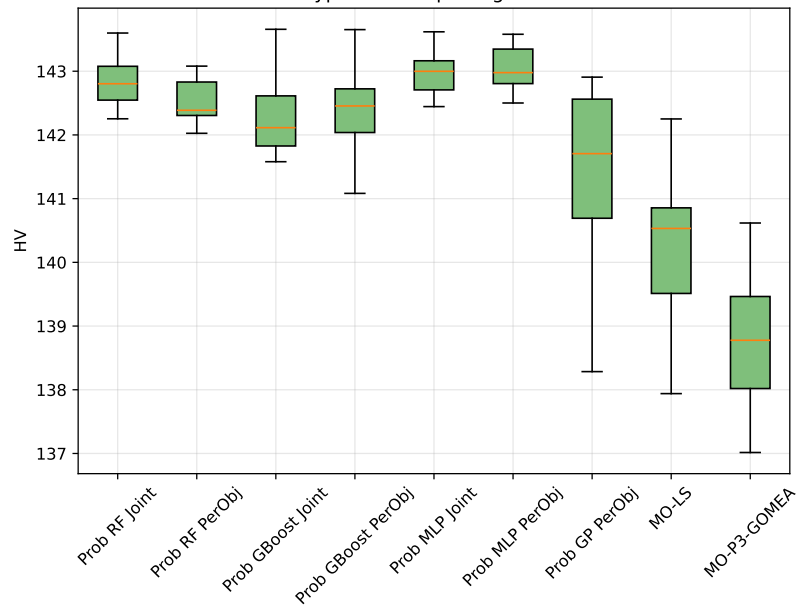
SVR HV: stat=21.00 p=0.5566
SVR IGD+: stat=17.00 p=0.3223
MLP HV: stat=5.00 p=0.0195 **
MLP IGD+: stat=25.00 p=0.8457
RF HV: stat=24.00 p=0.7695
RF IGD+: stat=22.00 p=0.6250
GBoost HV: stat=20.00 p=0.8203
GBoost IGD+: stat=14.00 p=0.3594

Probabilistic surrogates and baselines — Pareto overlay vs true oracle — NASInferenceEnergy

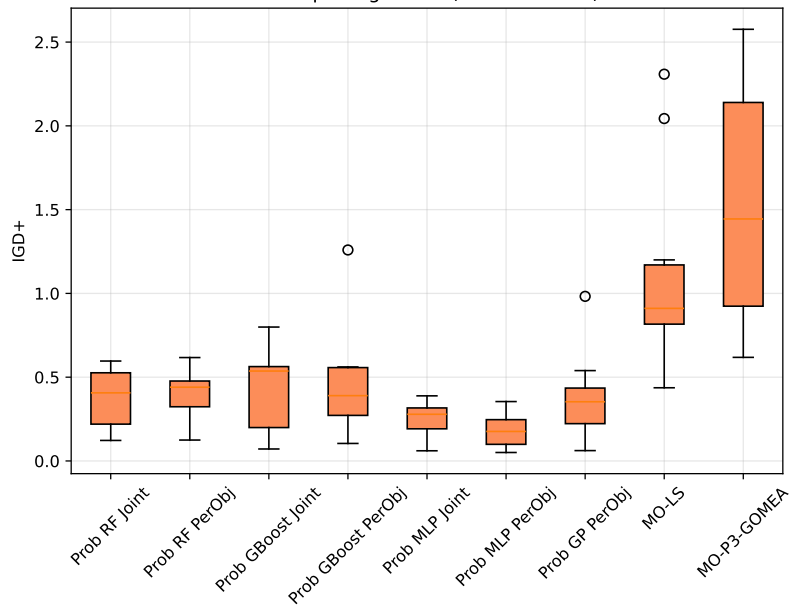


Probabilistic surrogates and baselines — performance across 10 seeds — NASInferenceEnergy

Hypervolume per algorithm



IGD+ per algorithm (vs true oracle)



Probabilistic surrogates and baselines — NASInferenceEnergy — true oracle Pareto size: 24

algorithm	HV	IGD+ (true oracle)	Pareto size
Prob RF Joint	142.864 $\pm$ 0.404	0.3752 $\pm$ 0.1615	12.2 $\pm$ 1.8
Prob RF PerObj	142.520 $\pm$ 0.358	0.4089 $\pm$ 0.1380	12.6 $\pm$ 2.9
Prob GBoost Joint	142.283 $\pm$ 0.601	0.4290 $\pm$ 0.2292	12.0 $\pm$ 1.5
Prob GBoost PerObj	142.363 $\pm$ 0.652	0.4523 $\pm$ 0.3124	11.3 $\pm$ 2.9
Prob MLP Joint	142.972 $\pm$ 0.345	0.2549 $\pm$ 0.0994	12.5 $\pm$ 2.8
Prob MLP PerObj	143.047 $\pm$ 0.328	0.1826 $\pm$ 0.0967	12.6 $\pm$ 1.9
Prob GP PerObj	141.468 $\pm$ 1.370	0.3716 $\pm$ 0.2513	10.0 $\pm$ 2.3
MO-LS	140.226 $\pm$ 1.193	1.1132 $\pm$ 0.5731	8.2 $\pm$ 1.7
MO-P3-GOMEA	138.846 $\pm$ 1.088	1.5223 $\pm$ 0.7079	5.2 $\pm$ 0.9

Wilcoxon signed-rank test (Joint vs PerObj, paired by seed):

RF HV: stat=8.00 p=0.0488 **

RF IGD+: stat=23.00 p=0.6953

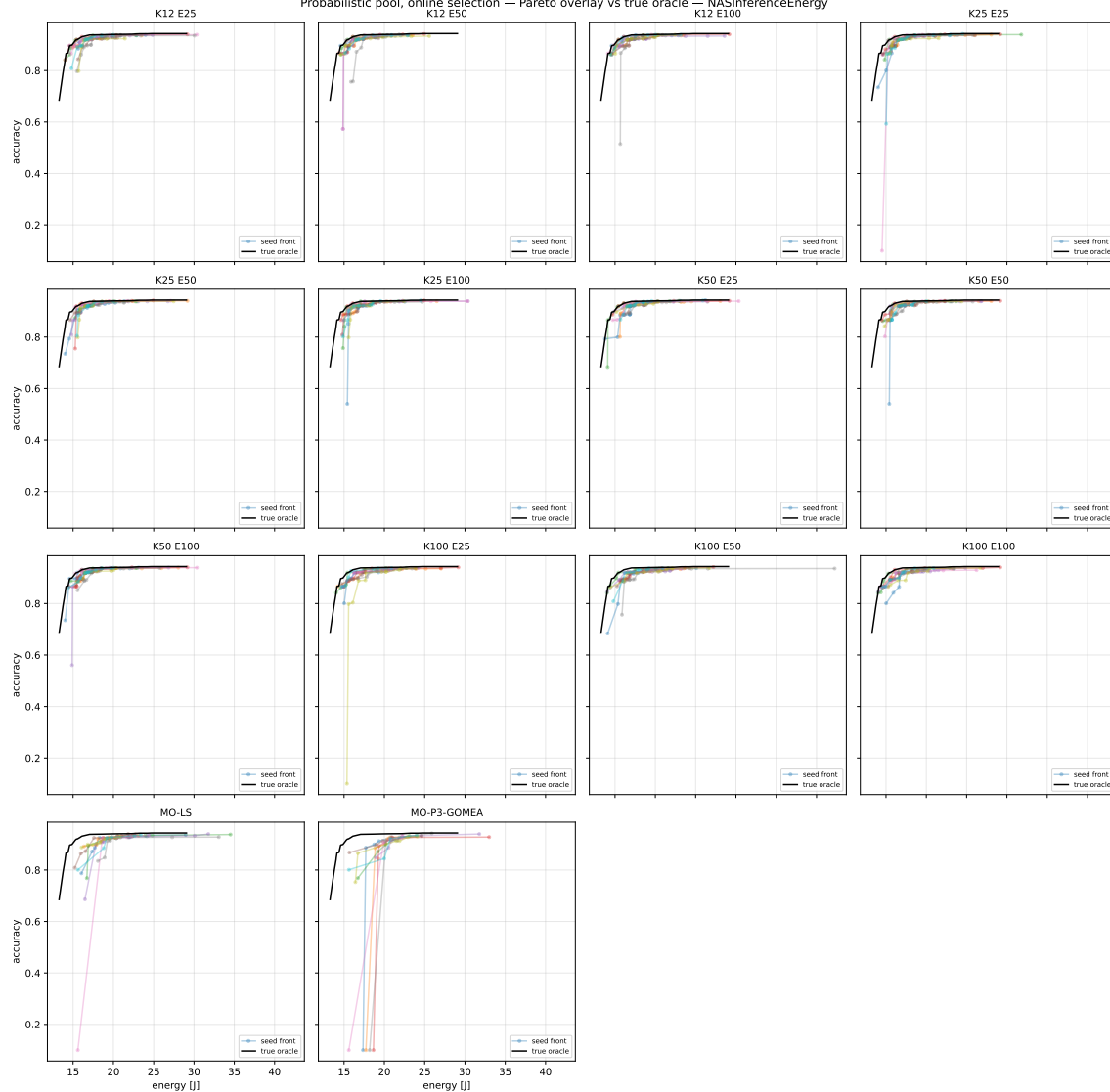
GBoost HV: stat=18.00 p=0.3750

GBoost IGD+: stat=27.00 p=1.0000

MLP HV: stat=25.00 p=0.8457

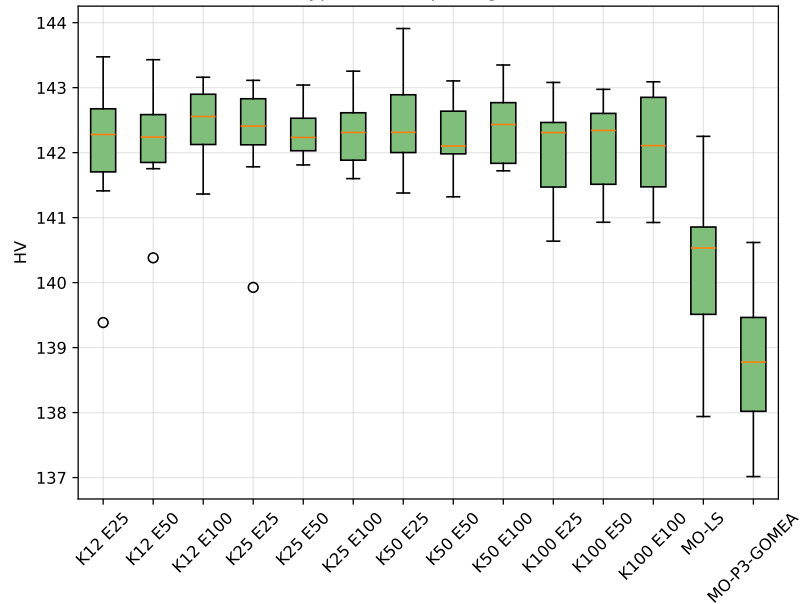
MLP IGD+: stat=16.00 p=0.2754

Probabilistic pool, online selection vs true oracle — NASInferenceEnergy

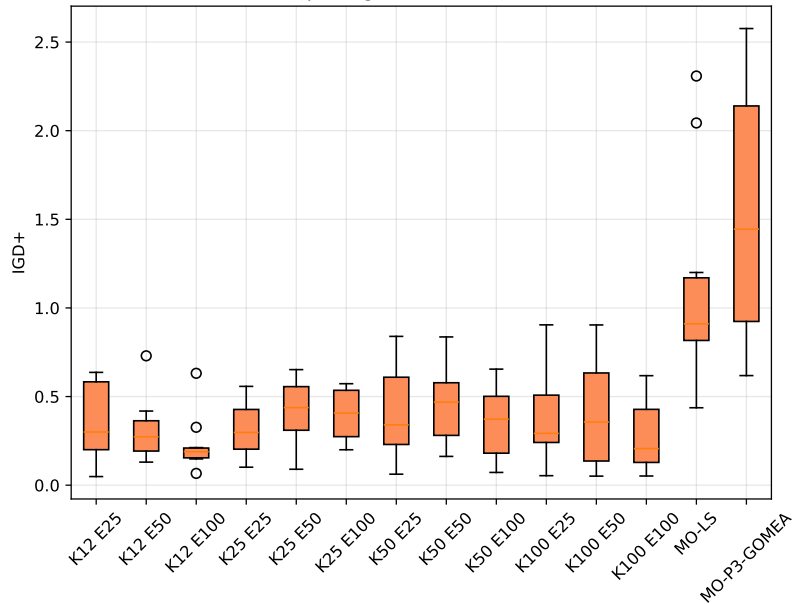


Probabilistic pool, online selection — performance across 10 seeds — NASInferenceEnergy

Hypervolume per algorithm



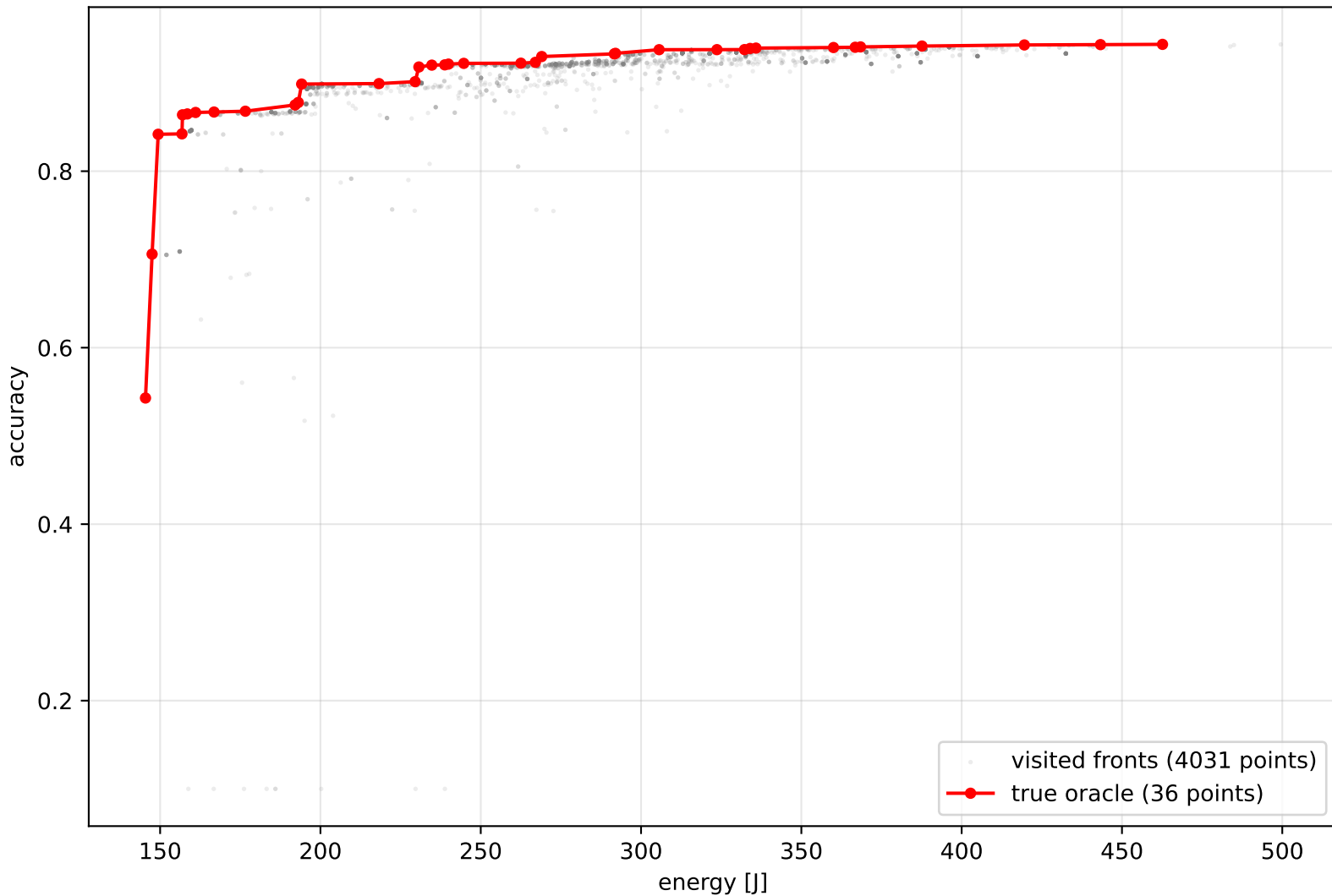
IGD+ per algorithm (vs true oracle)



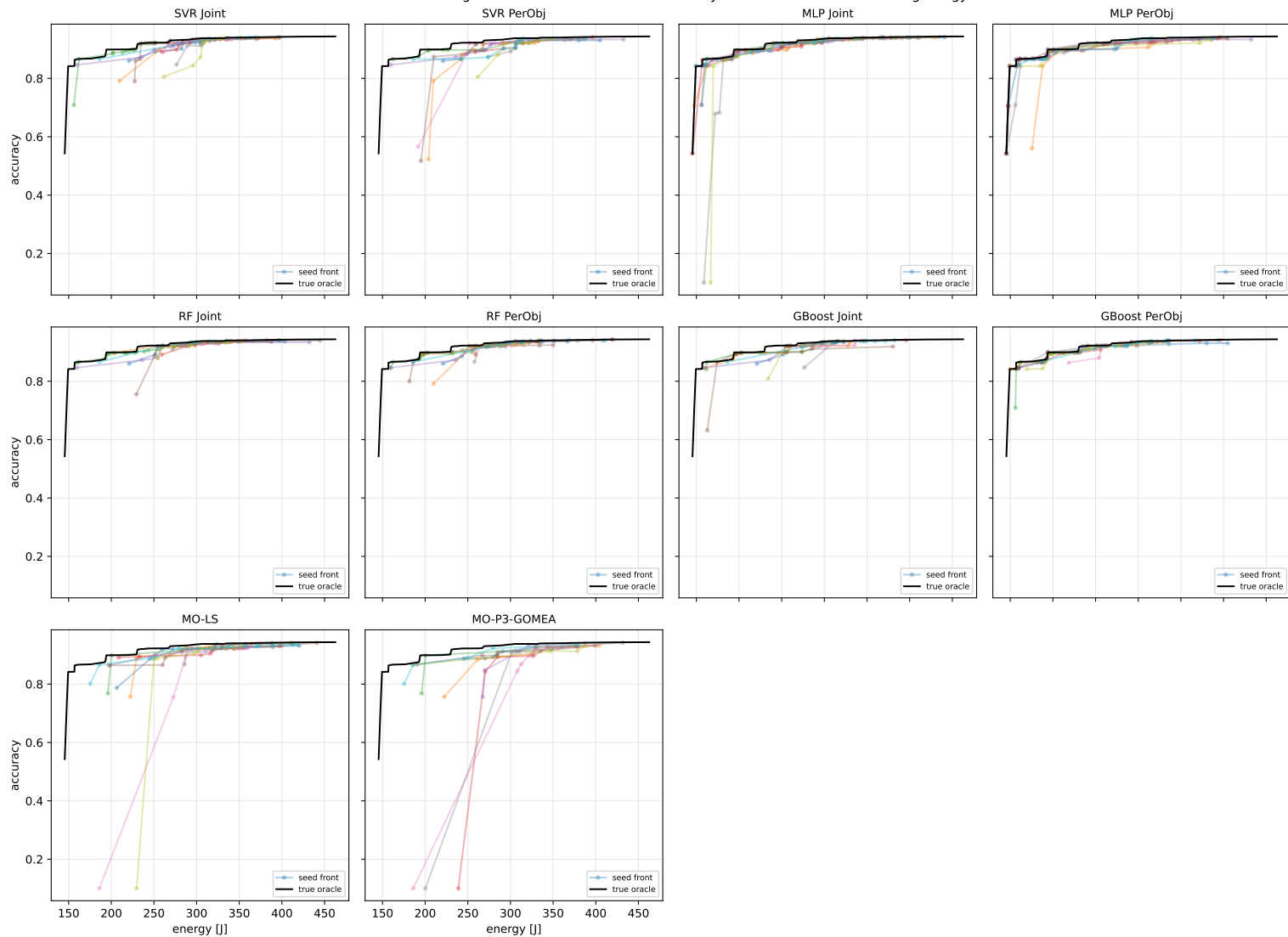
Probabilistic pool, online selection — NASInferenceEnergy — true oracle Pareto size: 24

algorithm	HV	IGD+ (true oracle)	Pareto size
K12 E25	142.070 $\pm$ 1.087	0.3498 $\pm$ 0.2160	11.4 $\pm$ 2.3
K12 E50	142.157 $\pm$ 0.758	0.3089 $\pm$ 0.1647	10.8 $\pm$ 2.1
K12 E100	142.485 $\pm$ 0.542	0.2278 $\pm$ 0.1480	10.6 $\pm$ 3.4
K25 E25	142.255 $\pm$ 0.873	0.3211 $\pm$ 0.1462	12.4 $\pm$ 2.9
K25 E50	142.313 $\pm$ 0.377	0.4193 $\pm$ 0.1699	10.5 $\pm$ 1.4
K25 E100	142.330 $\pm$ 0.528	0.4031 $\pm$ 0.1422	11.0 $\pm$ 2.3
K50 E25	142.443 $\pm$ 0.778	0.4124 $\pm$ 0.2708	11.6 $\pm$ 2.8
K50 E50	142.229 $\pm$ 0.496	0.4456 $\pm$ 0.2043	10.5 $\pm$ 2.2
K50 E100	142.393 $\pm$ 0.557	0.3569 $\pm$ 0.1880	12.0 $\pm$ 2.2
K100 E25	142.006 $\pm$ 0.733	0.3610 $\pm$ 0.2416	10.4 $\pm$ 1.9
K100 E50	142.081 $\pm$ 0.671	0.3969 $\pm$ 0.2887	12.0 $\pm$ 2.8
K100 E100	142.124 $\pm$ 0.742	0.2778 $\pm$ 0.1923	10.3 $\pm$ 2.9
MO-LS	140.226 $\pm$ 1.193	1.1132 $\pm$ 0.5731	8.2 $\pm$ 1.7
MO-P3-GOMEA	138.846 $\pm$ 1.088	1.5223 $\pm$ 0.7079	5.2 $\pm$ 0.9

True Pareto front (oracle) — NASTrainingEnergy

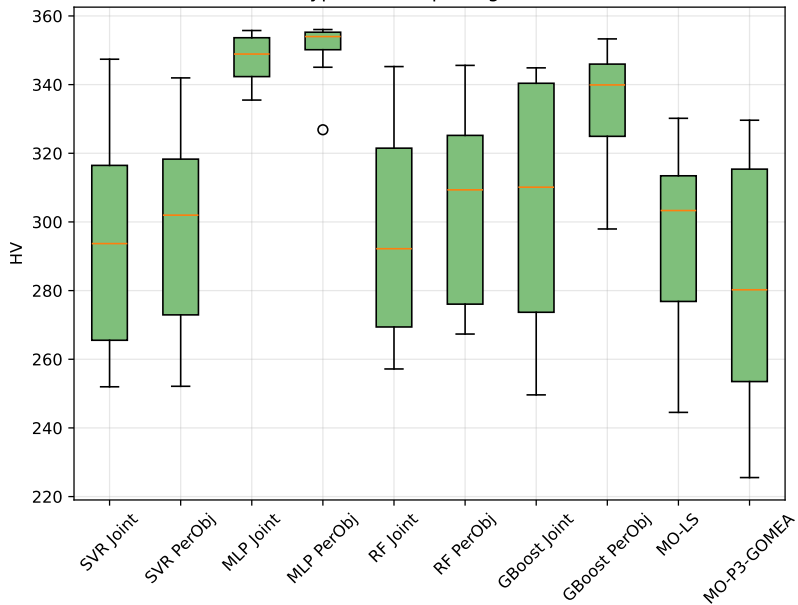


Deterministic surrogates and baselines — Pareto overlay vs true oracle — NASTrainingEnergy

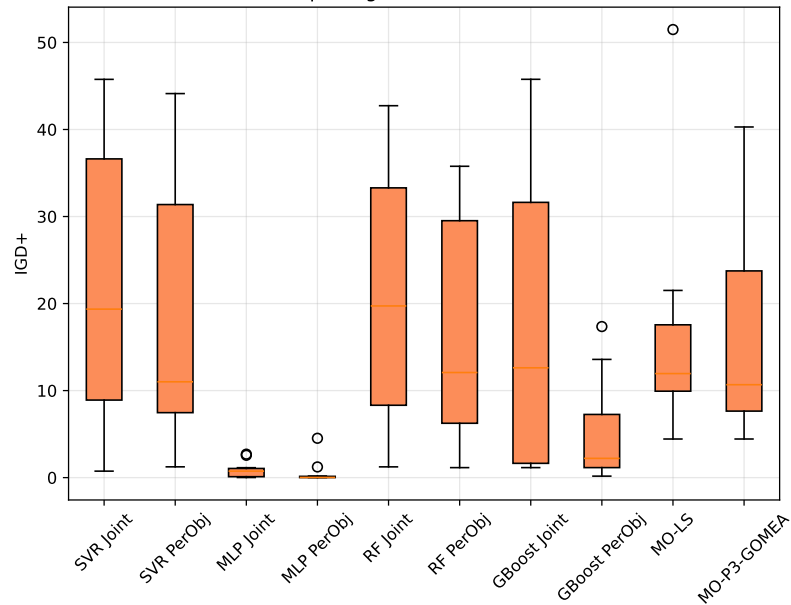


Deterministic surrogates and baselines — performance across 10 seeds — NASTrainingEnergy

Hypervolume per algorithm



IGD+ per algorithm (vs true oracle)



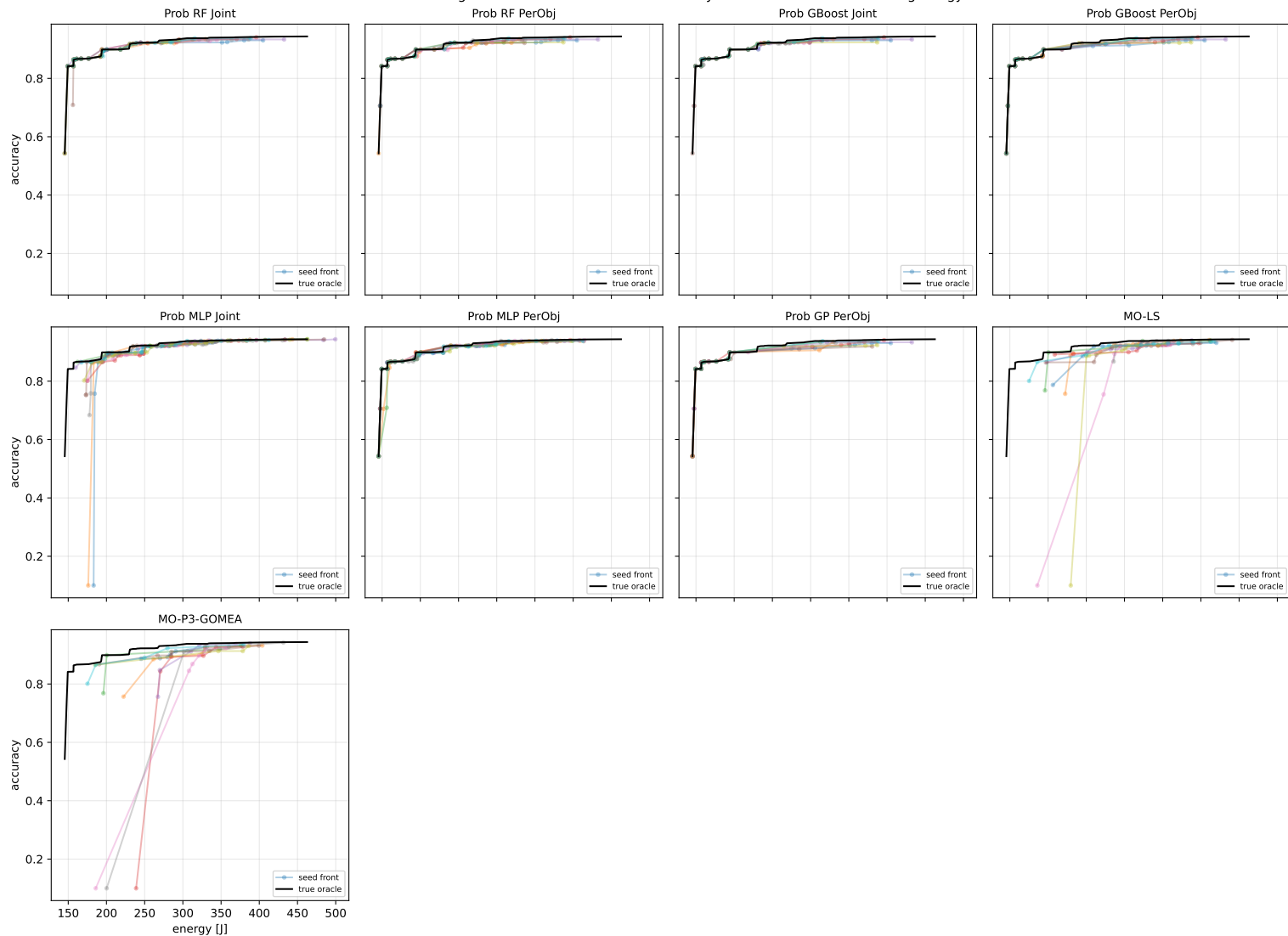
Deterministic surrogates and baselines — NASTrainingEnergy — true oracle Pareto size: 36

algorithm	HV	IGD+ (true oracle)	Pareto size
SVR Joint	294.511 ± 32.425	21.7757 ± 15.4450	11.1 ± 3.0
SVR PerObj	298.999 ± 30.009	17.5825 ± 14.9017	8.6 ± 2.2
MLP Joint	347.841 ± 6.804	0.8980 ± 0.9417	16.6 ± 2.6
MLP PerObj	350.292 ± 8.461	0.6046 ± 1.3543	13.1 ± 5.1
RF Joint	297.180 ± 30.690	20.6130 ± 14.3297	12.5 ± 3.0
RF PerObj	305.804 ± 28.462	16.1888 ± 13.1096	11.3 ± 3.9
GBoost Joint	305.176 ± 34.905	17.0562 ± 16.1422	9.3 ± 3.0
GBoost PerObj	332.859 ± 17.075	5.2970 ± 5.6903	12.7 ± 2.8
MO-LS	294.229 ± 27.401	16.1548 ± 12.7132	9.6 ± 2.5
MO-P3-GOMEA	280.290 ± 35.866	16.1406 ± 11.1833	5.5 ± 1.1

Wilcoxon signed-rank test (Joint vs PerObj, paired by seed):

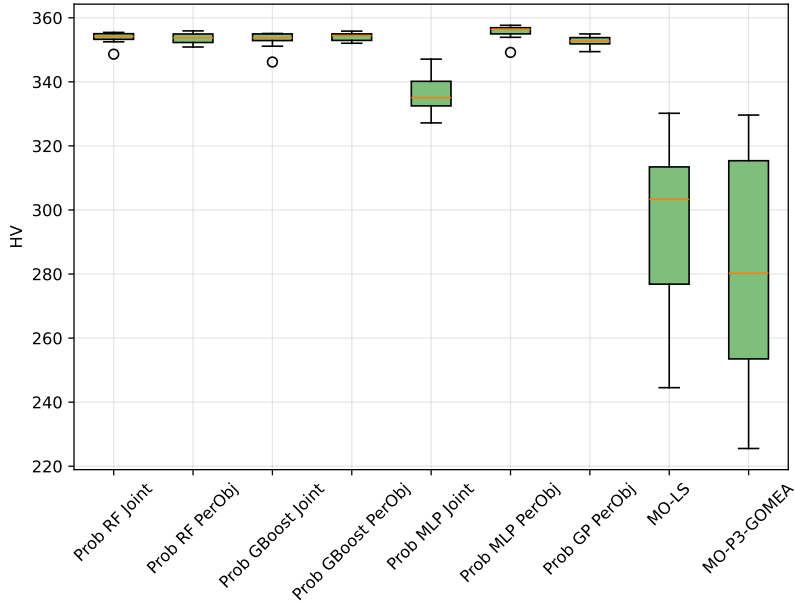
SVR HV: stat=23.00 p=0.6953
SVR IGD+: stat=14.00 p=0.1934
MLP HV: stat=17.00 p=0.3223
MLP IGD+: stat=16.00 p=0.2754
RF HV: stat=16.00 p=0.2754
RF IGD+: stat=14.00 p=0.1934
GBoost HV: stat=0.00 p=0.0020 **
GBoost IGD+: stat=0.00 p=0.0020 **

Probabilistic surrogates and baselines — Pareto overlay vs true oracle — NASTrainingEnergy

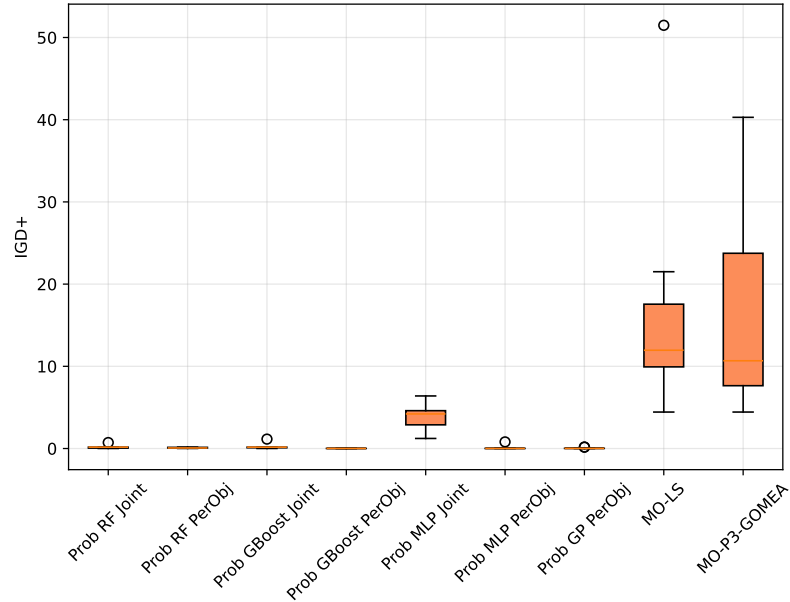


Probabilistic surrogates and baselines — performance across 10 seeds — NASTrainingEnergy

Hypervolume per algorithm



IGD+ per algorithm (vs true oracle)



Probabilistic surrogates and baselines — NASTrainingEnergy — true oracle Pareto size: 36

algorithm	HV	IGD+ (true oracle)	Pareto size
Prob RF Joint	353.661 ± 1.919	0.1767 ± 0.1962	16.7 ± 2.8
Prob RF PerObj	353.685 ± 1.678	0.0903 ± 0.0539	15.9 ± 1.9
Prob GBoost Joint	353.061 ± 2.589	0.2280 ± 0.3109	16.6 ± 1.6
Prob GBoost PerObj	354.007 ± 1.294	0.0099 ± 0.0012	14.9 ± 2.5
Prob MLP Joint	336.280 ± 6.236	3.7961 ± 1.4964	18.4 ± 2.0
Prob MLP PerObj	355.511 ± 2.379	0.0906 ± 0.2384	20.2 ± 3.1
Prob GP PerObj	352.728 ± 1.551	0.0466 ± 0.0632	12.7 ± 1.8
MO-LS	294.229 ± 27.401	16.1548 ± 12.7132	9.6 ± 2.5
MO-P3-GOMEA	280.290 ± 35.866	16.1406 ± 11.1833	5.5 ± 1.1

Wilcoxon signed-rank test (Joint vs PerObj, paired by seed):

RF HV: stat=21.00 p=0.5566

RF IGD+: stat=16.00 p=0.2754

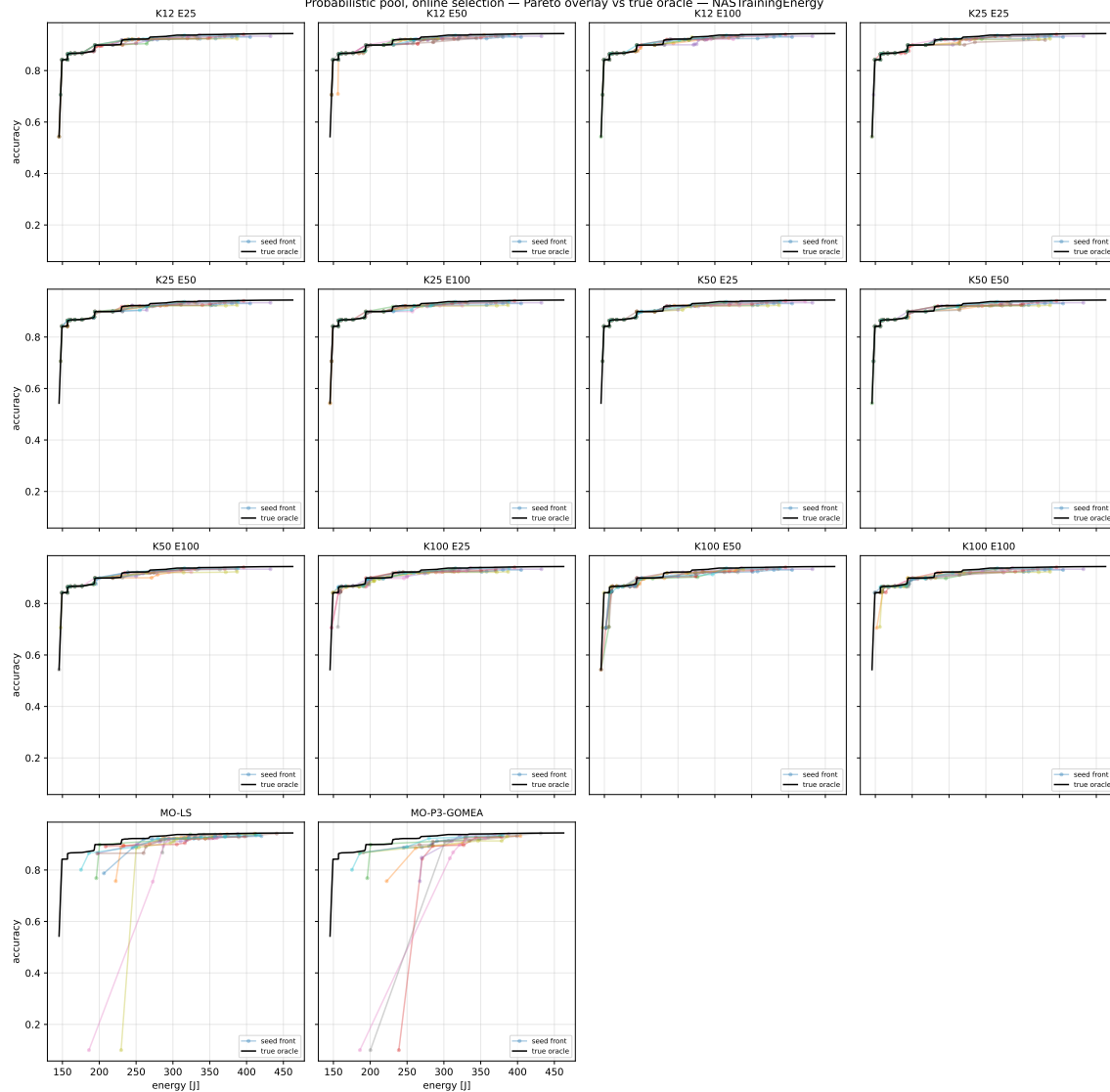
GBoost HV: stat=16.00 p=0.2754

GBoost IGD+: stat=1.00 p=0.0039 **

MLP HV: stat=0.00 p=0.0020 **

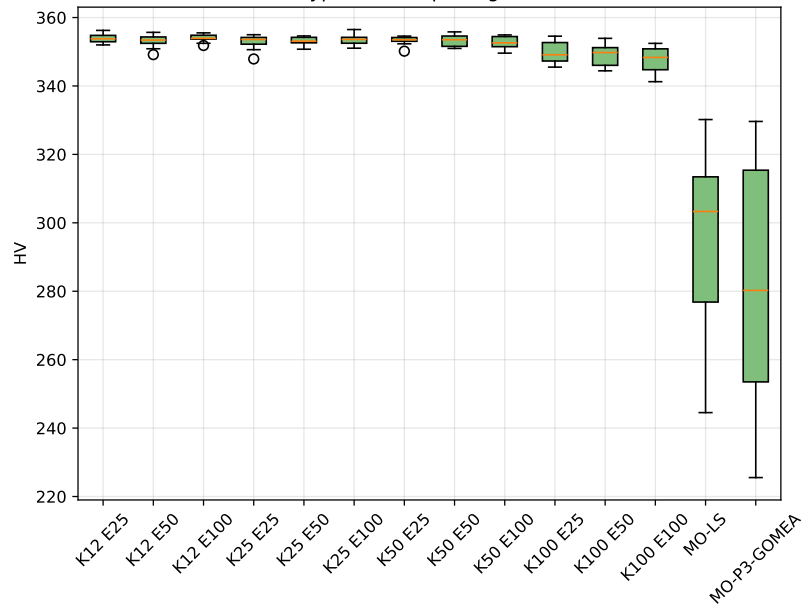
MLP IGD+: stat=0.00 p=0.0020 **

Probabilistic pool, online selection — Pareto overlay vs true oracle — NASTrainingEnergy

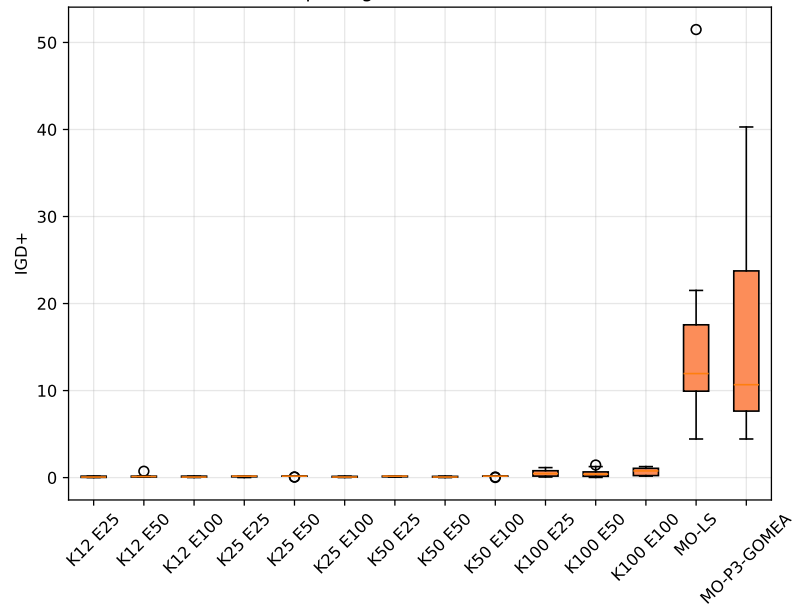


Probabilistic pool, online selection — performance across 10 seeds — NASTrainingEnergy

Hypervolume per algorithm



IGD+ per algorithm (vs true oracle)



Probabilistic pool, online selection — NASTrainingEnergy — true oracle Pareto size: 36

algorithm	HV	IGD+ (true oracle)	Pareto size
K12 E25	353.982 ± 1.307	0.0905 ± 0.0671	15.2 ± 3.6
K12 E50	353.126 ± 1.876	0.1737 ± 0.1923	17.3 ± 3.3
K12 E100	353.952 ± 1.080	0.1011 ± 0.0571	15.7 ± 2.0
K25 E25	352.876 ± 2.076	0.1279 ± 0.0651	16.1 ± 2.2
K25 E50	353.187 ± 1.139	0.1488 ± 0.0418	15.4 ± 2.1
K25 E100	353.468 ± 1.605	0.0858 ± 0.0580	15.8 ± 1.9
K50 E25	353.316 ± 1.241	0.1380 ± 0.0476	15.3 ± 2.1
K50 E50	353.331 ± 1.714	0.0913 ± 0.0545	16.1 ± 2.2
K50 E100	352.705 ± 1.772	0.1445 ± 0.0536	13.7 ± 1.6
K100 E25	349.859 ± 3.094	0.4938 ± 0.3782	12.8 ± 1.5
K100 E50	349.194 ± 3.285	0.5147 ± 0.4714	12.5 ± 2.5
K100 E100	347.676 ± 3.666	0.6920 ± 0.4181	11.5 ± 1.4
MO-LS	294.229 ± 27.401	16.1548 ± 12.7132	9.6 ± 2.5
MO-P3-GOMEA	280.290 ± 35.866	16.1406 ± 11.1833	5.5 ± 1.1